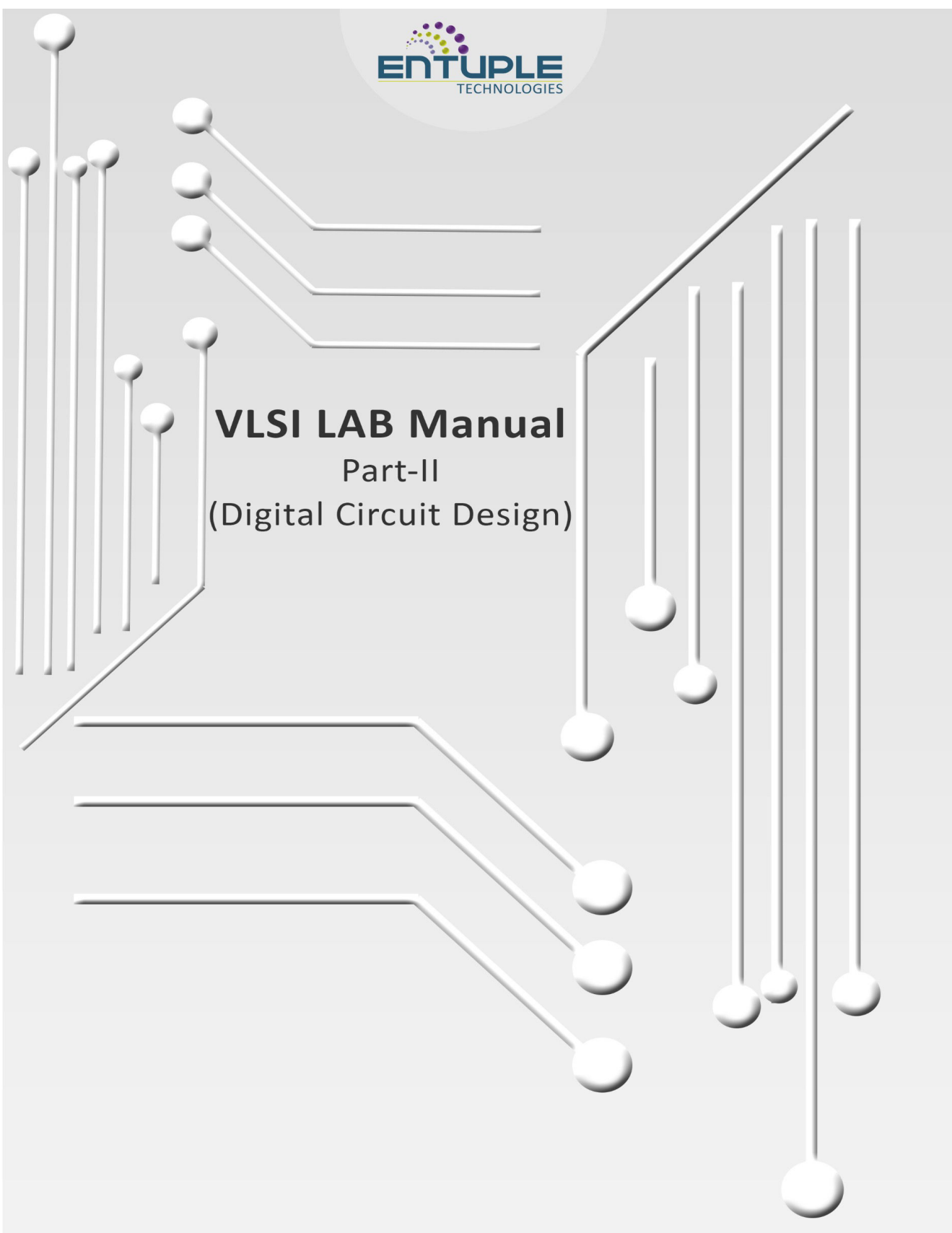




VLSI LAB Manual

(Anna University curriculum)

Developed by
Application Engineering Team
Entuple Technologies Pvt Ltd.

The background of the cover features a light gray background with a network of white, 3D-rendered lines and spheres. The lines are of varying lengths and angles, some connecting to spheres, creating a complex, circuit-like pattern that frames the central text.

VLSI LAB Manual

Part-II

(Digital Circuit Design)

Semi Custom IC Design Flow

Cadence Tool Suite

(Par-II Digital Circuit Design)

**INCISIVE
GENUS
INNOVUS**

**Developed By
Application Engineering Team
Entuple Technologies Pvt Ltd, Bengaluru**

Table of Contents

Lab 1: CMOS INVERTER	6
Module 1: Work Space Creation & Writing RTL code for Inverter.....	7
Module 2: Design Simulation using INCISIVE Simulator.....	11
Module 3: Synthesis Using GENUS Tool.....	24
 Lab 2: CMOS BASIC GATES & FLIP FLOPS.....	 32
Lab 2.1: CMOS BASIC GATES.....	32
Lab 2.1a: AND Gate	32
Lab 2.1b: OR Gate	35
Lab 2.1c: NAND Gate	37
Lab 2.1d: NOR Gate	39
Lab 2.1e: XOR Gate	41
 Lab 2.2: FLIP FLOPS	 43
Lab 2.2a: SR-Flip Flop	43
Lab 2.2b: JK-Flip Flop	46
Lab 2.2c: MS-JK Flip Flop	49
Lab 2.2d: T-Flip Flop	53
Lab 2.2e: D-Flip Flop	55
Module 1: Writing RTL Code for D-Flip Flop.....	56
Module 2: Design Simulation using INCISIVE Simulator....	57
Module 3: Synthesis Using GENUS Tool.....	59
 Lab 3: 4-BIT SYNCHRONOUS COUNTER	 63
Module 1: Work Space Creation & Writing RTL code for Counter.....	63
Module 2: Design Simulation using INCISIVE Simulator	65
Module 3: Synthesis Using GENUS Tool	69
Module 4: Pre-Layout Simulation/Post Synthesis simulation.....	74
Module 5: Physical Design /Automatic Layout Generation.....	79
Module 5.1 : Floor planning	87
Module 5.2 : Power Planning	89
Module 5.3 : Placement.....	99
Module 5.4 : Clock Tree Synthesis	108
Module 5.5 : Routing	117
Module 5.6 : Output File Generation (.spef, .sdf, .v & .gds)	126
Module 6: Post-Layout Simulation/Gate Level Simulation.....	131

Objective:

The main objective of this lab is to learn the Semi Custom IC Design Flow along with the usage of tools such as the INCISIVE Simulator, GENUS and INNOVUS. In this process, you will design and simulate designs like Inverter, Basic Gates & Flip flops and 4bit Synchronous Counter. You will start the lab by writing the Verilog and Test-bench code. Functionality can be verified by incisive simulation and further verilog code, sdc constraints and library will be taken for synthesis to generate the gate level netlist and then netlist will be taken for physical design process/Automatic Layout Generation with technology Node gpdk 90 / 45.

You will also able to check the pre layout and post layout Gate level simulation. In the process, one will learn to use Incisive, Genus and Innovus tool and able to generate reports with respect to PPA (Power Performance Area) for different lab experiments. Students will be able to compile, simulate and synthesize the verilog code. From this lab the students will be able to draw the schematic diagram and layout for the lab experiments and verifying their functionality.

Lab 1: CMOS INVERTER

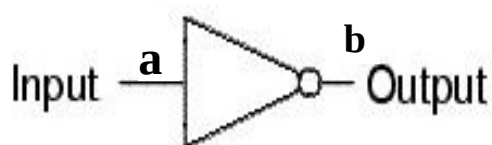
Aim:

Design and simulate a **CMOS Inverter** using Digital Flow.

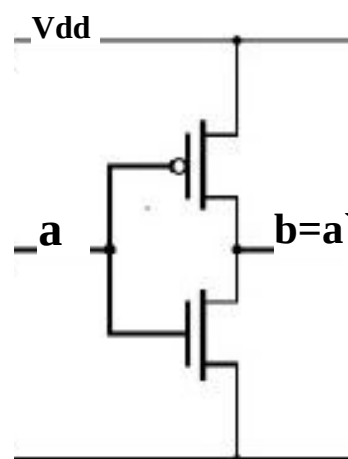
To write verilog code for an CMOS inverter circuit and its test bench for verification using incisive simulator, observe the waveform and synthesize the code with technological library with given constraints.

Theory:

The NOT gate or an CMOS inverter is an electronic circuit that produces an inverted version of the input at its output. It is also known as an inverter. If the input variable is “**a**”, the inverted output is known as “**NOT a**”. This is also shown as **a'**, or **a** with a **bar** over the top, as shown at the outputs.

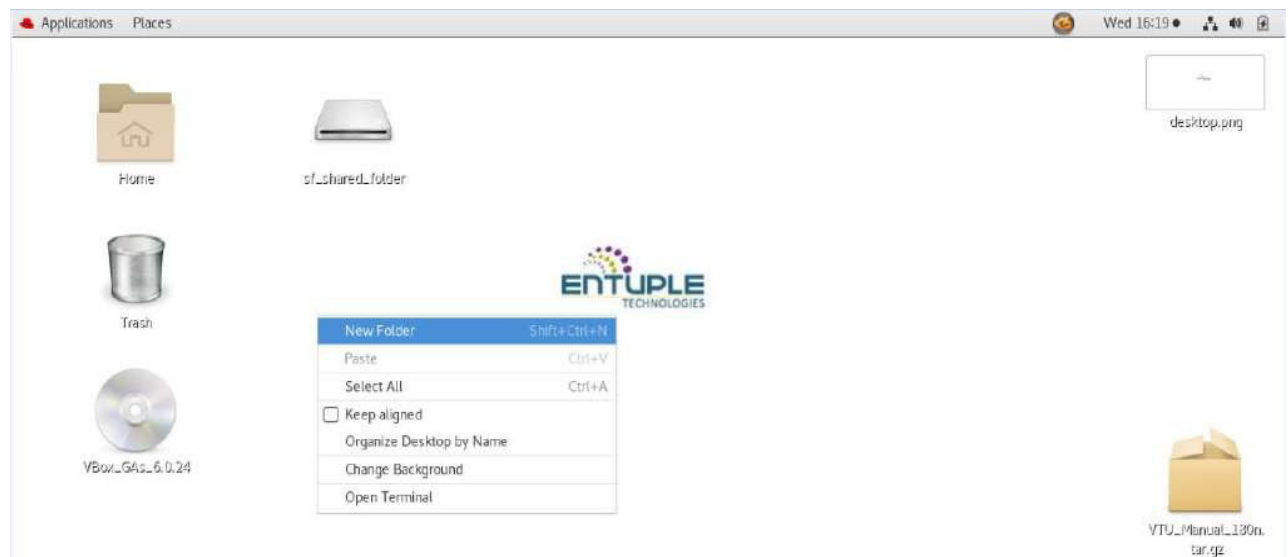


Input	Output
1	Vss
0	1

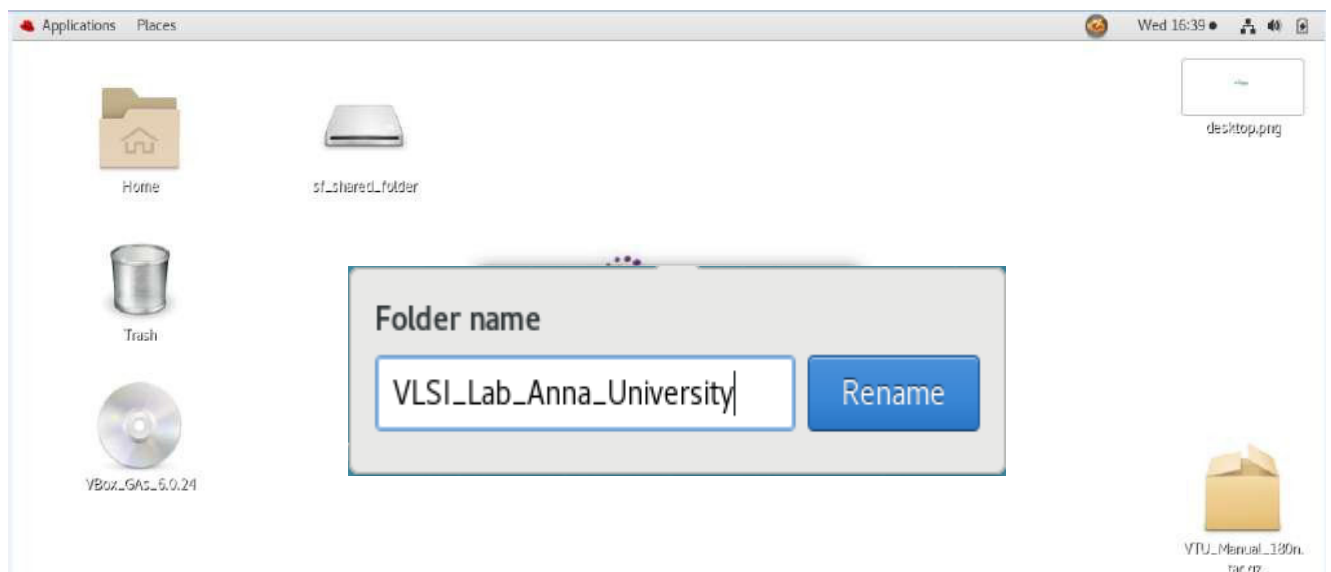


Module 1: Work Space Creation & Writing RTL Code for Inverter

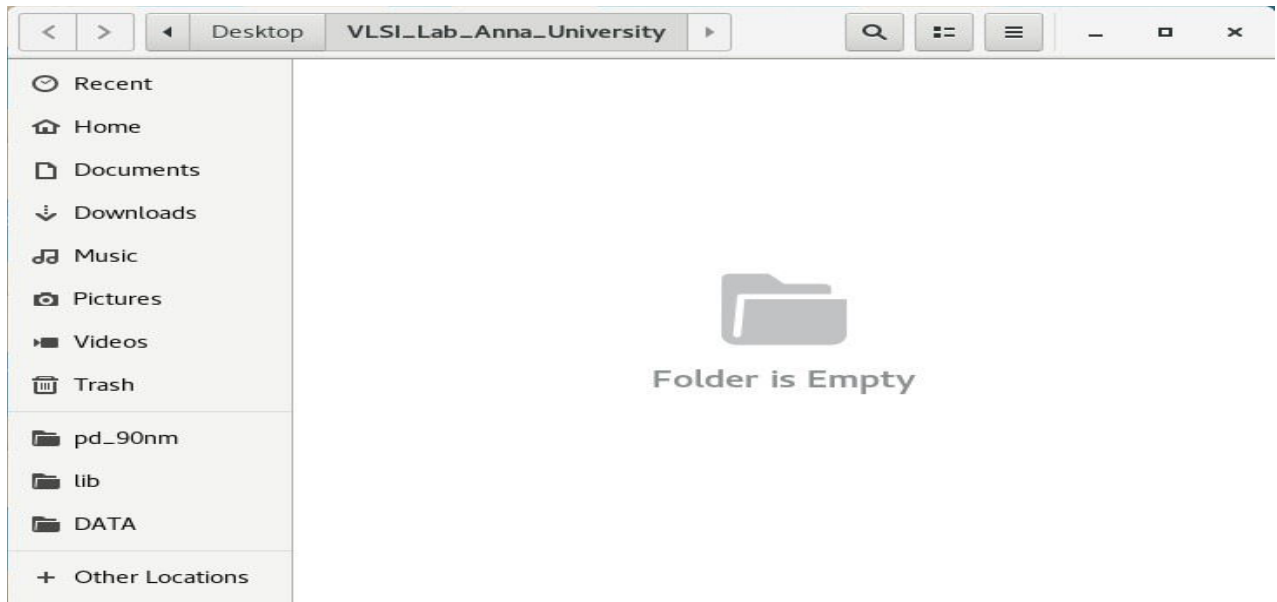
1. Make a **Right click** on the Desktop and select the option “**New Folder**” as shown in Figure below.



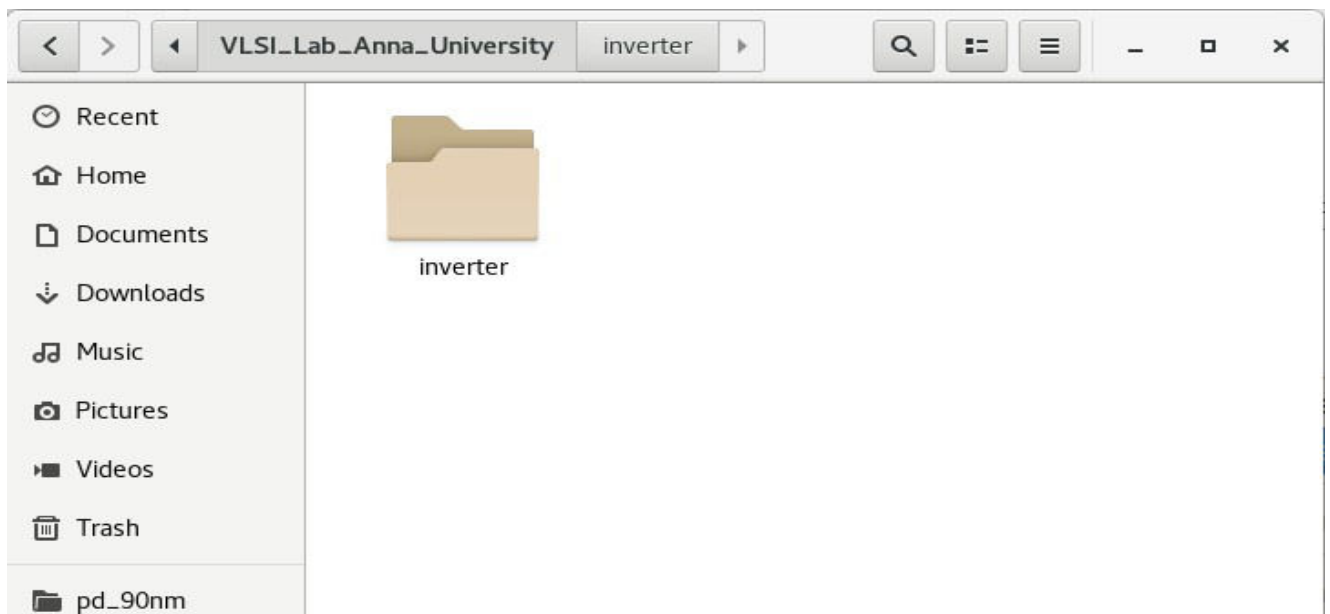
2. Name the folder (for example: VLSI_Lab_Anna_University) and click on “**Create**” as shown below.



3. Open the folder by a **double click** and the window can be seen as shown in below.



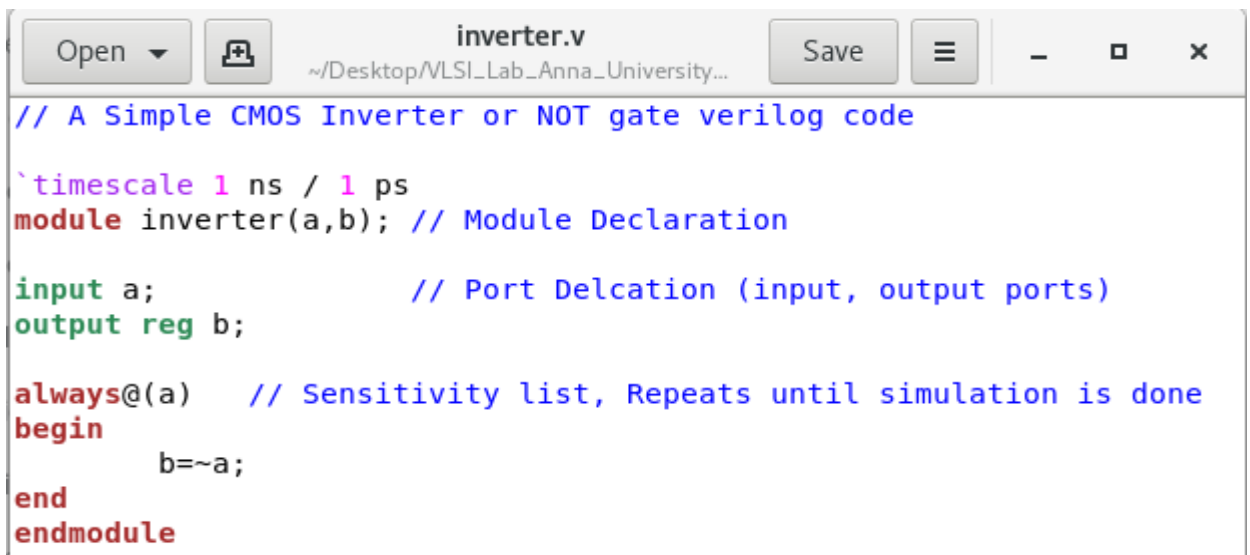
4. Create **sub directory** or folder with name inverter as shown below.



Note: Give folder name without any space

4. In order to create an RTL code for inverter, you can open a text editor and type verilog code or VHDL code.
5. Open the inverter directory and **Right click** and select “**Open terminal**” type the command given below

gedit inverter.v inverter_test.v

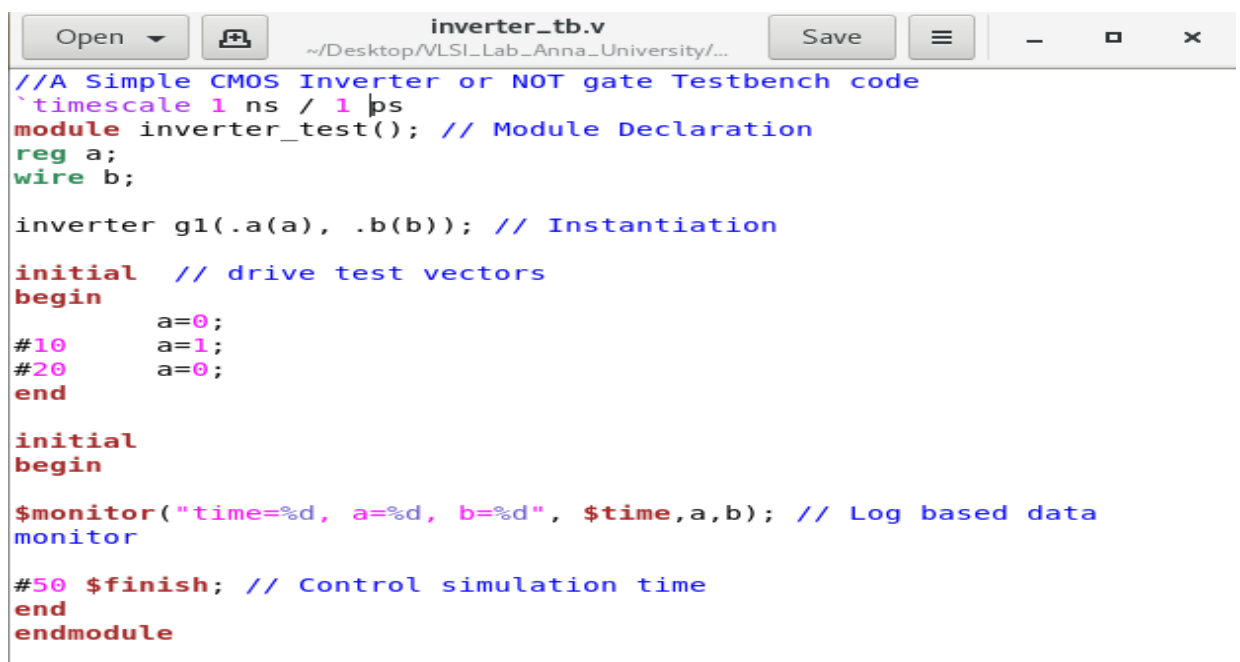


```
// A Simple CMOS Inverter or NOT gate verilog code

`timescale 1 ns / 1 ps
module inverter(a,b); // Module Declaration

input a;                // Port Declaration (input, output ports)
output reg b;

always@(a) // Sensitivity list, Repeats until simulation is done
begin
    b=~a;
end
endmodule
```



```
//A Simple CMOS Inverter or NOT gate Testbench code
`timescale 1 ns / 1 ps
module inverter_test(); // Module Declaration
reg a;
wire b;

inverter g1(.a(a), .b(b)); // Instantiation

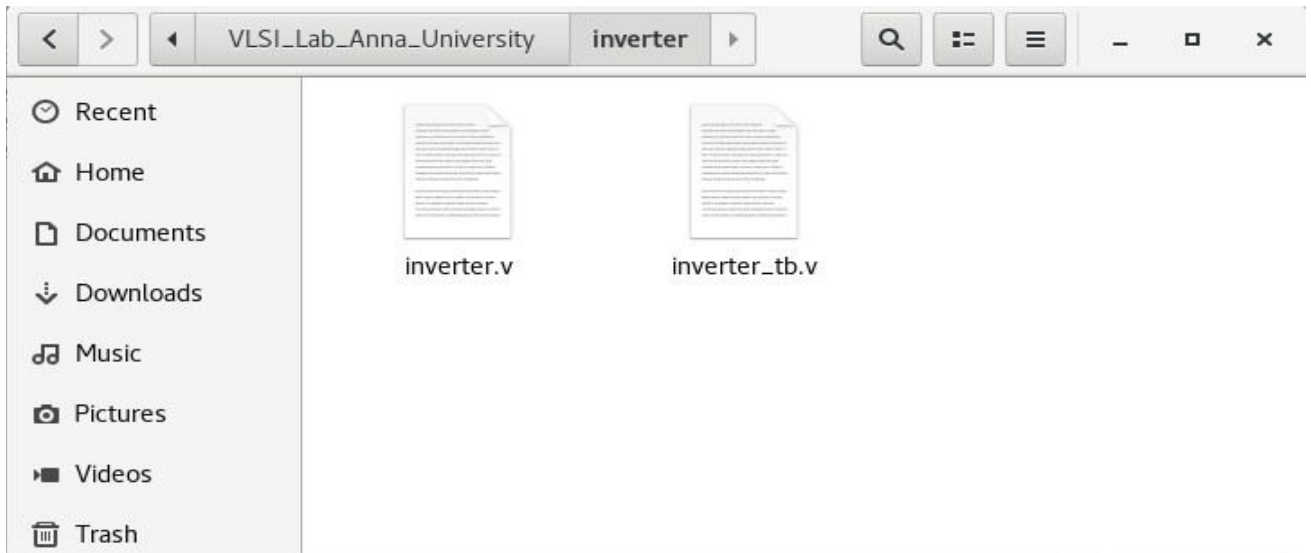
initial // drive test vectors
begin
    a=0;
    #10 a=1;
    #20 a=0;
end

initial
begin

$monitor("time=%d, a=%d, b=%d", $time,a,b); // Log based data monitor

#50 $finish; // Control simulation time
end
endmodule
```

6. Once after writing the verilog and test-bench code for inverter you can **save** the files and **close** it.



Steps to Invoke Tools:

Invoke the cadence environment by type the below commands

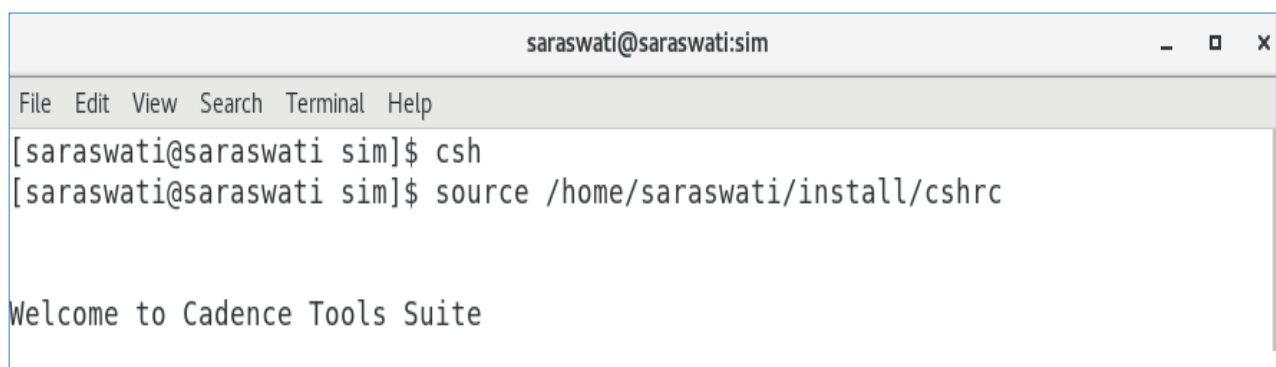
7. Before invoking any tool, invoke C shell by typing 'csh' in terminal.

#csh

8. Source the cshrc file by typing 'source <cshrc file>' e.g.: - source cshrc

#Source /home/install/cshrc (mention the path of the tools)

9. A welcome string "**Welcome to Cadence Tool Suite**" appears indicating terminal ready to invoke Cadence Tools available for you.

A screenshot of a terminal window titled 'saraswati@saraswati:sim'. The terminal shows the following commands and output:

```
[saraswati@saraswati sim]$ csh
[saraswati@saraswati sim]$ source /home/saraswati/install/cshrc

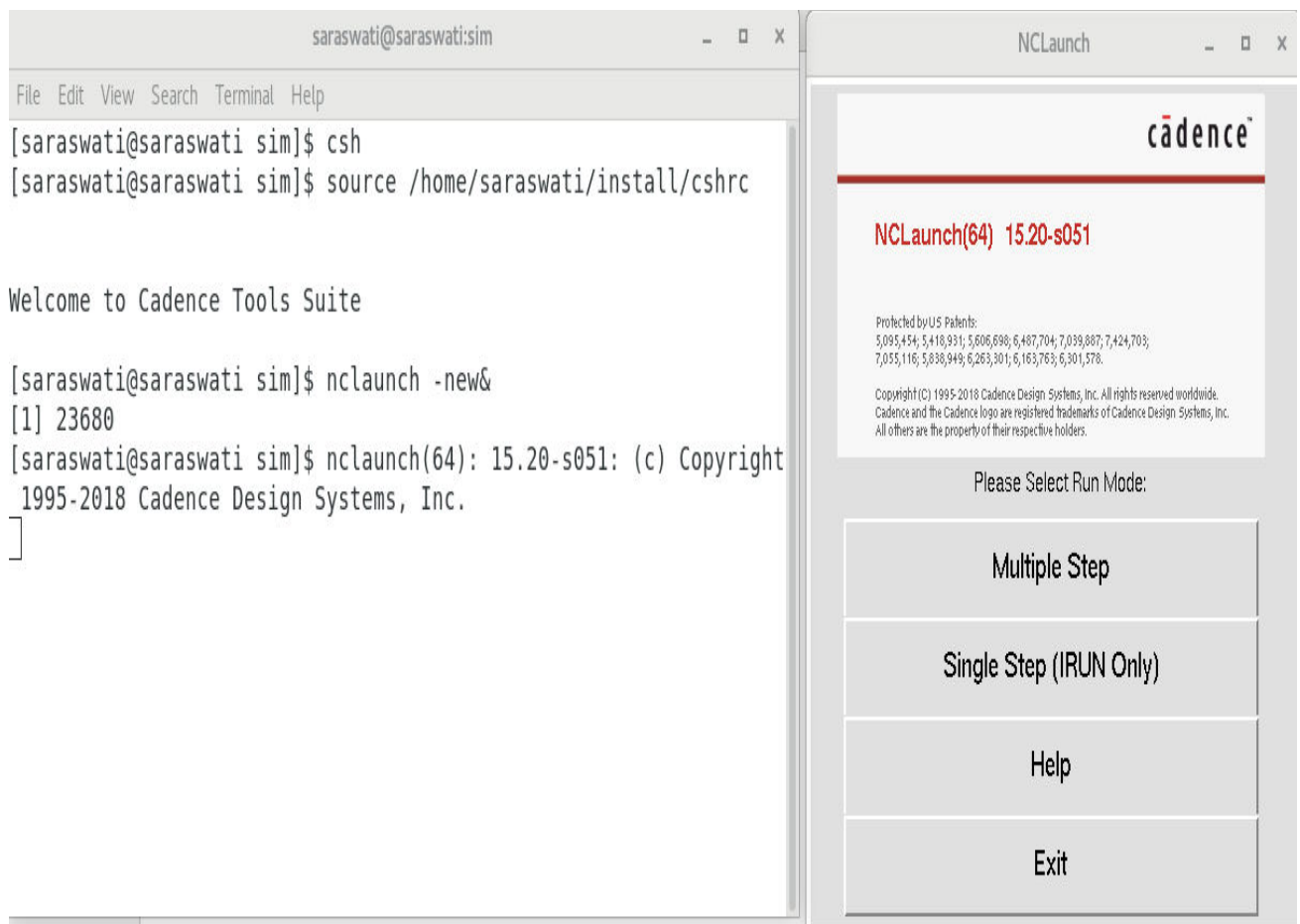
Welcome to Cadence Tools Suite
```

Module 2: Design Simulation Using the INCISIVE Simulator

Objective: To simulate a simple counter design with a testbench using the Incisive Enterprise Simulator or IES tool.

10. To perform functional simulation, “INCISIVE” simulator tool is to be used.

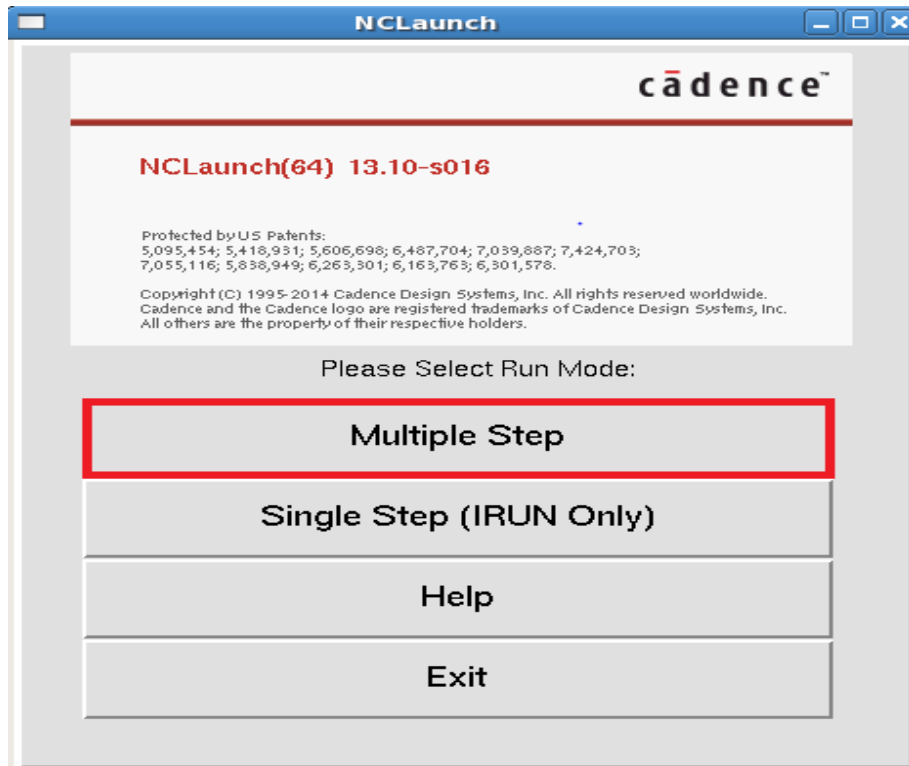
11. In your terminal, type the command “**nclaunch -new&**” to open the tool as shown below.



Note: The ‘**-new**’ switch is used only for the first time the design is being run. For the next time on wards, the command to be used could be ‘nclaunch’ only.

12. Select “**Multiple Step**” option.

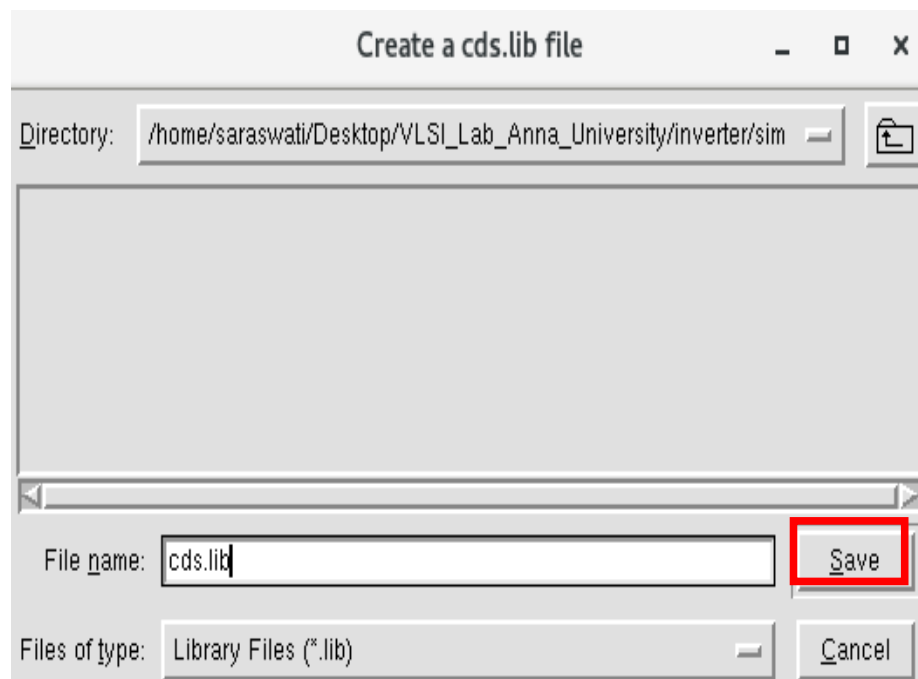
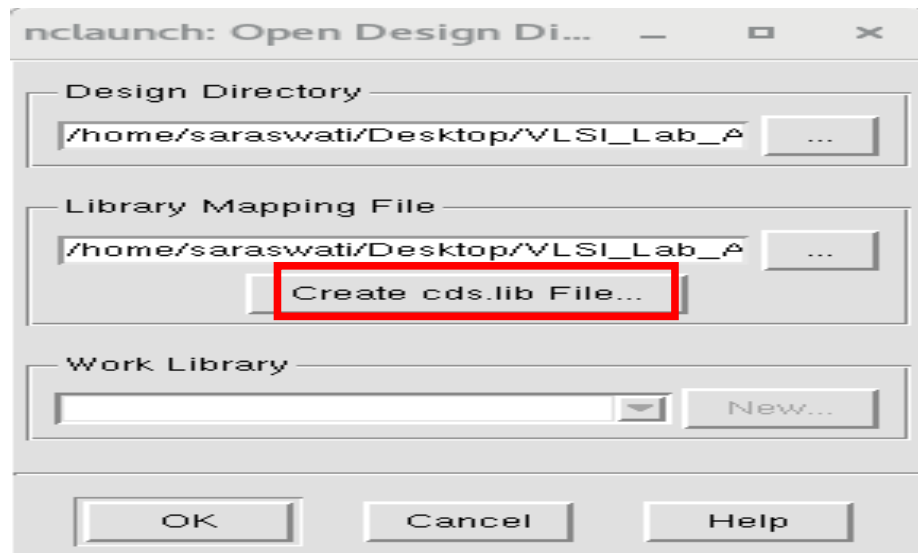
It will invoke the nclaunch window for functional simulation we can compile, elaborate and simulate it using Multistep



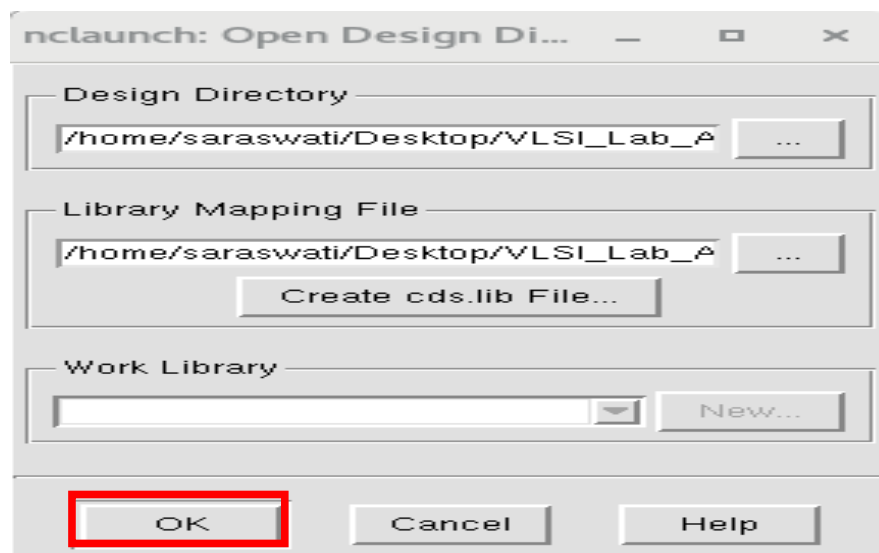
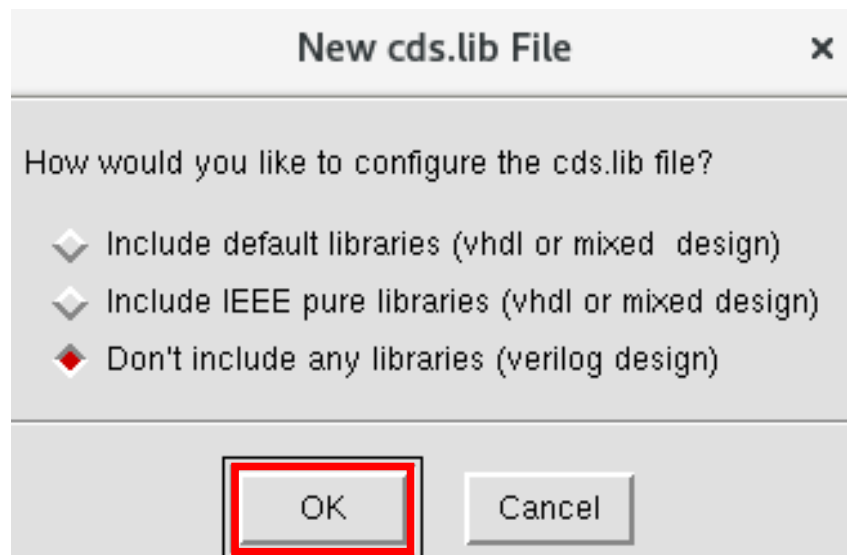
we can simulate a design using the Incisive simulator. For that we have to Create the **cds.lib** and **hdl.var** files for to Compile, elaborate and simulate the design and test bench

- **cds.lib:** It maps logical library names to physical directory paths. It also defines the path to the work library and to any reference libraries that the design required.
- **hdl.var:** Defines the work library. It may also define variables that configure your design environment, control the operation of incisive simulation tool and specify the location of supported files and invocation scripts.

13. Click the **Create cds.lib file** and save it.

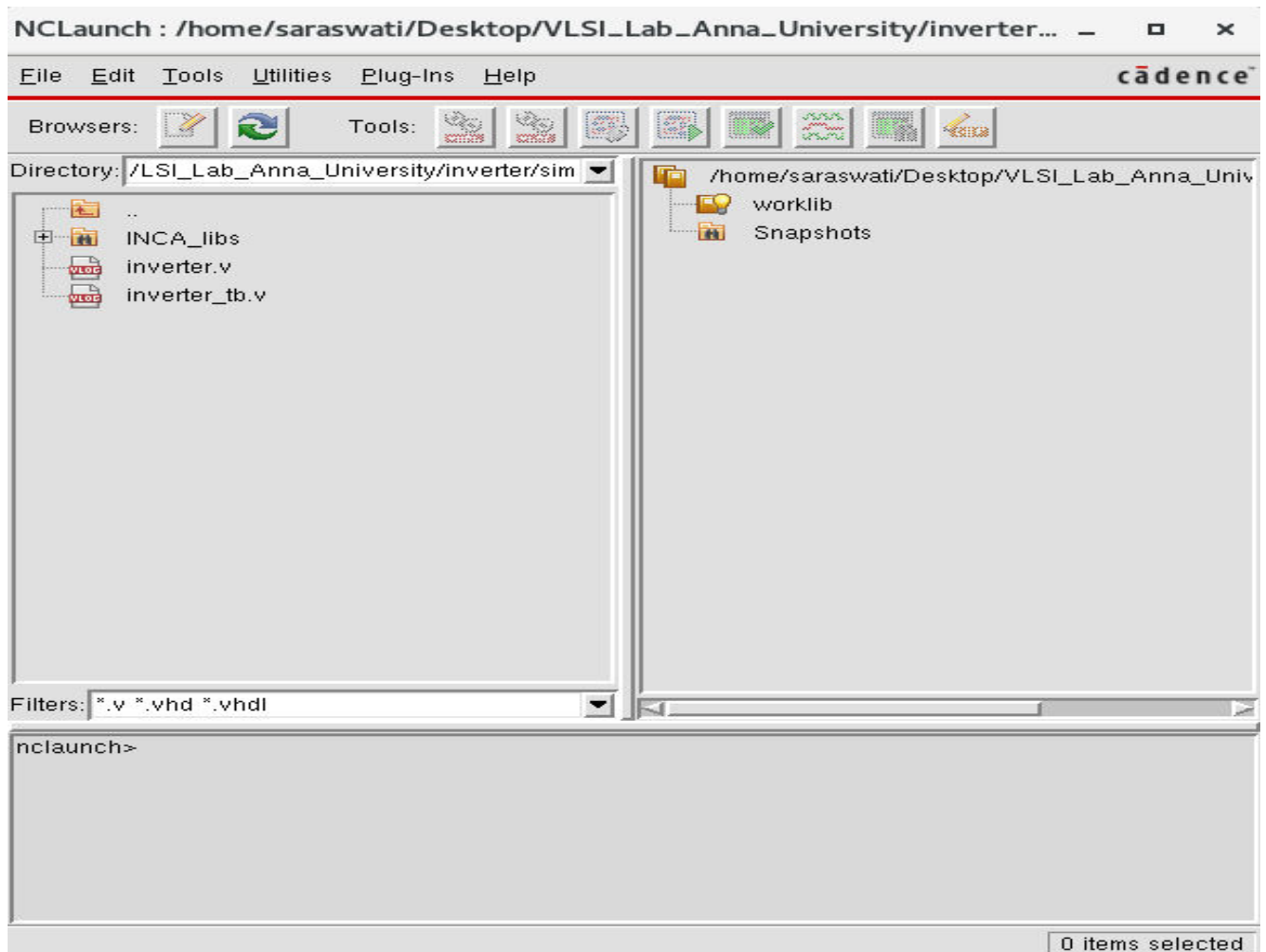


14. Choose any of the option listed below. Based on the libraries available and the type of RTL code written in verilog so, you need to choose “**Don’t include any libraries (verilog design)**” as shown below.



You can see the below window after giving “OK”

15. A new window pop-up as shown below that is a **NCLaunch** window.



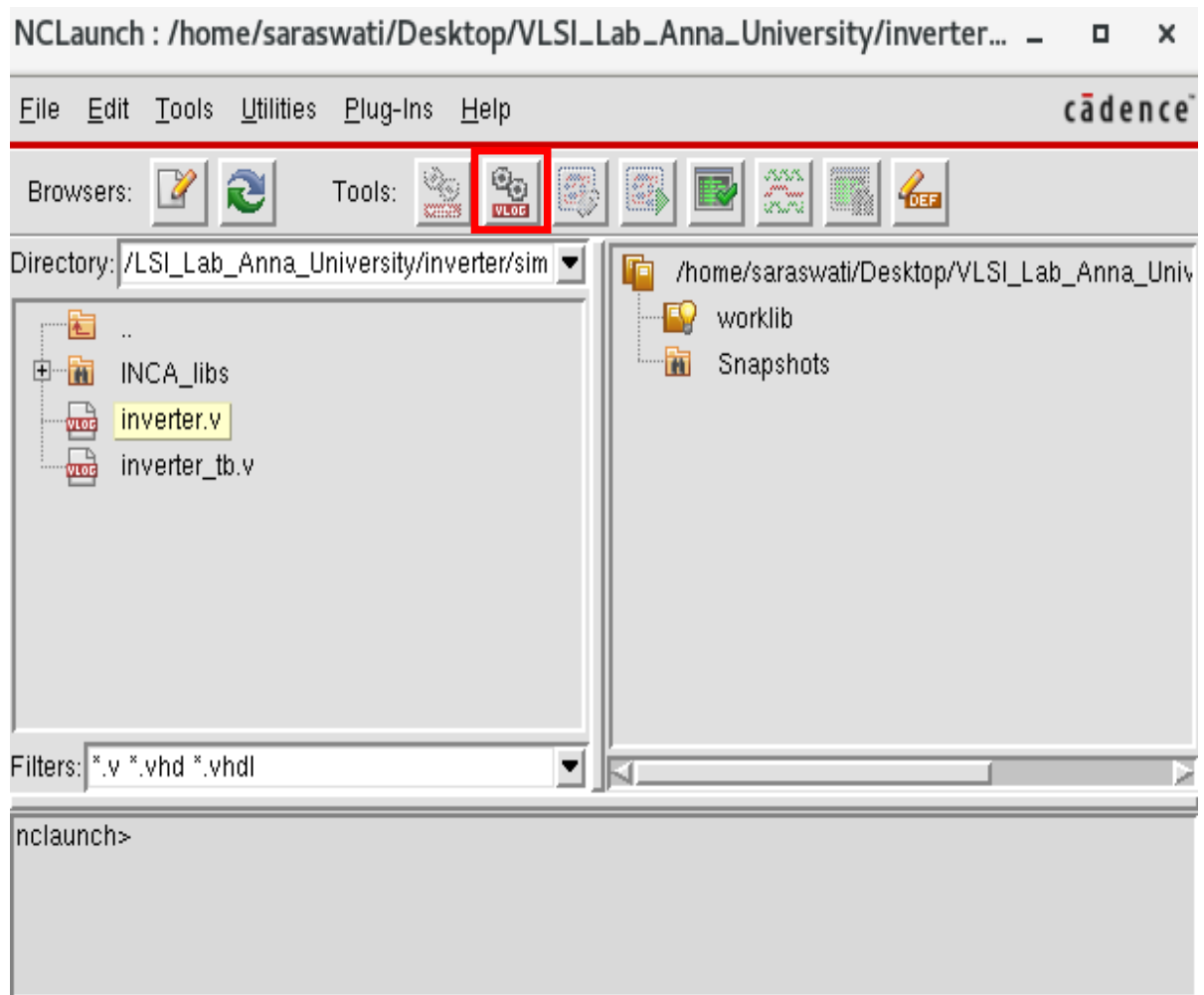
Left side you can see the **HDL files** and right side of the window has **worklib** and **snapshots** directories listed.

Worklib is the directory where all the compiled codes are stored while **Snapshot** will have output of elaboration which in turn goes for simulation

Functional simulation using cadence runs in 3 stages:

- i. Compilation
- ii. Elaboration
- iii. Simulation

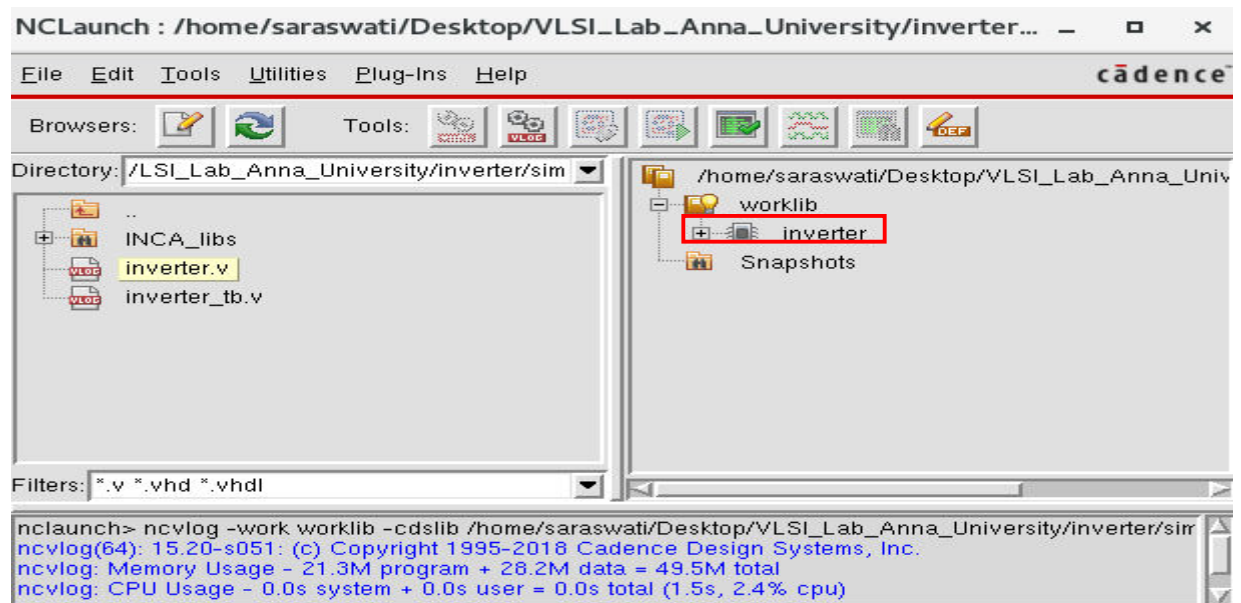
Compilation: (Checks syntax and semantics errors)



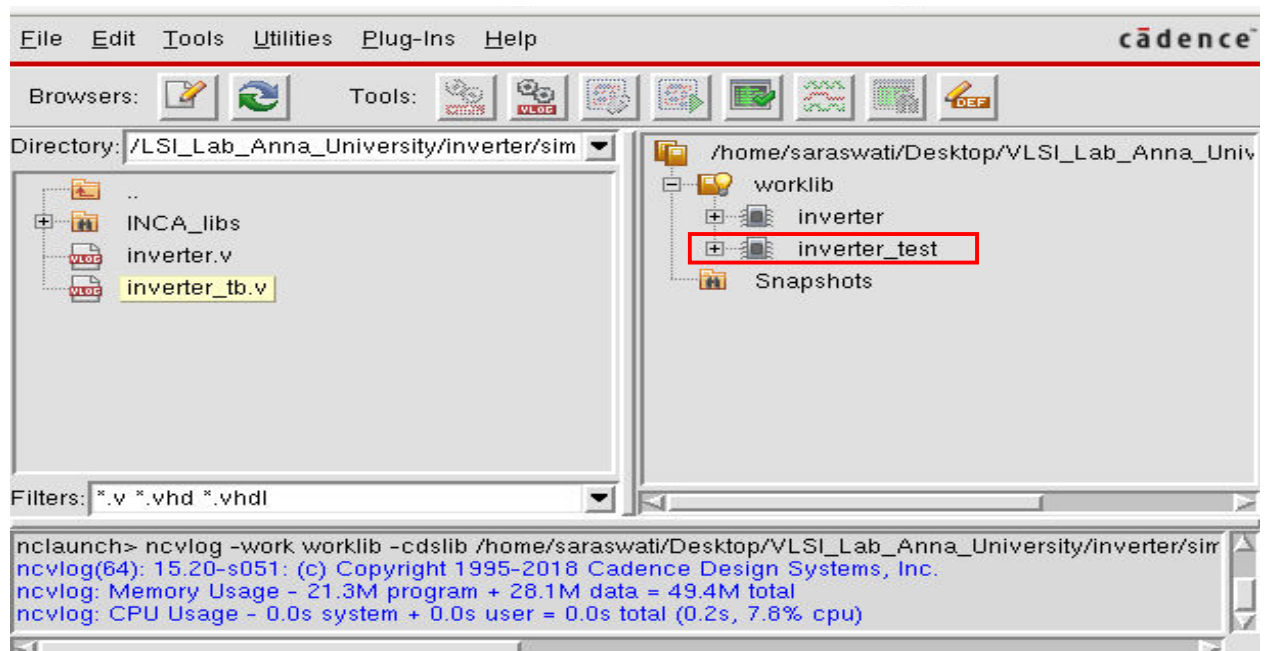
16. Left side select the file (inverter.v) and in **Tools : launch verilog compiler with current selection** will get enable.

Click it to compile the code

17. After compilation it will come under **worklib** you can see in right side window.



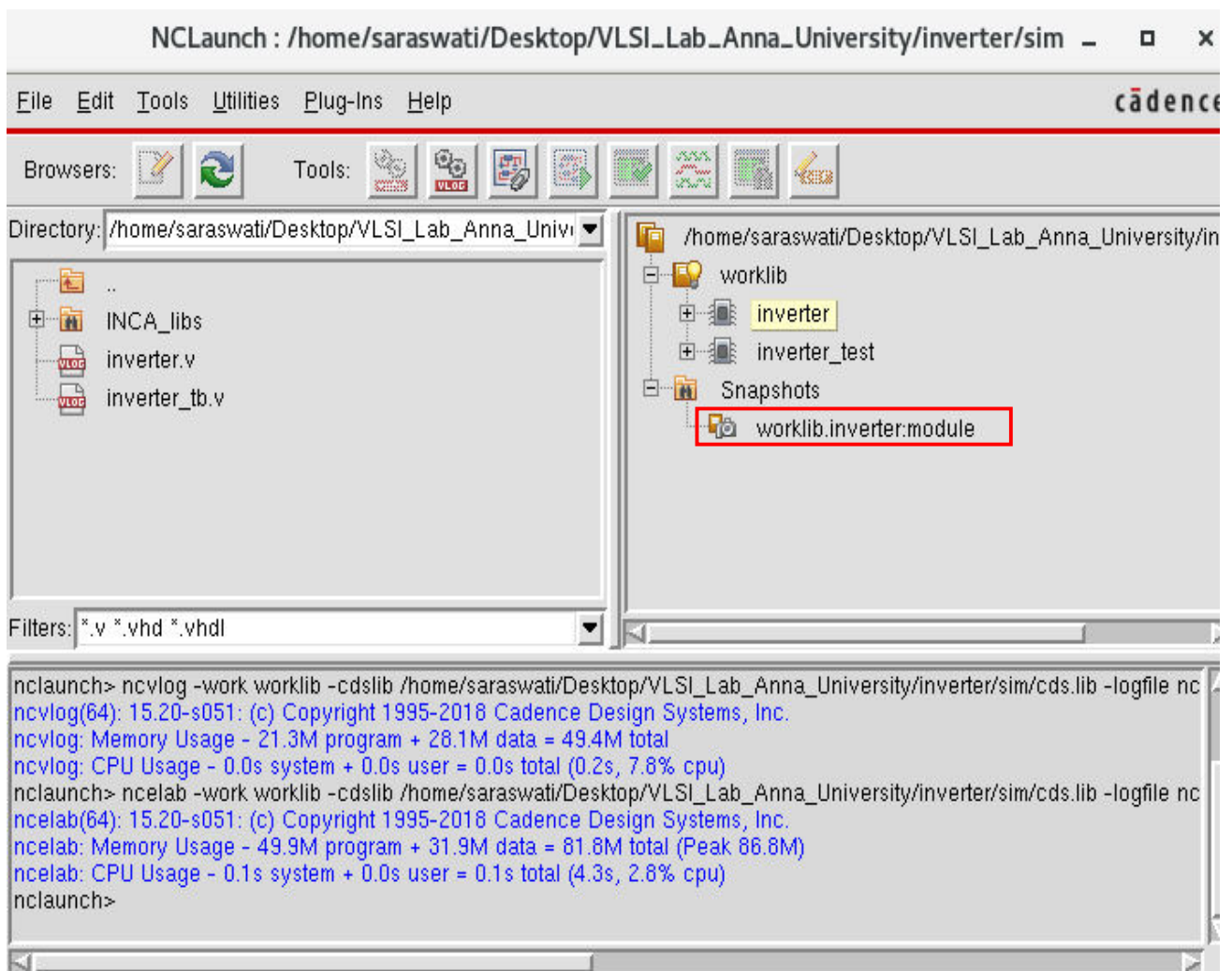
18. Select the test bench and compile it. It will come under **worklib**. Under Worklib you can see the module and testbench. Next is to elaborate the design.

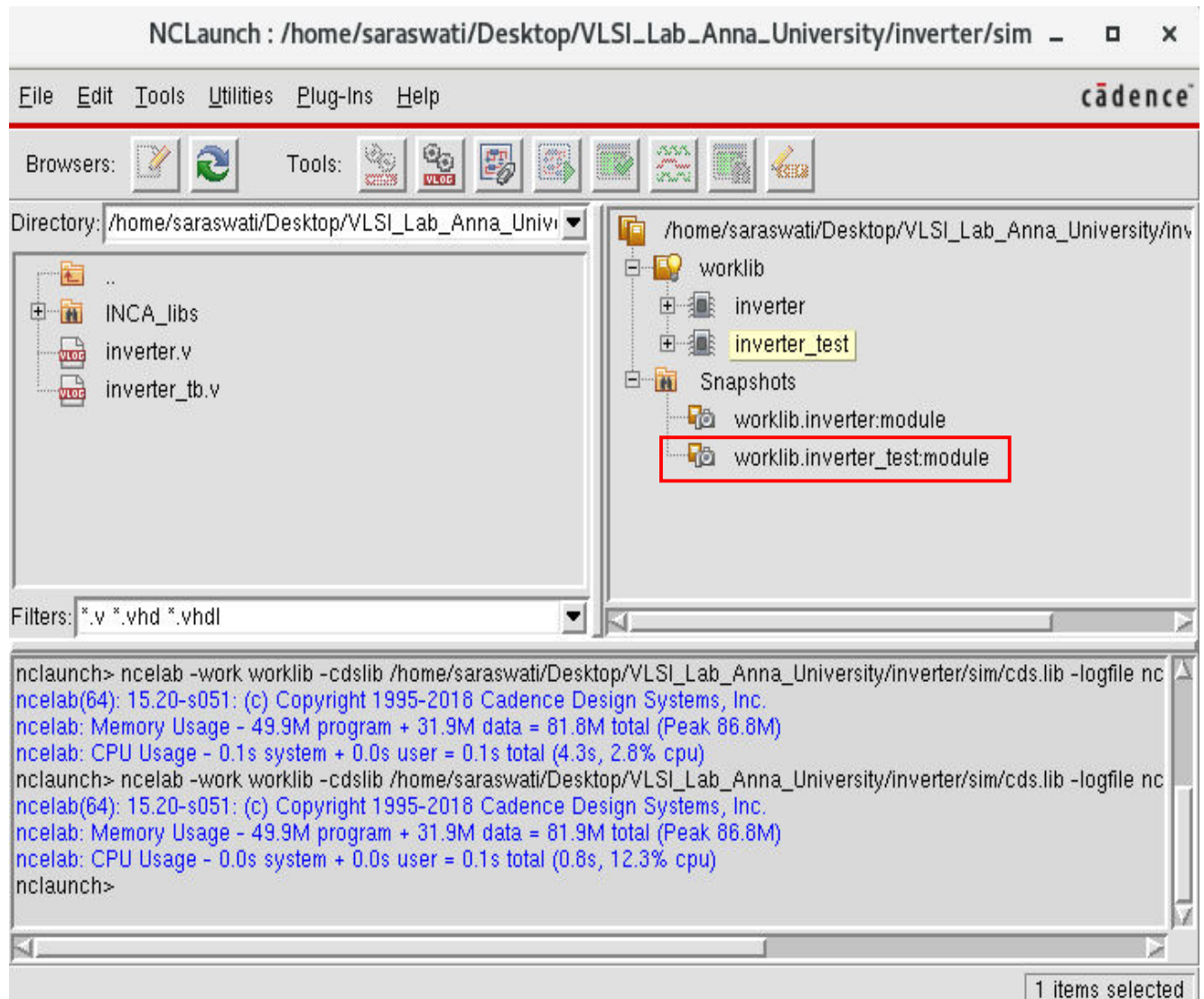


Elaboration:

19. select the file under worklib and in **Tools : launch elaborator with current selection** will get enable. select the **elaborator** to elaborate the design.

Choose the module and test bench and elaborate the design.

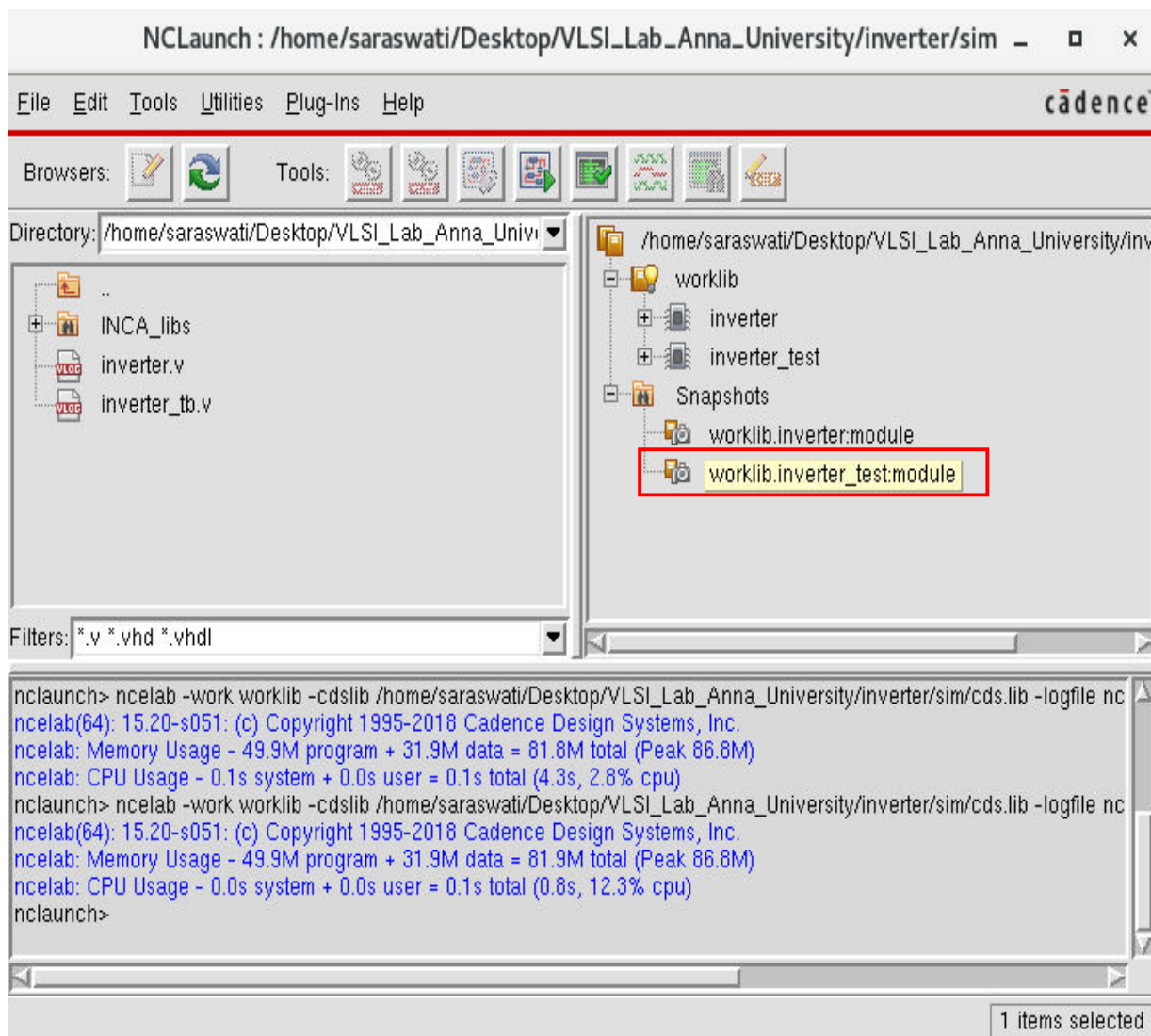




After elaboration the file will come under snapshot. Select the test bench and elaborate it.

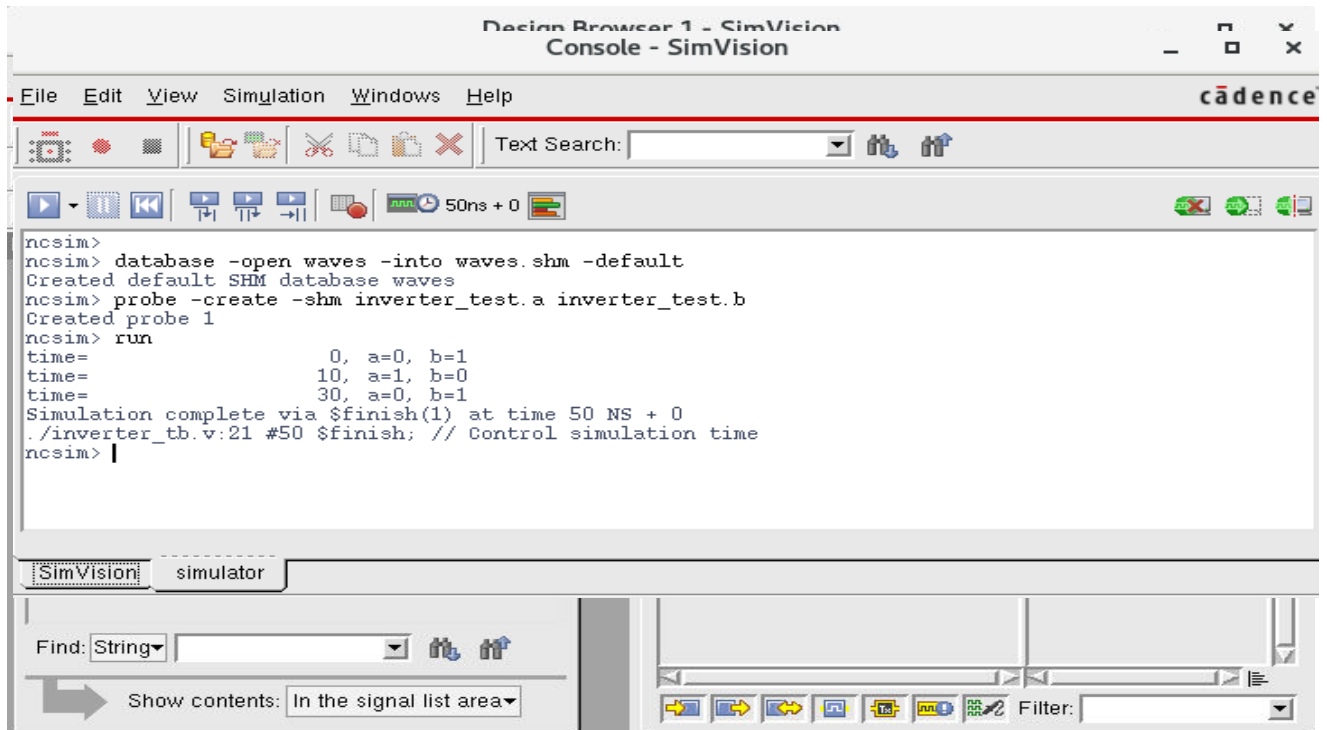
Simulation:

20. Select the test-bench file under snapshot and in **Tools: Launch simulator with current selection** will get enable.

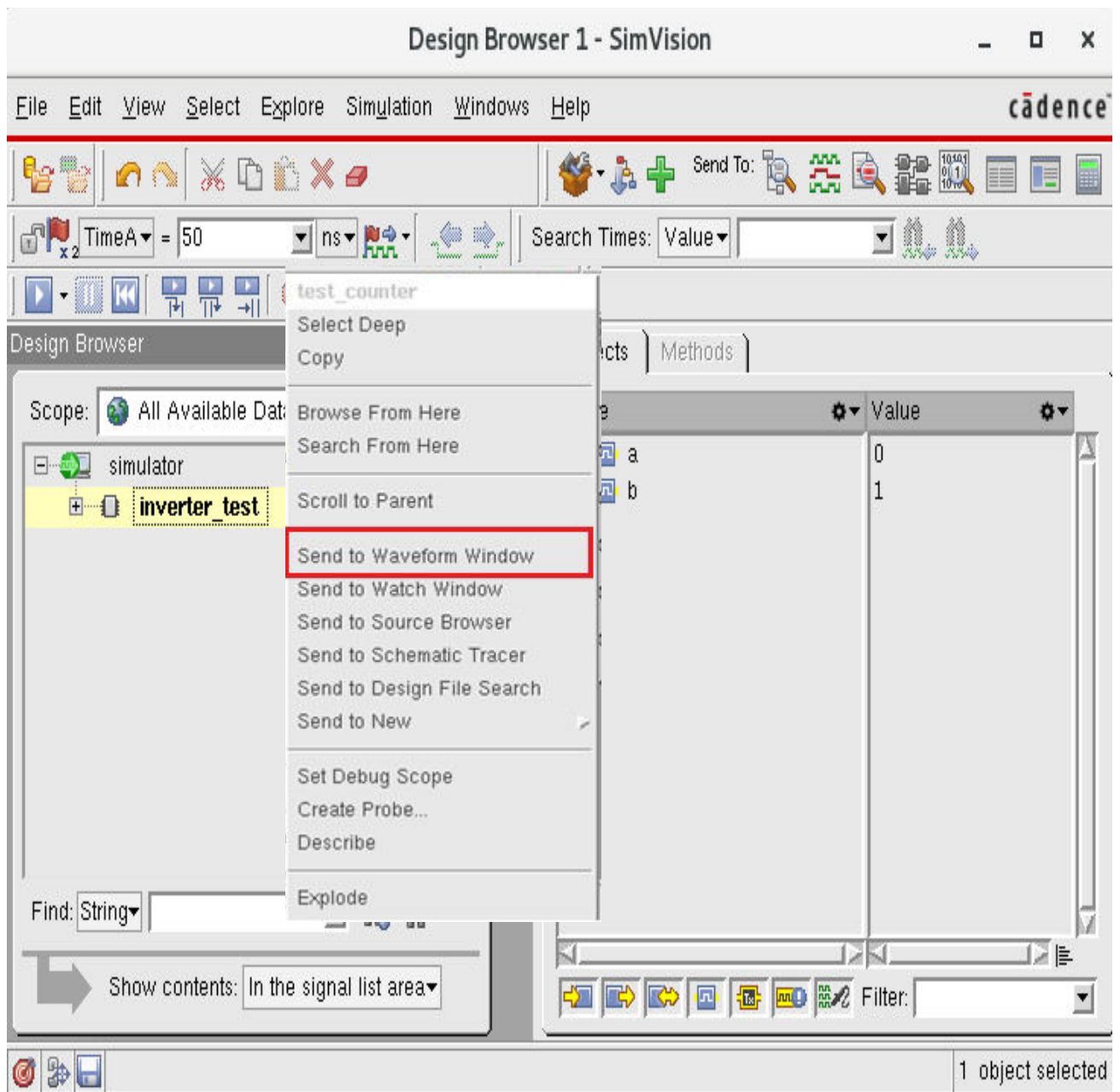


Select simulator to simulate the design. After simulation you will get the two windows like below image.

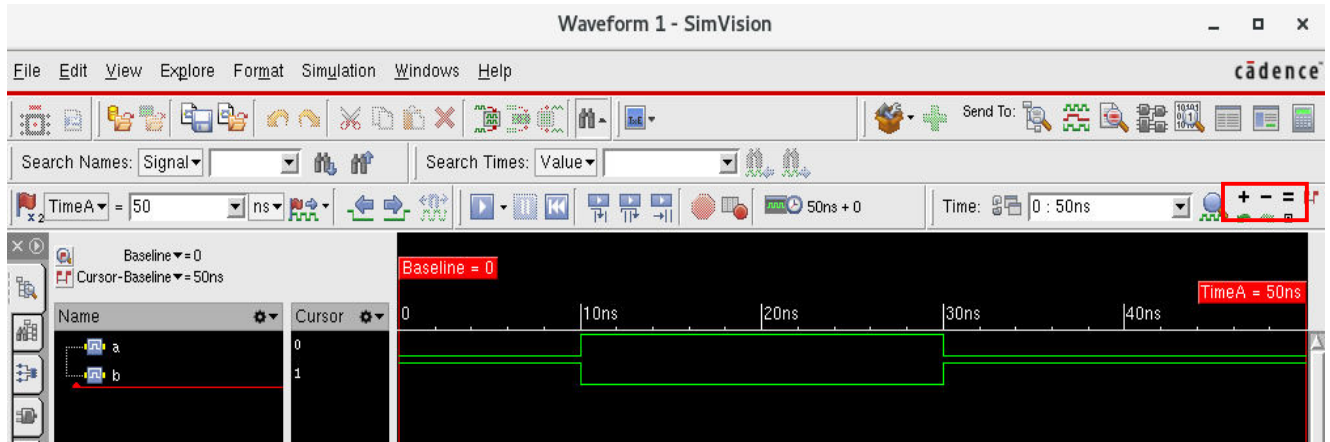
21. You will get the two windows **Design Browser 1 - SimVision** and **Console - SimVision**. In design browser you can see the test bench in left side window.



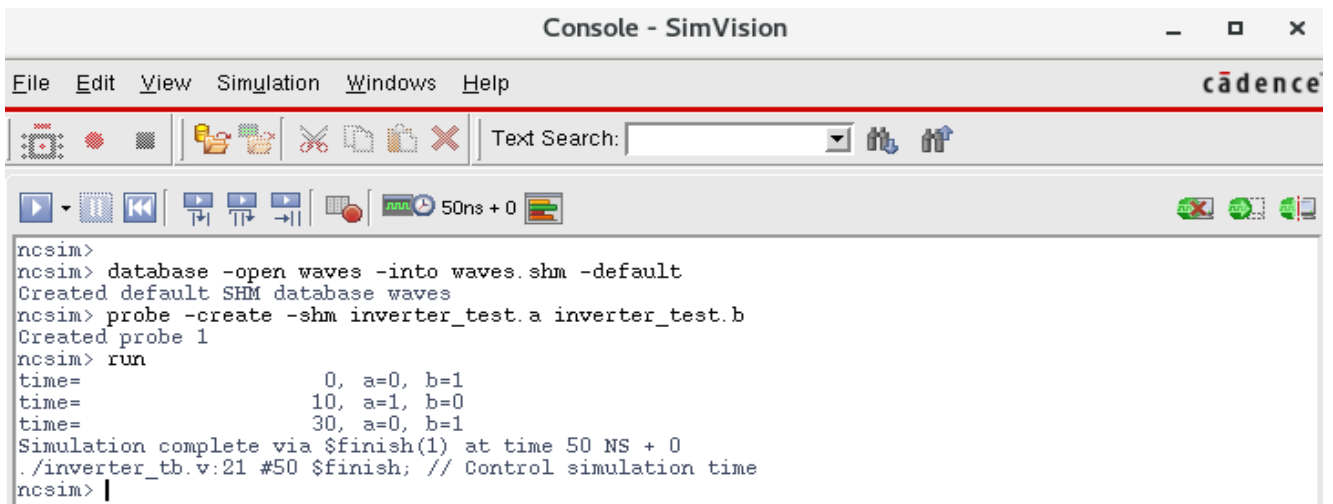
22. Select the test bench for the inverter and **Right click** it. **Select the send to waveform window** or select the **waveform icon**



23. You can see the waveform window after that click the **Run tool** to see the functional simulation for the inverter.



The equivalent command terminal output can be observed in the Simvision console window and also in the nclaunch console terminal.



```
ncsim>
ncsim> database -open waves -into waves.shm -default
Created default SHM database waves
ncsim> probe -create -shm inverter_test.a inverter_test.b
Created probe 1
ncsim> run
time=          0, a=0, b=1
time=         10, a=1, b=0
time=         30, a=0, b=1
Simulation complete via $finish(1) at time 50 NS + 0
./inverter_tb.v:21 #50 $finish; // Control simulation time
ncsim> |
```

Module 3: Synthesis Using GENUS Tool

Synthesis is the process of converting the **RTL Coding** into optimized **Gate Level Netlist**. The Tool used for doing the synthesis is **GENUS**. The tcl file (Tool Command Language) is used for scripting.

Inputs for Synthesis:

1. RTL Code (.v or .vhdl)
2. Chip Level SDC (System Design Constraints)
3. Liberty Files (.lib)

Expected Outputs of Synthesis:

1. Gate Level Netlist
2. Block Level Netlist
3. Timing, Area and Power Reports

Synthesis runs in following 3 stage process:

- i. **Translation** – Converts RTL code to boolean expression
- ii. **Mapping** – Boolean expression is mapped to logic /standard cells available from libraries
- iii. **Optimization** - Tool tries to reduce cell count without affecting the functionality

To run the synthesis, the following script can be used. Inside the `genus_script` file we have to mention the commands as shown below.

Script for synthesis (genus_script.tcl):

```

genus_script.tcl
~/Desktop/VLSI_Lab_Anna_University/inverter/synthesis

# Setting library and RTL paths
set_db / .init_lib_search_path {/home/saraswati/install/FOUNDRY/digital/90nm/dig/lib}
set_db / .init_hdl_search_path {.}

# Read Lib, RTL and SDC files
set_db / .library "slow.lib"
set DESIGN inverter
read_hdl "inverter.v"
elaborate $DESIGN
check_design -unresolved
read_sdc constraints_top.sdc

# Setting effort medium
set_db syn_generic_effort medium
set_db syn_map_effort medium
set_db syn_opt_effort medium
syn_generic
syn_map
syn_opt

write_hdl > output/inverter_netlist.v
write_sdc > output/inverter_sdc.sdc
# PPA Reports
report_power > output/inverter_power.rpt
report_area > output/inverter_area.rpt
report_timing -unconstrained > output/inverter_timing.rpt
report_gates > output/inverter_gatecount.rpt|
gui_show

Tcl Tab Width: 8 Ln 27, Col 45 INS

```

Chip Level SDC (constraints_top.sdc):

```

constraints_top.sdc
~/Desktop/VLSI_Lab_Anna_...

set_input_delay -max 1.0 [get_ports "a"]
set_output_delay -max 1.0 [get_ports "b"]

```

Script File Explanation:

1. Give the path of the library w.r.t to the directory you are using the command:
set_db / .init_lib_search_path {/home/install/FOUNDRY/digital/90nm/dig/lib}
2. Give the path of the RTL files w.r.t to the directory you are using the command:
set_db / .init_hdl_search_path {.}
3. Read the library file from the directory specified in giving the path for the library files in First line using the command:
set_db / .library "slow.lib"
4. Set the top module name for the design you are running using the command:
set DESIGN inverter
5. Read the RTL files from the directory specified in the second line. The RTL files are in the directory name : **read_hdl "inverter.v"**
6. Now Elaborate the design using the command:
elaborate \$DESGIN
7. To check the any unresolved references with the RTL code you can check by using the command:
check_design -unresolved
8. Constraint File : Not Mandatory, If you are having constraint file then you can read the constraint file using command:
read_sdc constraints_top.sdc
9. Set the effort level to (low, medium & high) the all the 3-stages of synthesis based on the user specification by using the commands:
set_db syn_generic_effort medium
set_db syn_map_effort medium
set_db syn_opt_effort medium
10. Synthesize the circuit using the commands:
syn_generic
syn_map
syn_opt

OUTPUT Files Generated from Synthesis Process

11. Write the Gate level netlist (GLN) or hdl code in terms of library components for the synthesized circuit using the command:

write_hdl > inverter_netlist.v

12. Similarly write the constraint file using:

write_sdc > inverter_const.sdc

REPORTS Files Generated from Synthesis Process

13. Check Power dissipation using:

report_power > inverter_pwr.rpt

14. Check area using:

report_area > inverter_area.rpt

15. Timing could be check using command:

report_timing -unconstrained (only for combinational circuit)

16. Similarly for Gates:

report_gates > inverter_gates.rpt

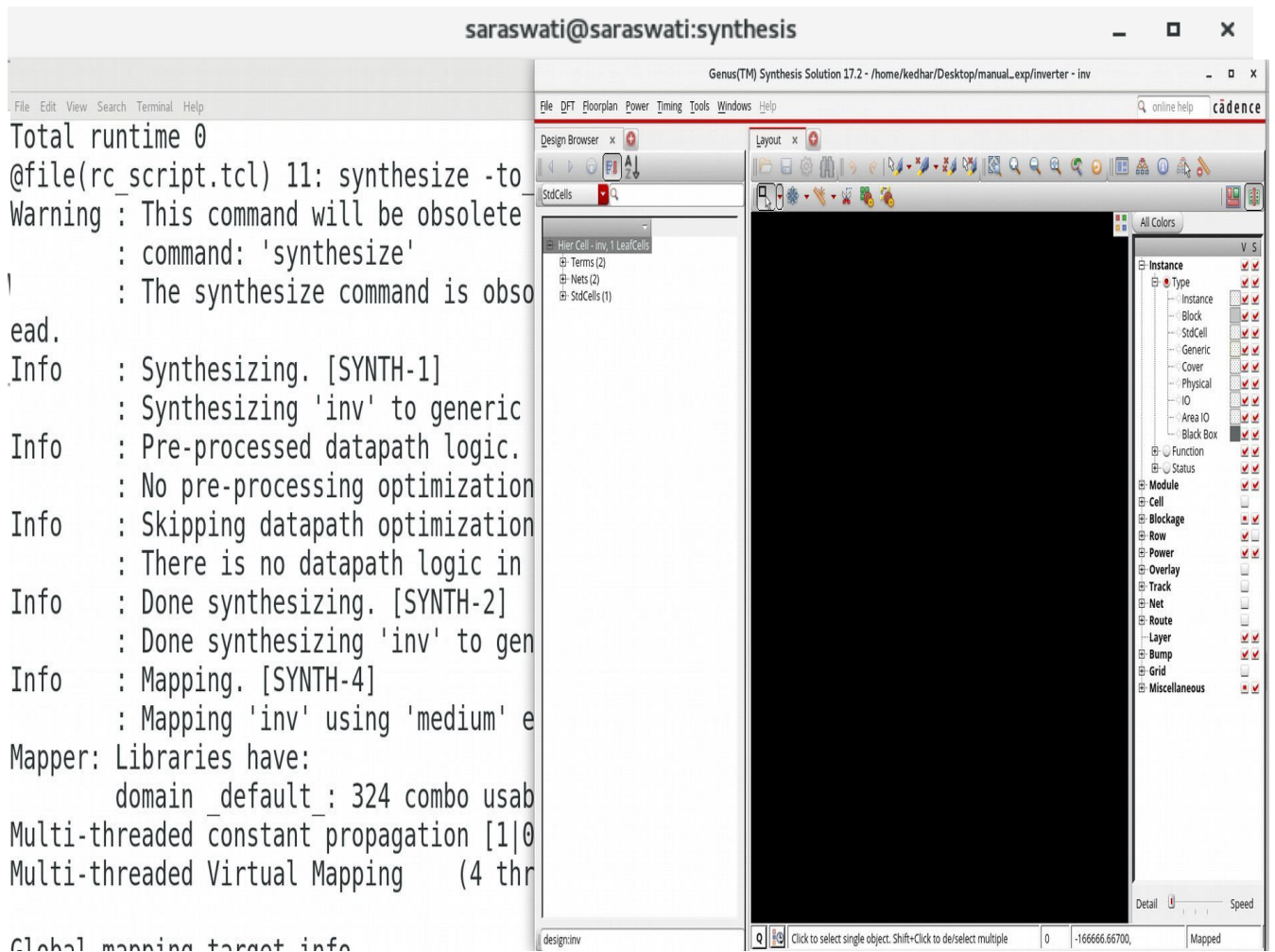
17. To see the schematic capture of invert use the command:

gui_show

Invoke the Genus tool by typing the below command on your terminal:

genus -f genus_script.tcl

The tcl [Tool Command Language] script runs executing each command one after the other.



The screenshot displays two windows side-by-side. The left window is a terminal titled 'saraswati@saraswati:synthesis' showing the output of the 'genus -f genus_script.tcl' command. The output includes a warning about the 'synthesize' command being obsolete, followed by synthesis steps for 'inv' to generic, pre-processed datapath logic, and mapping. The right window is the 'Genus(TM) Synthesis Solution 17.2' interface, showing a hierarchical design browser on the left and a layout view on the right. The design browser shows a hierarchy starting with 'Hier Cell - inv, 1 LeafCells', which contains 'Terms (2)', 'Nets (2)', and 'StdCells (1)'. The layout view shows a large black area representing the design, with a 'Detail' and 'Speed' indicator at the bottom.

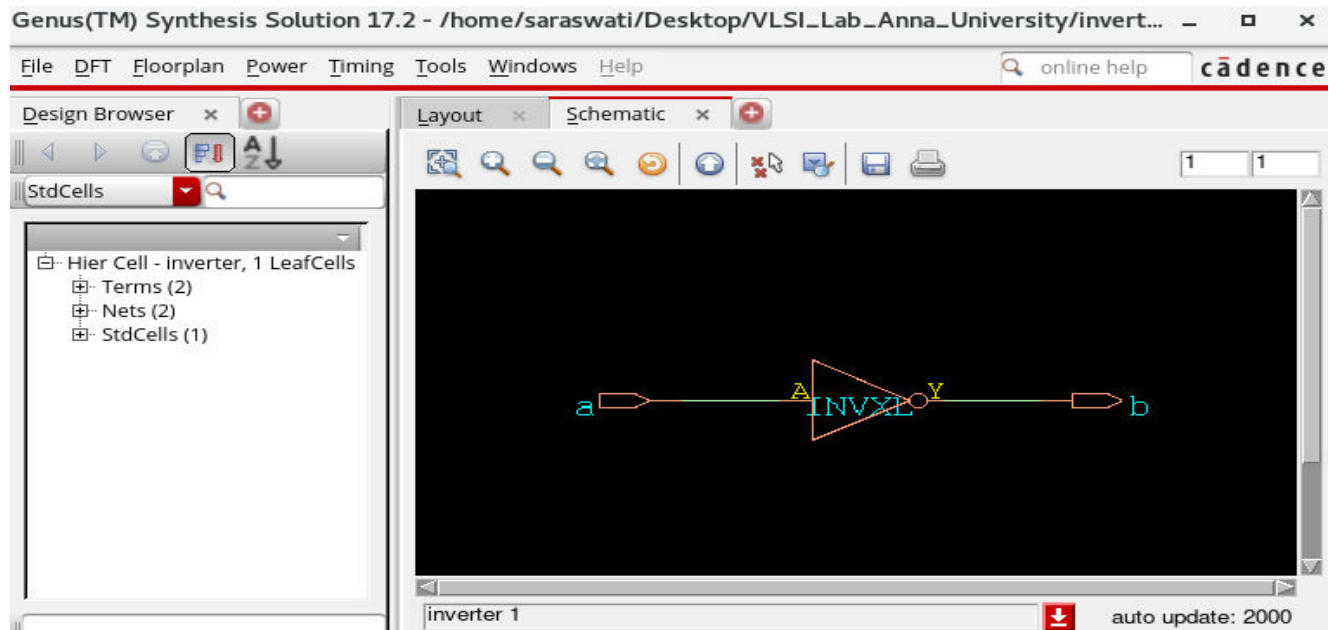
```

Total runtime 0
@file(rc_script.tcl) 11: synthesize -to
Warning : This command will be obsolete
        : command: 'synthesize'
        : The synthesize command is obsolete.
Info    : Synthesizing. [SYNTH-1]
        : Synthesizing 'inv' to generic
Info    : Pre-processed datapath logic.
        : No pre-processing optimization
Info    : Skipping datapath optimization
        : There is no datapath logic in
Info    : Done synthesizing. [SYNTH-2]
        : Done synthesizing 'inv' to generic
Info    : Mapping. [SYNTH-4]
        : Mapping 'inv' using 'medium' effort
Mapper: Libraries have:
        domain_default: 324 combo usable
Multi-threaded constant propagation [1]0
Multi-threaded Virtual Mapping (4 threads)

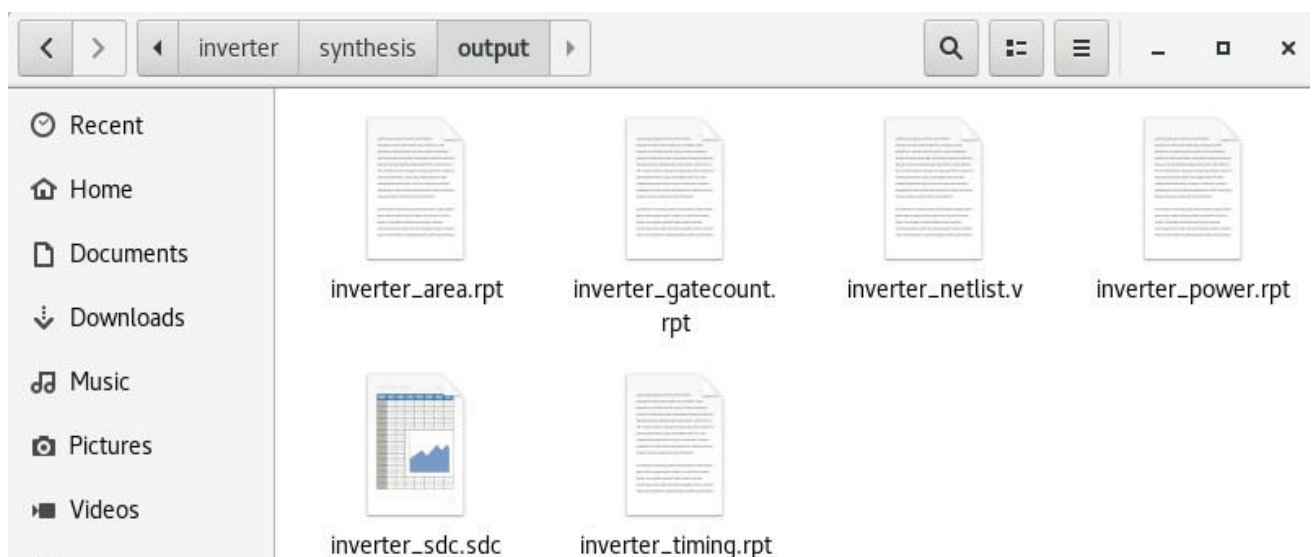
Global mapping target info
  
```


A window of **Genus GUI** pops – up with the top hier cell on the left top.

Make a **Right Click** and select **Schematic Viewer** → **In Main**.



It will generate the **Gate Level Netlist** (inverter_nelist.v), **SDC constraints** (inverter_sdc.sdc) and **Report (Timing, Area, Power and Gates)** for the design as shown below.



Power Report:

```

=====
Generated by:      Genus(TM) Synthesis Solution 17.22-s017_1
Generated on:      Jan 11 2022  02:22:09 pm
Module:           inverter
Technology library: slow
Operating conditions: slow (balanced_tree)
Wireload mode:    enclosed
Area mode:        timing library
=====

```

		Leakage	Dynamic	Total
Instance	Cells	Power(nW)	Power(nW)	Power(nW)
inverter	1	8.757	44.417	53.174

Area Report:

```

=====
Generated by:      Genus(TM) Synthesis Solution 17.22-s017_1
Generated on:      Jan 11 2022  02:22:09 pm
Module:           inverter
Technology library: slow
Operating conditions: slow (balanced_tree)
Wireload mode:    enclosed
Area mode:        timing library
=====

```

Instance	Module	Cell Count	Cell Area	Net Area	Total Area	Wireload
inverter		1	2.271	0.000	2.271	<none> (D)

(D) = wireload is default in technology library

Timing Report:

```

=====
Generated by:      Genus(TM) Synthesis Solution 17.22-s017_1
Generated on:      Jan 11 2022  02:22:09 pm
Module:            inverter
Operating conditions:  slow (balanced_tree)
Wireload mode:     enclosed
Area mode:         timing library
=====

```

Path 1: UNCONSTRAINED Late External Delay Assertion at pin b
 Startpoint: (F) a
 Endpoint: (R) b

```

Output Delay:-      1000
Input Delay:-        1000
Data Path:-         12

```

```

Exceptions/Constraints:
input_delay          1000          constraints_top.sdc_line_1
output_delay         1000          constraints_top.sdc_line_2

```

```

#-----
# Timing Point   Flags   Arc   Edge   Cell       Fanout Load Trans Delay Arrival Instance
#                                     (fF) (ps) (ps) (ps) Location
#-----
# a               -       -     F     (arrival)   1  1.6    0    0    1000    (-,-)
# g2/Y            -       A->Y  R     INVXL       1  0.0    6   12   1012    (-,-)
# b               -       -     R     (port)      -   -     -    0   1012    (-,-)
#-----

```

Gates Report:

```

=====
Generated by:      Genus(TM) Synthesis Solution 17.22-s017_1
Generated on:      Jan 11 2022  02:22:09 pm
Module:            inverter
Operating conditions:  slow (balanced_tree)
Wireload mode:     enclosed
Area mode:         timing library
=====

```

```

Gate  Instances  Area  Library
-----
INVXL          1  2.271    slow
-----
total          1  2.271

```

```

Type           Instances  Area Area %
-----
inverter        1  2.271  100.0
physical_cells  0  0.000   0.0
-----
total          1  2.271  100.0

```

Lab 2: CMOS BASIC GATES & FLIP FLOPS

Lab 2.1 CMOS BASIC GATES

Aim:

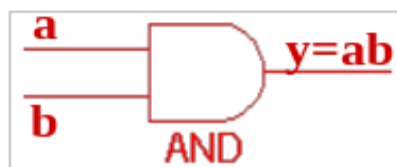
Design and Simulate a **CMOS Basic Gates and Flip Flops**

To write verilog code for a CMOS Basic Gates and its test bench for verification using incisive simulator, observe the waveform and synthesize the code with technological library with given constraints.

Theory:

Digital systems are said to be constructed by using logic gates. These gates are the AND, OR, NAND, NOR and XOR gates. The basic operations are described below with the aid of truth tables.

Lab 2.1a: AND Gate:



2 Input AND gate		
a	b	y=ab
0	0	0
0	1	0
1	0	0
1	1	1

The AND gate is an electronic circuit that gives a high output (1) only if all its inputs are high. “a” dot (.) is used to show the AND operation i.e. “a.b”. Bear in mind that this dot is sometimes omitted i.e. **ab**

Verilog and Test bench Code for AND Gate:

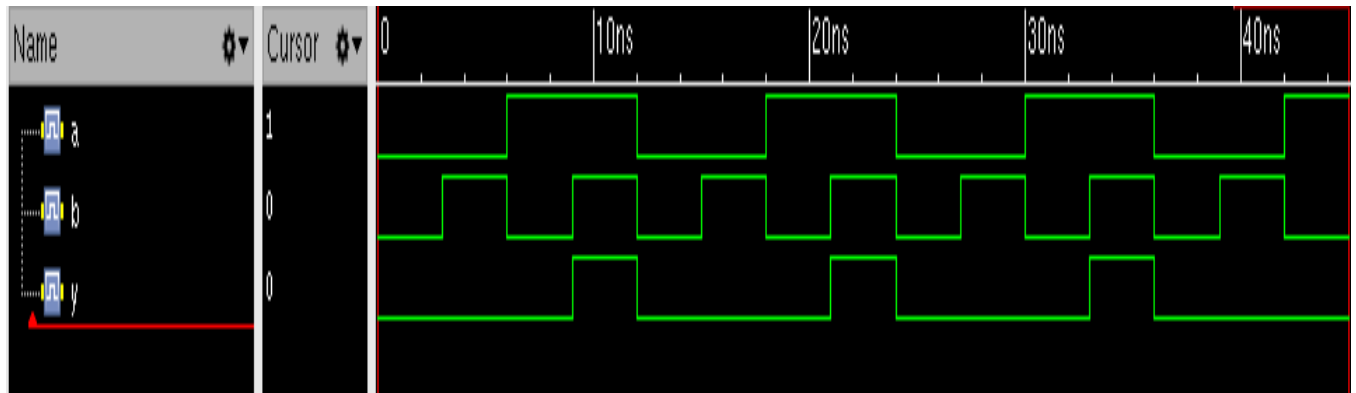
// Verilog source code:

```
module andgate (a, b, y);  
input a, b;  
output y;  
assign y = a & b;  
endmodule
```

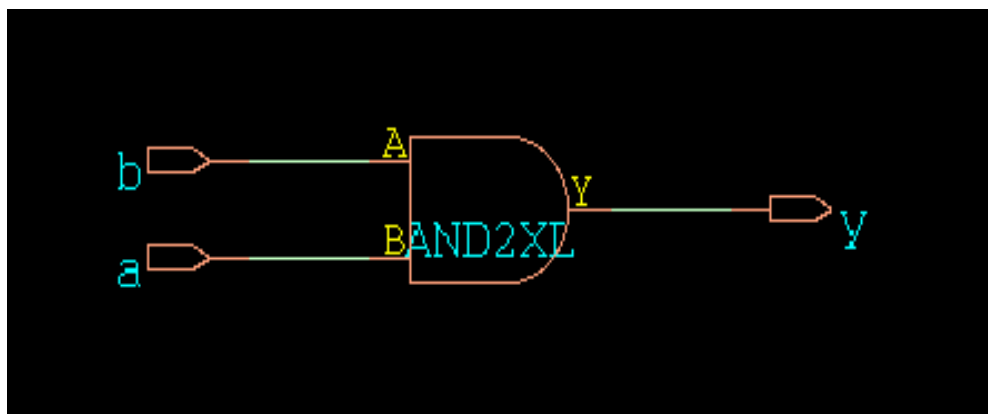
// Test-bench source code :

```
module tb_and_gate;  
reg a,b;  
wire y;  
andgate test (.a(a), .b(b), .y(y));  
//Above style is connecting by names  
initial begin  
a = 1'b0;  
b = 1'b0;  
#45 $finish;  
end  
always #6 a =~a;  
always #3 b =~b;  
always @(y)  
$display( "time =%0t \t INPUT VALUES: \t a=%b b=%b \t output value y =%b", $time, a, b, y);  
endmodule
```

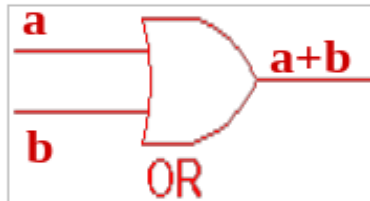
Simulation Result For AND Gate:



Schematic Capture of AND Gate:



Lab 2.1b: OR Gate:



2 Input OR gate		
a	b	y=a+b
0	0	0
0	1	1
1	0	1
1	1	1

The OR gate is an electronic circuit that gives a high output (**1**) if one or more of its inputs are high. “**a**” plus (+) is used to show the **OR operation**.

Verilog and Test bench Code for OR Gate:

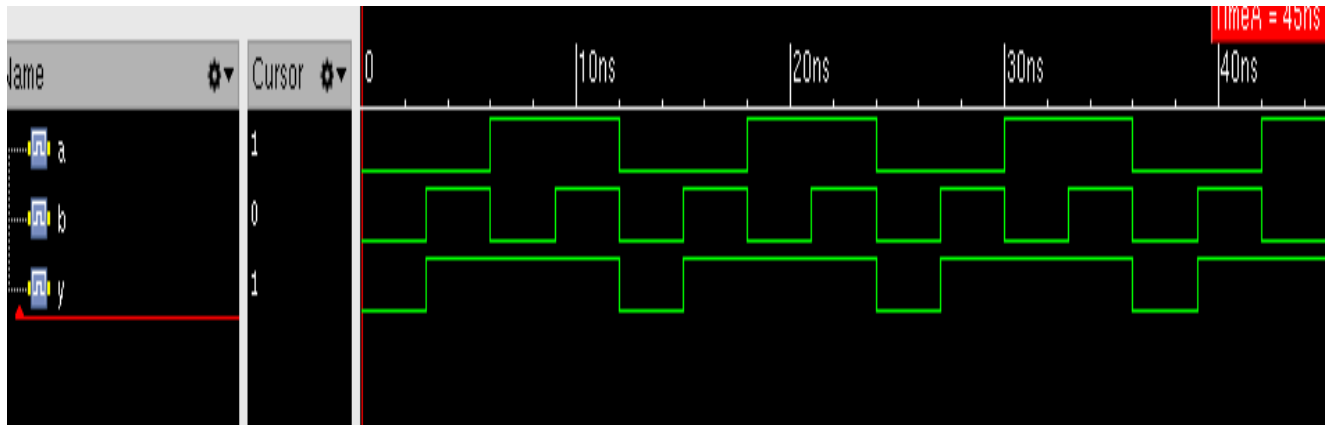
// Verilog source code:

```
module orgate (a, b, y);
  input a, b;
  output y;
  assign y = a | b;
endmodule
```

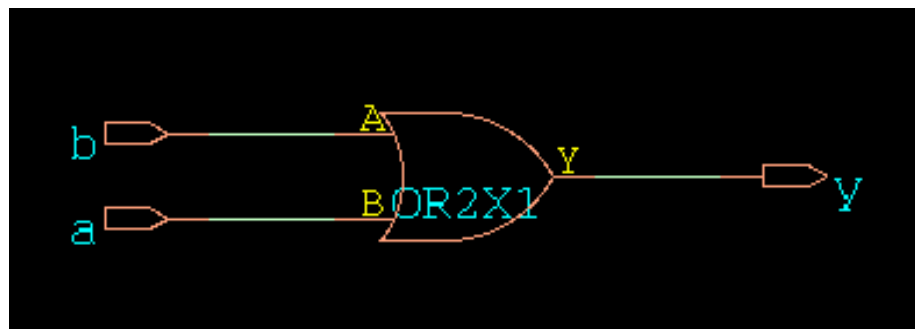
// Test bench source code:

```
module tb_or_gate;
  reg a,b;
  wire y;
  orgate test (.a(a), .b(b), .y(y));
  //Above style is connecting by names
  initial begin
    a =1'b0;
    b = 1'b0;
    #45 $finish;
  end
  always #6 a =~a;
  always #3 b =~b;
  always @(y)
    $display( "time =%0t \t INPUT VALUES: \t a=%b b =%b \t output value y =%b"$time,a,b,y);
endmodule
```

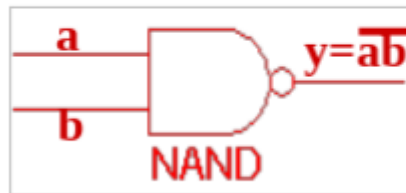
Simulation Result for OR Gate:



Schematic Capture of OR Gate:



Lab 2.1c: NAND Gate:



2 Input <u>NAND</u> Gate		
a	b	$y = \overline{ab}$
0	0	1
0	1	1
1	0	1
1	1	0

This is a NOT-AND gate which is equal to an AND gate followed by a NOT gate. The outputs of all NAND gates are high if any of the inputs are low. The symbol is an AND gate with a small circle on the output. The small circle represents inversion.

Verilog and Test bench Code for NAND Gate:

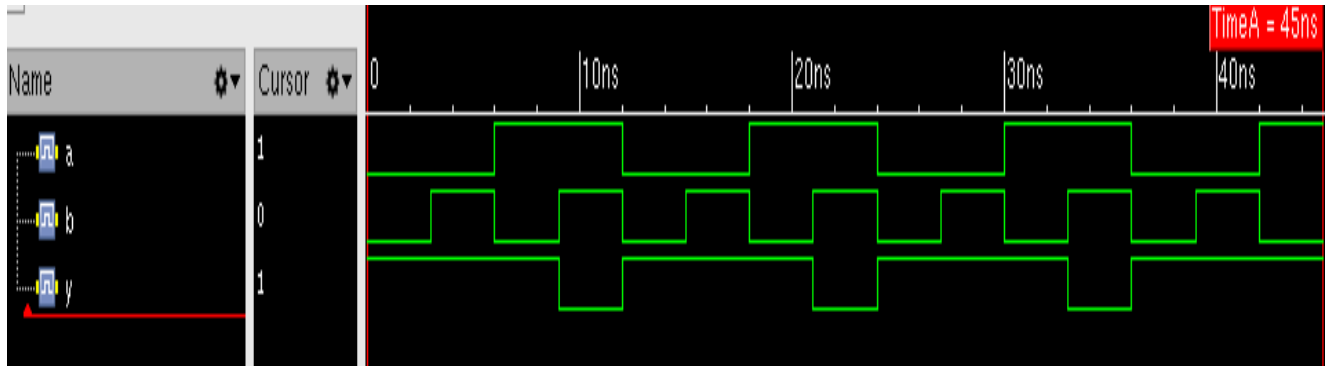
// Verilog source code:

```
module nandgate (a, b, y);
    input a, b;
    output y;
    assign y = ~(a & b);
endmodule
```

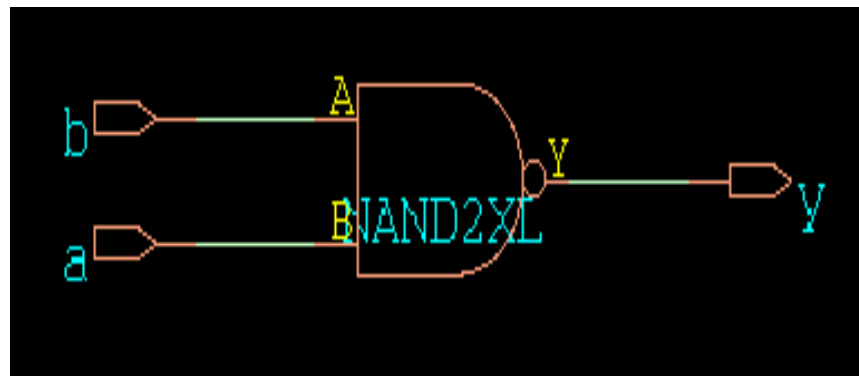
// Test-bench source code:

```
module tb_nand_gate;
    reg a,b;
    wire y;
    nandgate test (.a(a), .b(b), .y(y));
    //Above style is connecting by names
    initial begin
        a = 1'b0;
        b = 1'b0;
        #45 $finish;
    end
    always #6 a = ~a;
    always #3 b = ~b;
    always @(y)
        $display( "time =%0t \t INPUT VALUES: \t a=%b b =%b \t output value y =%b", $time, a, b, y);
endmodule
```

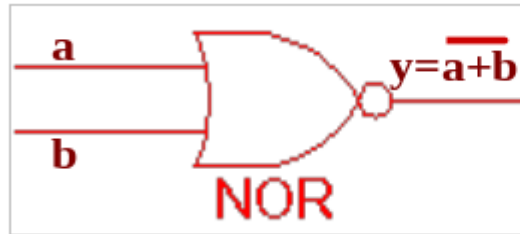
Simulation Result for NAND Gate:



Schematic Capture of NAND Gate:



Lab 2.1d: NOR Gate:



2 Input NOR Gate		
a	b	$y = \overline{a+b}$
0	0	1
0	1	0
1	0	0
1	1	0

This is a **NOT-OR** gate which is equal to an OR gate followed by a NOT gate. The outputs of all **NOR** gates are low if any of the inputs are high. The symbol is an OR gate with a small circle on the output. The small circle represents inversion.

Verilog and Test bench Code for NOR Gate:

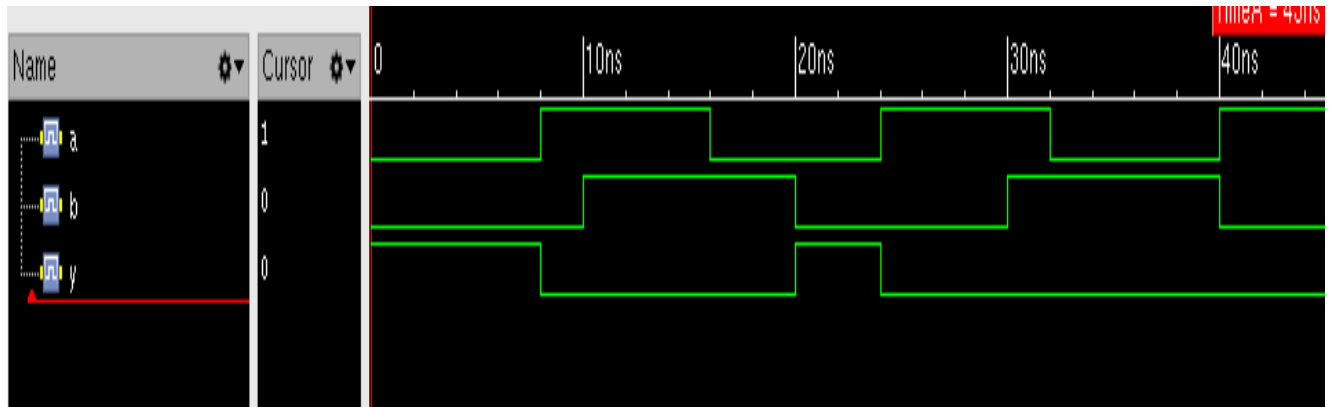
// Verilog Source Code:

```
module norgate (a, b, y);
    input a, b;
    output y;
    assign y = ~(a | b);
endmodule
```

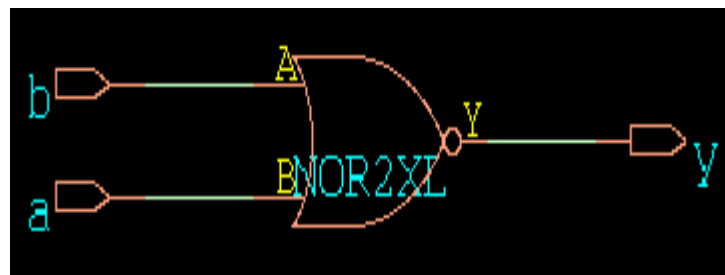
// Test-bench Source Code:

```
module tb_nor_gate;
    reg a,b;
    wire y;
    norgate test (.a(a), .b(b), .y(y));
    //Above style is connecting by names
    initial begin
        a = 1'b0;
        b = 1'b0;
        #45 $finish;
    end
    always #8 a = ~a;
    always #10 b = ~b;
    always @(y)
        $display( "time =%0t \t INPUT VALUES: \t a=%b b=%b \t output value y =%b", $time, a, b, y);
endmodule
```

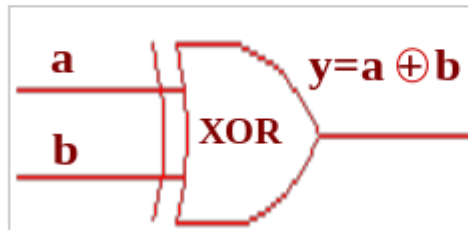
Simulation Result for NOR Gate:



Schematic Capture of NOR Gate:



Lab 2.1e: XOR Gate:



2 Input XOR Gate		
a	b	y = a ⊕ b
0	0	0
0	1	1
1	0	1
1	1	0

The '**Exclusive-OR**' gate is a circuit which will give a high output if either, but not both, of its two inputs are high. An encircled plus sign ($\oplus$) is used to show the **XOR** operation.

Verilog and Test bench Code for XOR Gate:

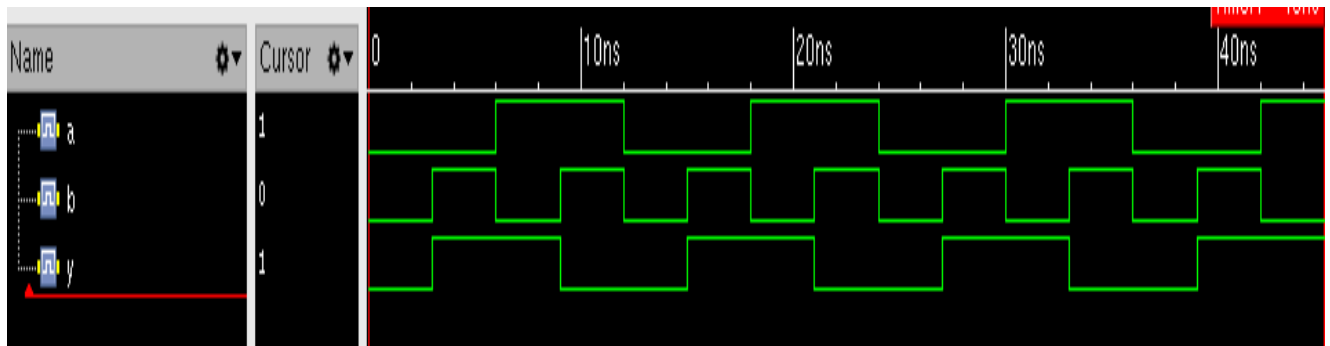
// Verilog Source Code:

```
module xorgate (a, b, y);
    input a, b;
    output y;
    assign y = a ^ b;
endmodule
```

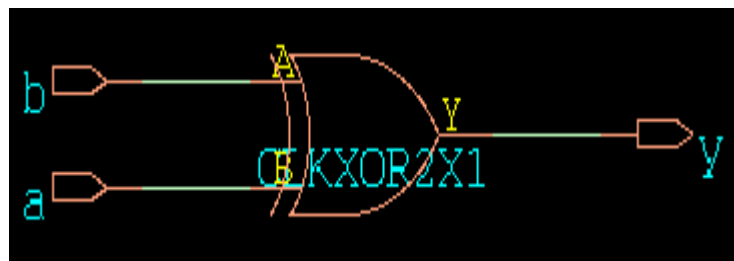
// Test-bench Source Code:

```
module tb_xor_gate;
    reg a,b;
    wire y;
    xorgate test (.a(a), .b(b), .y(y));
    //Above style is connecting by names
    initial begin
        a = 1'b0;
        b = 1'b0;
        #45 $finish;
    end
    always #6 a = ~a;
    always #3 b = ~b;
    always @(y)
        $display( "time =%0t \t INPUT VALUES: \t a=%b b =%b \t output value y =%b", $time, a, b, y);
endmodule
```

Simulation Result for XOR Gate:



Schematic Capture for XOR Gate:



Lab 2.2 FLIP FLOPS

Aim:

Design and Simulate a **Flip Flops** (SR, JK, MS, D and T)

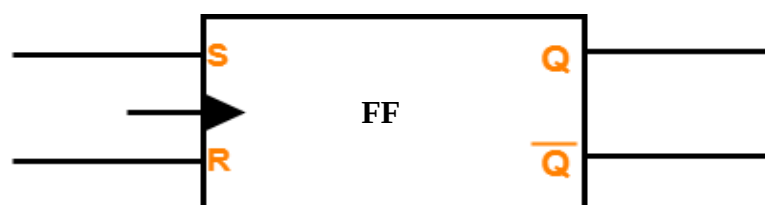
To write verilog code for Flip-Flops and its test bench for verification using incisive simulator, observe the waveform and synthesize the code with technological library with given constraints.

Theory:

In electronics, a flip-flop or latch is a circuit that has two stable states and can be used to store state information. A flip-flop is a bistable multi-vibrator. The circuit can be made to change state by signals applied to one or more control inputs and will have one or two outputs. It is the basic storage element in sequential logic. Flip-flops and latches are fundamental building blocks of digital electronics systems used in computers, communications, and many other types of systems.

Lab 2.2a: SR-Flip Flop:

SR flip-flop is one of the fundamental **sequential circuits**. **SR flip flop is the simplest type of flip flop basically a one-bit memory storage device that has two inputs, one which will “Set” the device (i.e. the output is 1), and is labelled as S and other which will Reset the device (i.e. the output is 0), labelled R. The name SR stands for “Set-Reset”.**



Logic Symbol

INPUTS			OUTPUTS	REMARKS
S	R	Q_n (Present State)	Q_{n+1} (Next State)	States and Conditions
0	0	X	Q_n	Hold State condition $S = R = 0$
0	1	X	0	Reset state condition $S = 0, R = 1$
1	0	X	1	Set state condition $S = 1, R = 0$
1	1	X	Indeterminate	Indeterminate state condition $S = R = 1$

Truth Table

Verilog and Test bench Code for SR Flip Flop:

// Verilog Source Code:

```

module SR_flipflop(q,q1,r,s,clk);
output q,q1;
input r,s,clk;
reg q,q1;
initial begin
q=1'b0; q1=1'b1;
end
always @(posedge clk)
begin
case({s,r})
{1'b0,1'b0}: begin q=q; q1=q1; end
{1'b0,1'b1}: begin q=1'b0; q1=1'b1; end
{1'b1,1'b0}: begin q=1'b1; q1=1'b0; end
{1'b1,1'b1}: begin q=1'bx; q1=1'bx; end
endcase
end
endmodule

```

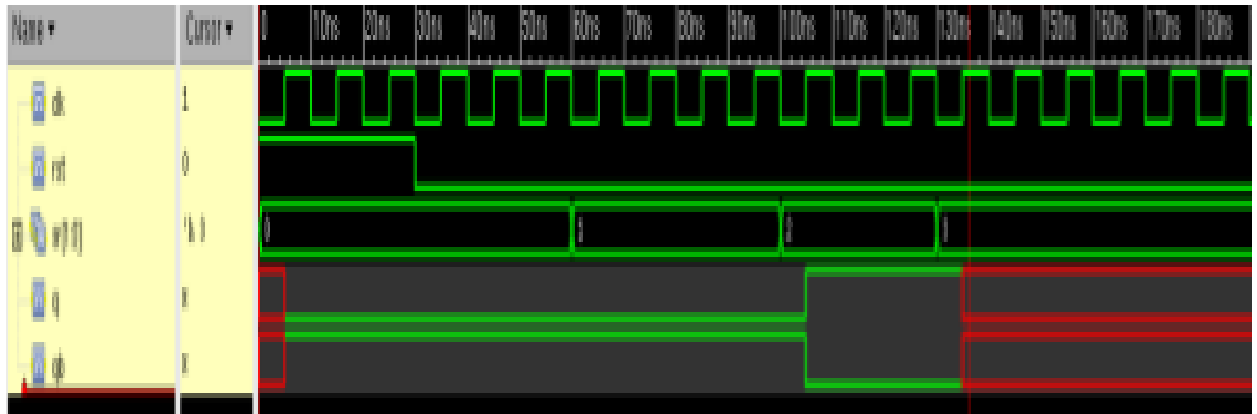

// Test-bench Source Code:

```

module test;
  reg clk=0; reg s=0; reg r=0;
  wire q, qnot;
  jkff dut(reset, clk, j, k, q, qnot);
  initial begin
    s=1'b1; r=1'b1;
    #25 $finish;
  end
  always #1 clk=~clk;
endmodule

```

Simulation Result for SR-Flip Flop:



Lab 2.2b: JK-Flip Flop:

JK flip flop is a refined & improved version of SR flip-flop that has been introduced to solve the problem of indeterminate state that occurs in SR flip flop when both the inputs are 1. The JK flip-flop is probably the most widely used and is considered the universal flip-flop because it can be used in many ways.



Logic Symbol

INPUTS			OUTPUTS	REMARKS
J	K	Q_n (Present State)	Q_{n+1} (Next State)	States and Conditions
0	0	X	Q_n	Hold State condition $J = K = 0$
0	1	X	0	Reset state condition $J = 0, K = 1$
1	0	X	1	Set state condition $J = 1, K = 0$
1	1	X	Q'_n	Toggle state condition $J = K = 1$

Truth Table

Verilog and Test bench Code for JK Flip Flop:

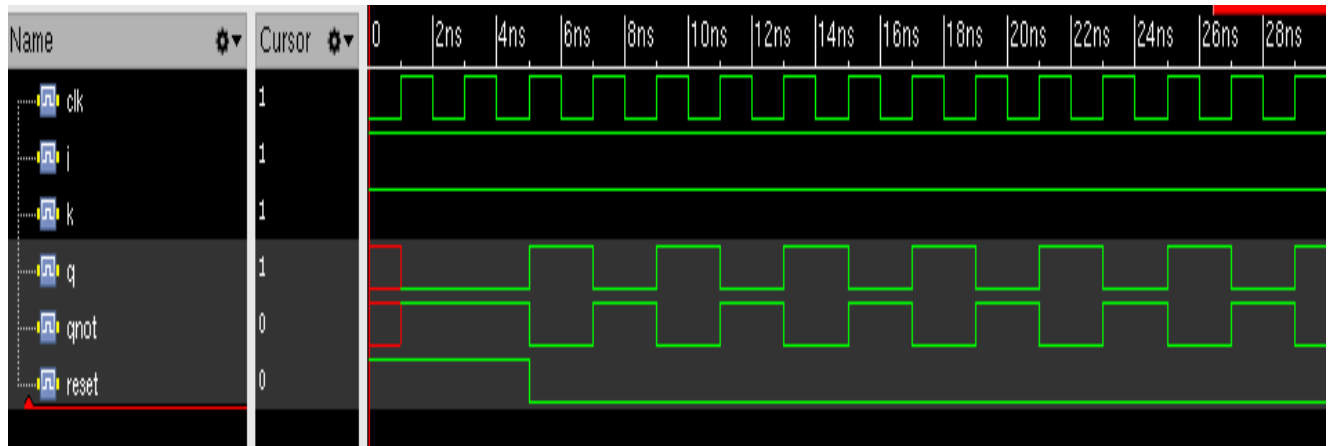
// Verilog Source Code:

```
module jkff(input reset, input clk, input j, input k, output reg q, output qnot);
    assign qnot=~q;
    always @(posedge clk)
        if (reset) q<=1'b0;
        else
            case ({j, k})
                2'b00: q<=q;
                2'b01: q<=1'b0;
                2'b10: q<=1'b1;
                2'b11: q<=~q;
            endcase
    endmodule
```

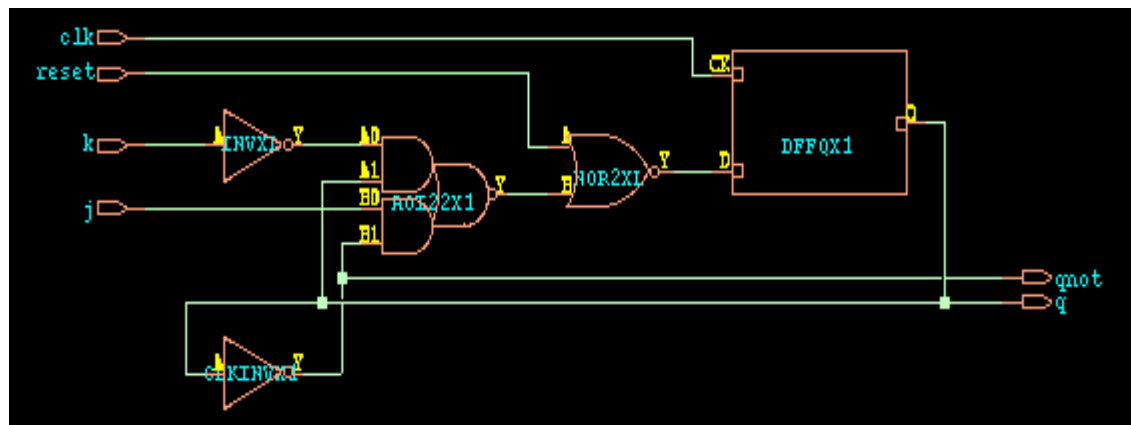
// Test-bench Source Code:

```
module test;
    reg clk=0;
    reg j=0;
    reg k=0;
    reg reset=1;
    wire q, qnot;
    jkff dut(reset, clk, j, k, q, qnot);
    initial
        begin
            $dumpfile("dump.vcd");
            $dumpvars(1);
            j=1'b1; // set your JK here
            k=1'b1;
            #5 reset=1'b0;
            #25 $finish;
        end
    always #1 clk=~clk;
endmodule
```

Simulation Result for JK-Flip Flop:



Schematic Capture of JK Flip Flop:

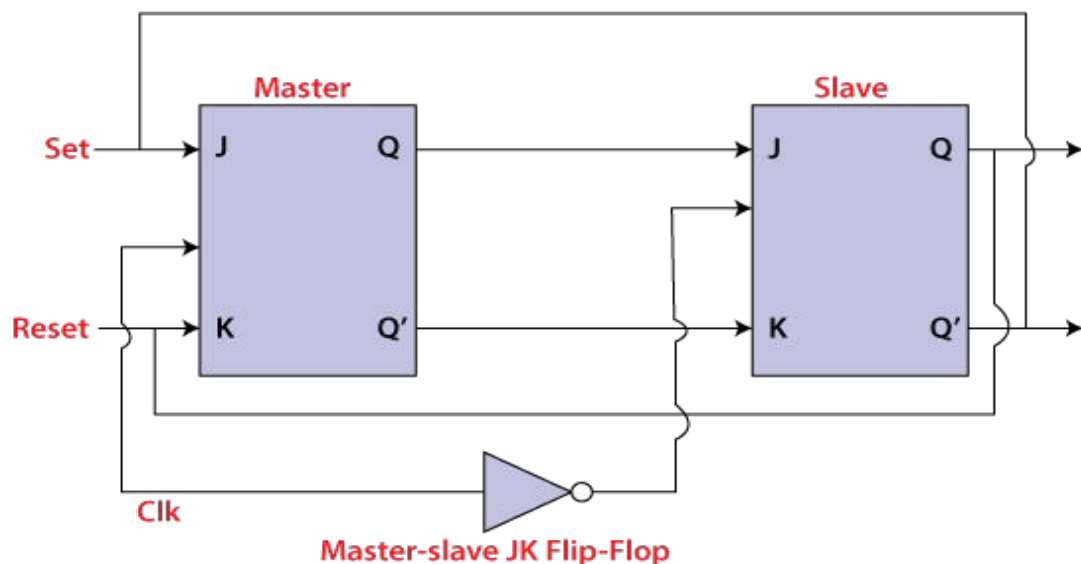


Lab 2.2c: MS-JK Flip Flop:

The Master-Slave Flip-Flop is basically a combination of two JK flip-flops connected together in a series configuration. Out of these, one acts as the “**master**” and the other as a “**slave**”. The output from the master flip flop is connected to the two inputs of the slave flip flop whose output is fed back to inputs of the master flip flop.

In addition to these two flip-flops, the circuit also includes an **inverter**. The inverter is connected to clock pulse in such a way that the inverted clock pulse is given to the slave flip-flop. In other words if CP=0 for a master flip-flop, then CP=1 for a slave flip-flop and if CP=1 for master flip flop then it becomes 0 for slave flip flop.

Race Around Condition In JK Flip-flop – For J-K flip-flop, if $J=K=1$, and if $\text{clk}=1$ for a long period of time, then Q output will toggle as long as CLK is high, which makes the output of the flip-flop unstable or uncertain. This problem is called race around condition in J-K flip-flop. This problem (Race Around Condition) can be avoided by ensuring that the clock input is at logic “1” only for a very short time



Verilog and Test bench Code for MS Flip Flop:

// Verilog Source Code:

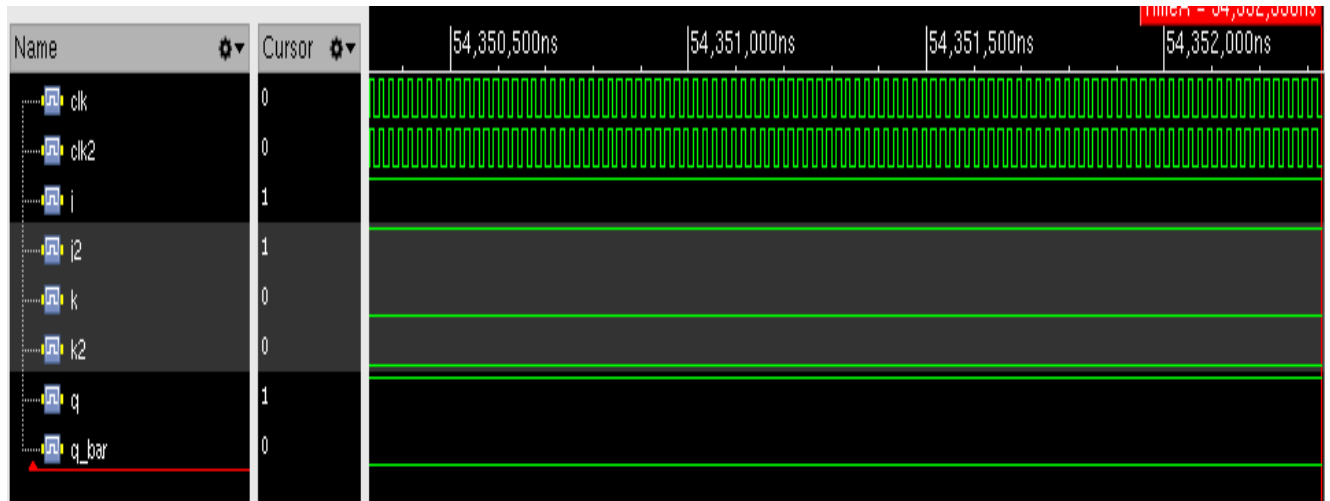
```
module ms_jkff(q,q_bar,clk,j,k);
    output q,q_bar;
    input clk,j,k;
    reg tq,q_bar;
    always @(clk)
    begin
        if (!clk)
            begin
                if (j==1'b0 && k==1'b1)
                    tq <= 1'b0;
                else if (j==1'b1 && k==1'b0)
                    tq <= 1'b1;
                else if (j==1'b1 && k==1'b1)
                    tq <= ~tq;
            end
        if (clk)
            begin
                q <= tq;
                q_bar <= ~tq;
            end
        end
    endmodule
```

// Test-bench Source Code:

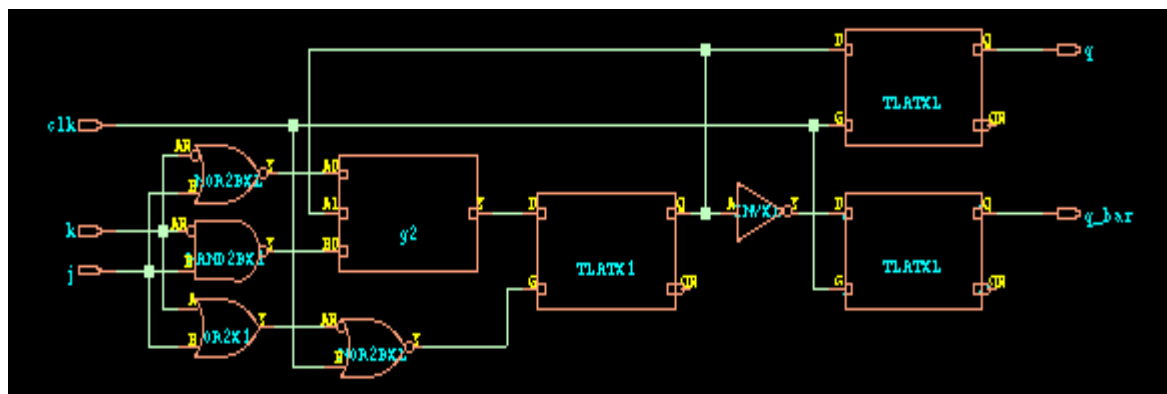
```
module tb_ms_jkff;
  reg clk,j,k;
  wire q,q_bar;
  wire clk2,j2,k2;
  ms_jkff inst(q,q_bar,clk,j,k);
  assign clk2=clk;
  assign j2=j;
  assign k2=k;
  initial
    clk = 1'b0;
    always #10
      clk = ~clk;

  initial
  begin
    j = 1'b0; k = 1'b0;
    #60 j = 1'b0; k = 1'b1;
    #40 j = 1'b1; k = 1'b0;
    #20 j = 1'b1; k = 1'b1;
    #40 j = 1'b1; k = 1'b0;
    #5 j = 1'b0; #20 j = 1'b1;
  end
endmodule
```

Simulation Result for MS-Flip Flop:



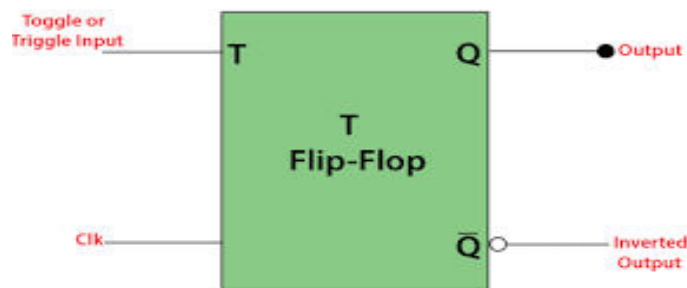
Schematic Capture of MS Flip Flop:



Lab 2.2d: T-Flip Flop:

T flip flop is known as a **toggle** flip flop because of its toggling operation. It is a modified form of the JK flip flop. A T-flip flop is constructed by connecting J and K inputs, creating a single input called T. Hence why a T flip flop is also known as a **single input JK flip flop**.

The defining characteristic of T flip flop is that it can change its output state. You can change the output signal from one state (on or off) to another state (off or on). The clock signal must set high to toggle the output. When the clock is set low, the output remains as it is whether the input signal is set high or low. So, to change the output condition, the clock signal has to be high.



Symbol of T-FF

T	Q _n	Q _{n+1}	Action
0	0	0	Unchanged/hold
0	1	1	Unchanged/hold
1	0	1	Toggle
1	1	0	Toggle

Truth Table of T-FF

Verilog and Test bench Code for T Flip Flop:

// Verilog Source Code:

```

module tffmod(t, clk, q);
input t; input clk;
output q;
reg q;
initial
q <= 0;
always @(posedge clk)
q <= q^t;
endmodule

```

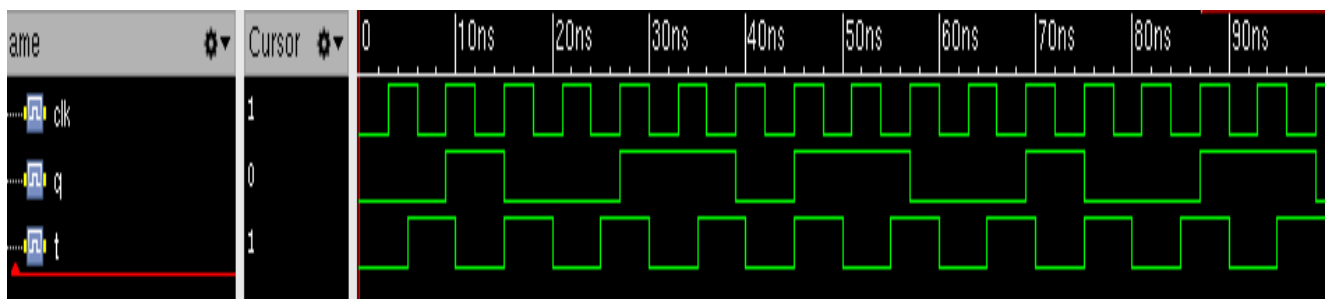
// Test-bench Source Code:

```

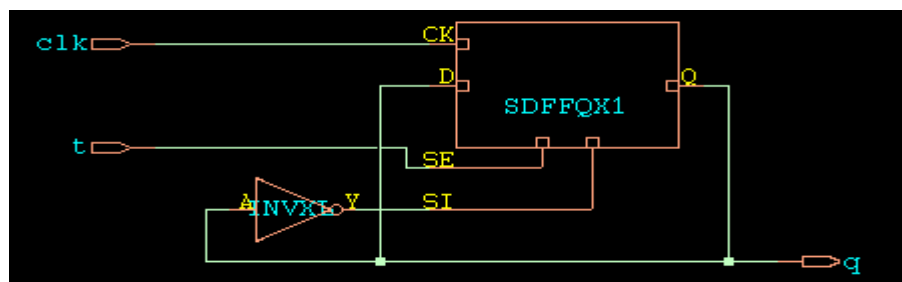
module tfflfflopt_b;
  reg t;
  reg clk;
  wire q;
  tffmod uut (.t(t), .clk(clk), .q(q));
  initial begin
    t = 0; clk = 0;
    #100;
  end
  always #3 clk=~clk;
  always #5 t=~t;
  initial
    #100 $stop;
endmodule

```

Simulation Result for T-Flip Flop:



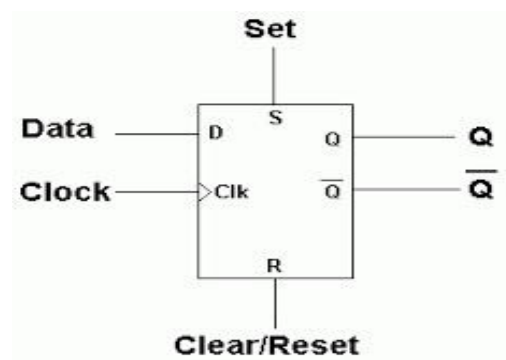
Schematic Capture of T Flip Flop:



Lab 2.2e: D-Flip Flop:

D Flip-Flop is one of that Flip Flop that can store data. It can be used to store data statically or dynamically depends on the design of the circuit. D Flip-Flop is used in many sequential circuits as register, counter, etc.

D flip-flop or Data flip flop is a type of flip Flop that has only one data input that is 'D' and one clock pulse input with two outputs Q and Q bar. This Flip Flop is also called a delay flip flop because when the input data is provided into the d flip-flop, the output follows the input data delay by one clock pulse.



Symbol of D-FF

Input			Output	
D	reset	clock	Q	Q'
0	0	0	0	1
0	0	1	0	1
0	1	0	0	1
0	1	1	0	1
1	0	0	0	1
1	0	1	1	0
1	1	0	0	1
1	1	1	0	1

Characteristics table of D-FF

Module 1: Writing RTL Code for D-Flip Flop

Verilog and Test bench Code for D Flip Flop:

// Verilog Source Code:

```
module dff_sync_reset (data,clk,reset,q);
input data, clk, reset ;
output q;
reg q;
always @ ( posedge clk)
if (~reset) begin
q <= 1'b0;
end else begin
q <= data;
end
endmodule
```

// Test-bench Source Code:

```
module tb_DFF();
reg data;
reg clk;
reg reset;
wire Q;
dff_sync_reset dut(data, clk, reset, Q);
initial begin
clk=0;
forever #10 clk = ~clk;
end
initial begin
reset=1;
data <= 0;
#100;
reset=0;
data <= 1;
#100;
data <= 0;
#100;
data <= 1;
end
endmodule
```

Module 2: Design Simulation using INCISIVE Simulator

saraswati@saraswati:Flipflops

File Edit View Search Terminal Help

```
c[saraswati@saraswati Flipflops]$ csh
[saraswati@saraswati Flipflops]$ source /home/saraswati/install/cshrc
```

Welcome to Cadence Tools Suite

```
[saraswati@saraswati Flipflops]$ nclaunch&
[1] 20423
[saraswati@saraswati Flipflops]$ nclaunch(64): 15.20-s051: (c) Copyright 1995-2018 Cadence Design Systems, Inc.
```

NCLaunch : /home/saraswati/Desktop/VLSI_Lab_Anna_University_2022/Flipflops

File Edit Tools Utilities Plug-Ins Help cadence

Browsers:  Tools: 

Directory: p/VLSI_Lab_Anna_University_2022/Flipflops

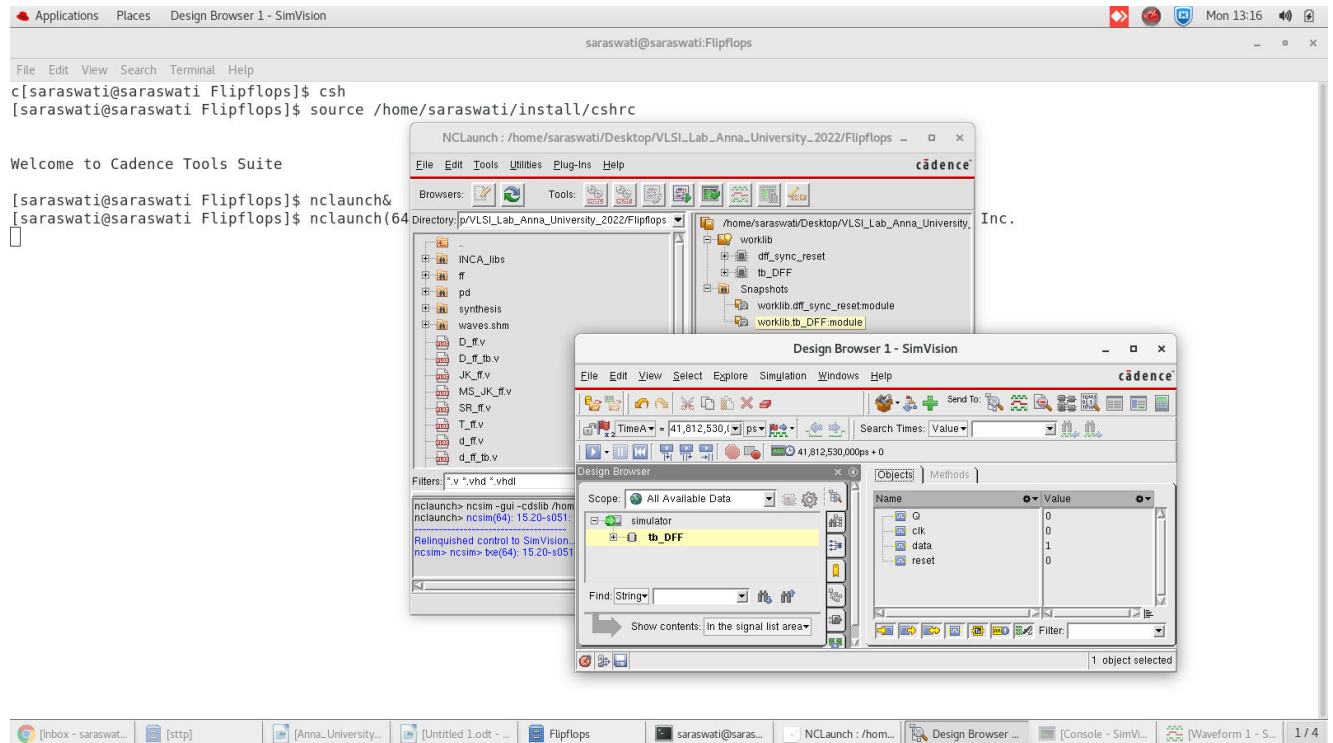
- ..
- INCA_libs
- ff
- pd
- synthesis
- waves.shm
- D_ff.v
- D_ff_tb.v
- JK_ff.v
- MS_JK_ff.v
- SR_ff.v
- T_ff.v
- d_ff.v
- d_ff_tb.v

- /home/saraswati/Desktop/VLSI_Lab_Anna_University_2022/Flipflops
- worklib
- Snapshots

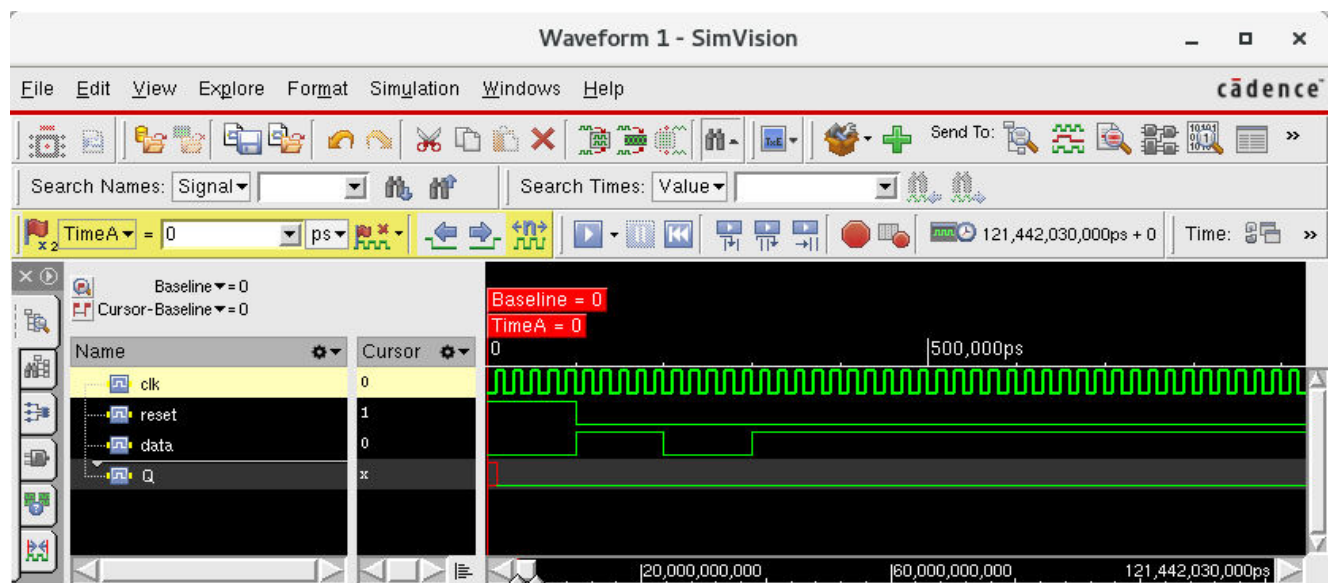
Filters: *.v *.vhd *.vhdl

```
nclab: CPU Usage - 0.0s system + 0.0s user = 0.1s total (2.0s, 3.2% cpu)
nclaunch> nclab -work worklib -cdslib /home/saraswati/Desktop/VLSI_Lab_Anna_University_2022/Flipflops/cds.lib -
nclab(64): 15.20-s051: (c) Copyright 1995-2018 Cadence Design Systems, Inc.
nclab: Memory Usage - 49.9M program + 31.9M data = 81.9M total (Peak 86.9M)
nclab: CPU Usage - 0.0s system + 0.0s user = 0.1s total (0.9s, 7.2% cpu)
nclaunch>
```

You can Compile, Elaborate and Simulate the D-flip flop with source codes available with verilog and test-bench and observe the simulation output as shown below.

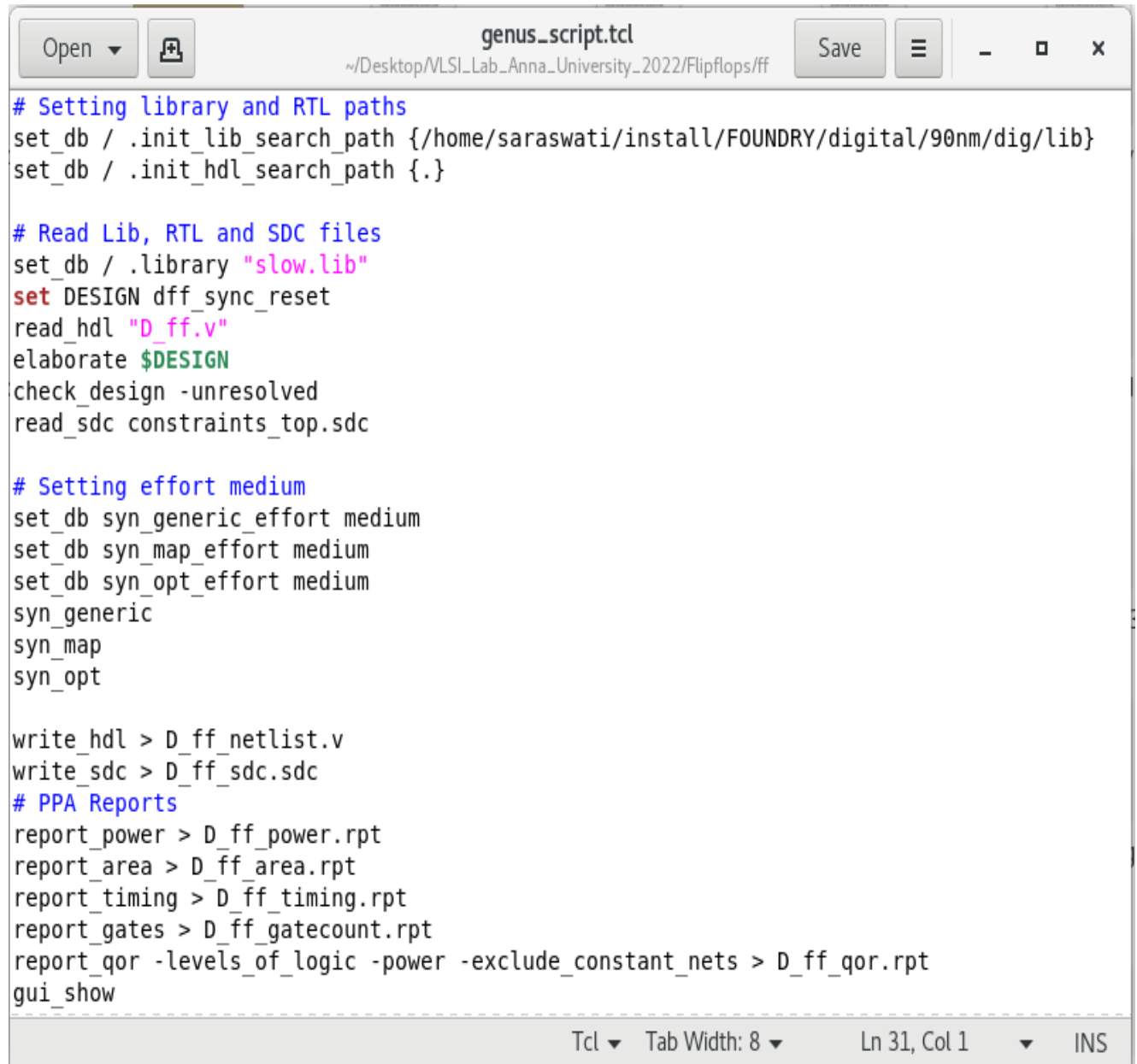


Simulation Result for D-Flip Flop:



Module 3: Synthesis Using GENUS Tool

To run the synthesis, the following script can be used. Inside the genus script file we have to mention the commands along with sdc constraints file as shown below.



```

genus_script.tcl
~/Desktop/VLSI_Lab_Anna_University_2022/Flipflops/ff

# Setting library and RTL paths
set_db / .init_lib_search_path {/home/saraswati/install/FOUNDRY/digital/90nm/dig/lib}
set_db / .init_hdl_search_path {.}

# Read Lib, RTL and SDC files
set_db / .library "slow.lib"
set DESIGN dff_sync_reset
read_hdl "D_ff.v"
elaborate $DESIGN
check_design -unresolved
read_sdc constraints_top.sdc

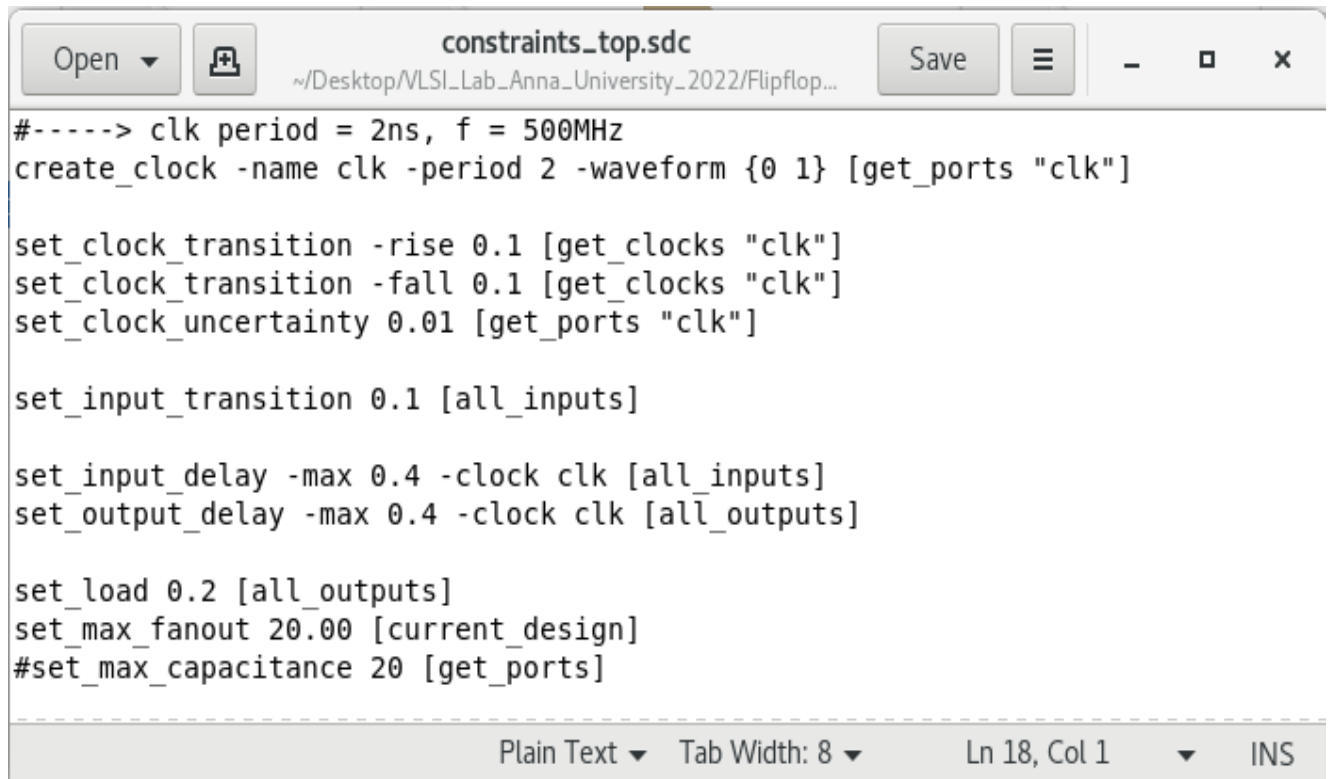
# Setting effort medium
set_db syn_generic_effort medium
set_db syn_map_effort medium
set_db syn_opt_effort medium
syn_generic
syn_map
syn_opt

write_hdl > D_ff_netlist.v
write_sdc > D_ff_sdc.sdc
# PPA Reports
report_power > D_ff_power.rpt
report_area > D_ff_area.rpt
report_timing > D_ff_timing.rpt
report_gates > D_ff_gatecount.rpt
report_qor -levels_of_logic -power -exclude_constant_nets > D_ff_qor.rpt
gui_show

Tcl Tab Width: 8 Ln 31, Col 1 INS

```

Chip Level SDC (constraints_top.sdc):



```
#-----> clk period = 2ns, f = 500MHz
create_clock -name clk -period 2 -waveform {0 1} [get_ports "clk"]

set_clock_transition -rise 0.1 [get_clocks "clk"]
set_clock_transition -fall 0.1 [get_clocks "clk"]
set_clock_uncertainty 0.01 [get_ports "clk"]

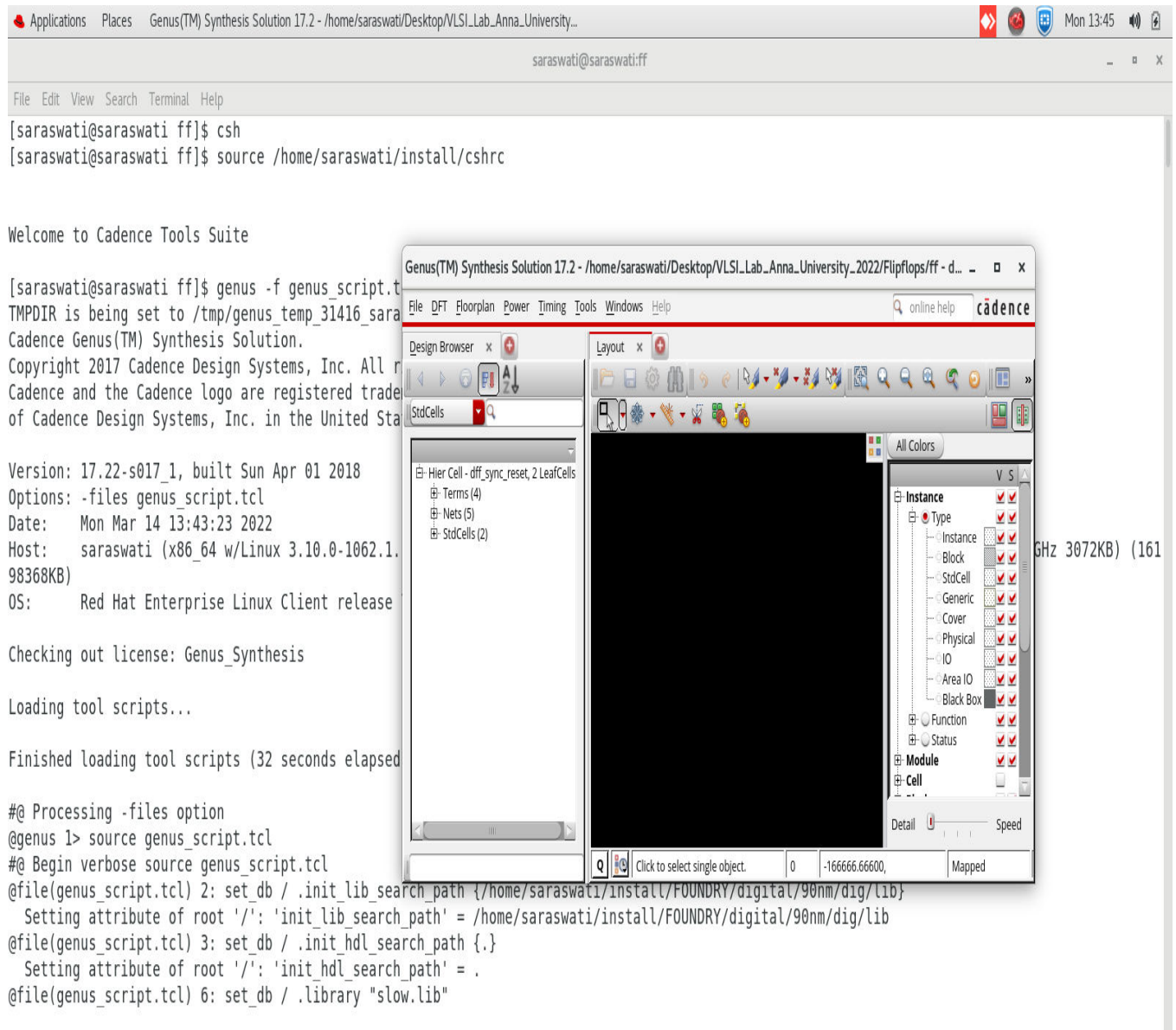
set_input_transition 0.1 [all_inputs]

set_input_delay -max 0.4 -clock clk [all_inputs]
set_output_delay -max 0.4 -clock clk [all_outputs]

set_load 0.2 [all_outputs]
set_max_fanout 20.00 [current_design]
#set_max_capacitance 20 [get_ports]
```

Plain Text ▼ Tab Width: 8 ▼ Ln 18, Col 1 ▼ INS

Invoke the Genus tool by typing the below command on your terminal: **genus -f genus_script.tcl**
The tcl [Tool Command Language] script runs executing each command one after the other.



```

[saraswati@saraswati ff]$ csh
[saraswati@saraswati ff]$ source /home/saraswati/install/cshrc

Welcome to Cadence Tools Suite

[saraswati@saraswati ff]$ genus -f genus_script.tcl
TMPDIR is being set to /tmp/genus_temp_31416_sara
Cadence Genus(TM) Synthesis Solution.
Copyright 2017 Cadence Design Systems, Inc. All r
Cadence and the Cadence logo are registered trade
of Cadence Design Systems, Inc. in the United Sta

Version: 17.22-s017_1, built Sun Apr 01 2018
Options: -files genus_script.tcl
Date:   Mon Mar 14 13:43:23 2022
Host:   saraswati (x86_64 w/Linux 3.10.0-1062.1.
98368KB)
OS:     Red Hat Enterprise Linux Client release

Checking out license: Genus_Synthesis

Loading tool scripts...

Finished loading tool scripts (32 seconds elapsed)

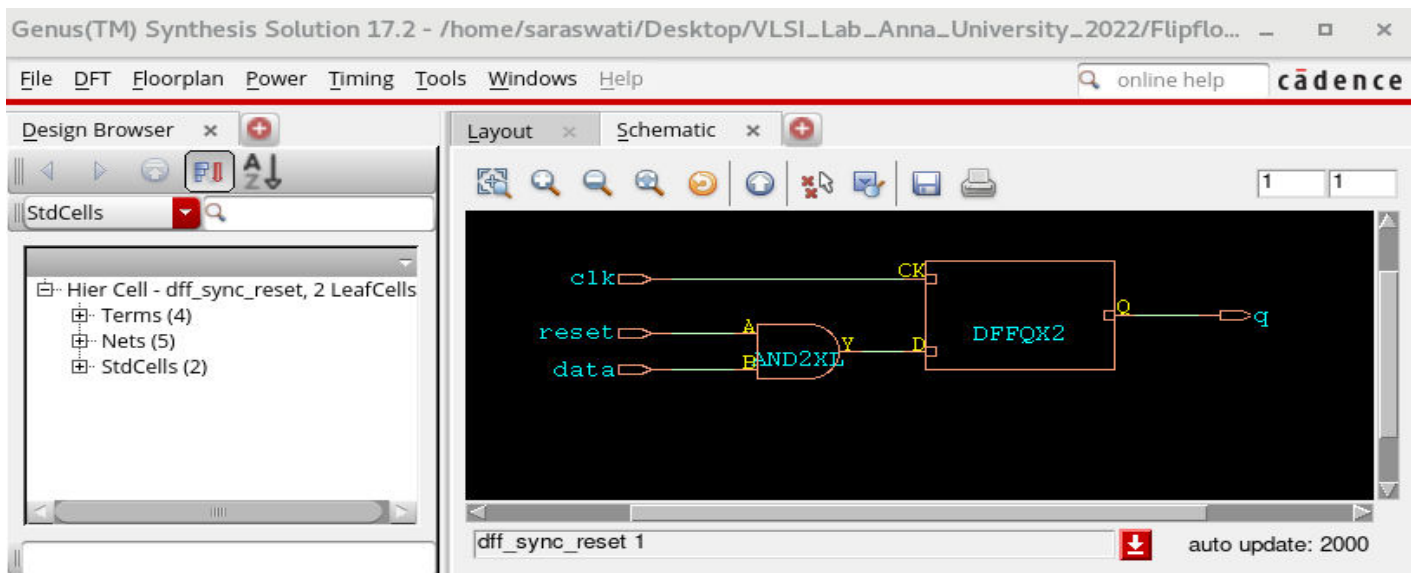
#@ Processing -files option
@genus 1> source genus_script.tcl
#@ Begin verbose source genus_script.tcl
@file(genus_script.tcl) 2: set_db / .init_lib_search_path {/home/saraswati/install/FOUNDRY/digital/90nm/dig/lib}
    Setting attribute of root '/': 'init_lib_search_path' = /home/saraswati/install/FOUNDRY/digital/90nm/dig/lib
@file(genus_script.tcl) 3: set_db / .init_hdl_search_path {.}
    Setting attribute of root '/': 'init_hdl_search_path' = .
@file(genus_script.tcl) 6: set_db / .library "slow.lib"
  
```

GHZ 3072KB) (161

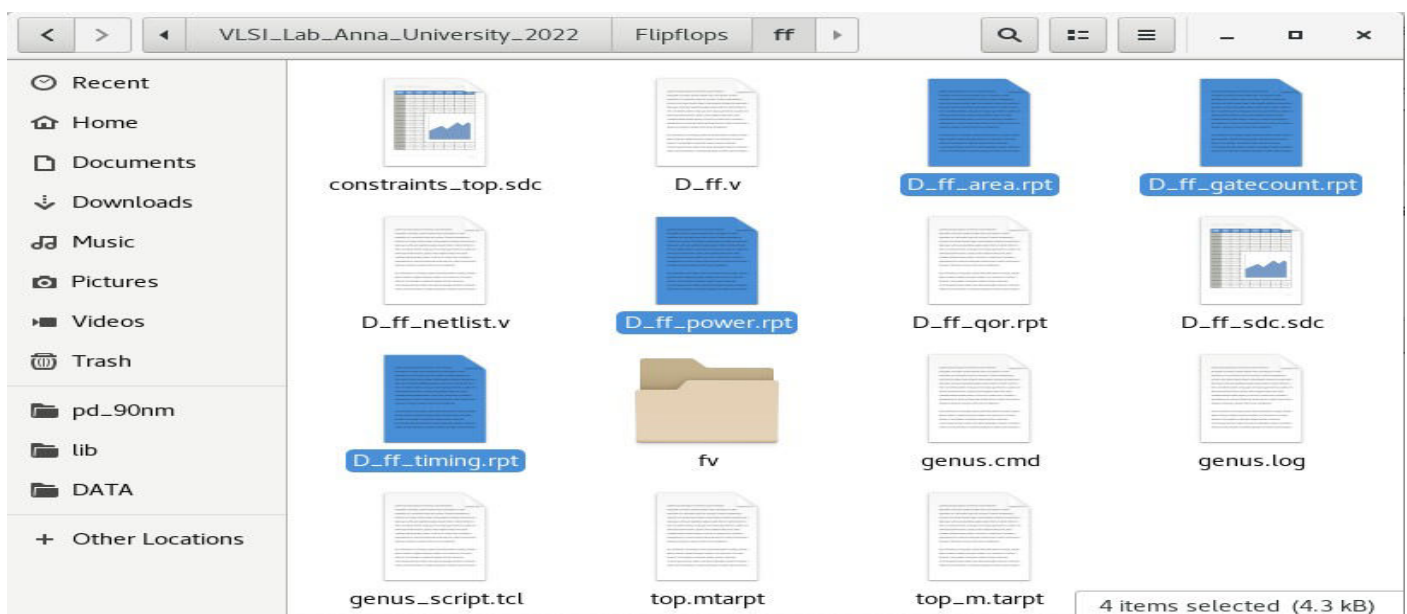
A window of **Genus GUI** pops – up with the top hier cell on the left top. Make a **Right Click** and select **Schematic Viewer** → **In Main**.

Or

You can click on + **option** form the Genus GUI where you can choose the **Schematic** option see the schematic capture of D-flip flop Gate level netlist.



Output & Report Generation:



Lab 3: 4-BIT SYNCHRONOUS COUNTER

Aim: Design and Simulate a 4-bit Synchronous counter using a Flip-Flops

- Manual/Automatic Layout Generation and Post Layout Extraction
- Analyzing the Power, Area and Timing for Exp 7 to 9 by Performing Pre Layout and Post Layout Simulations.

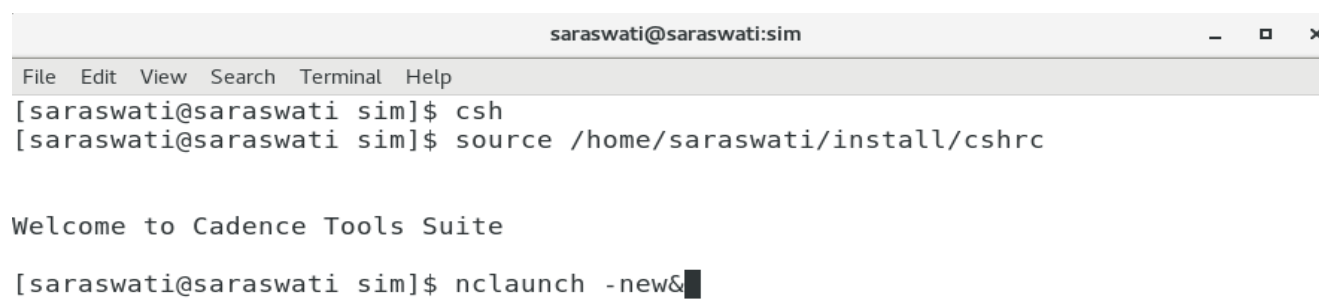
Tool Required:

- Functional Simulation: INCISIVE Simulator (ncvlog, ncelab, ncsim)
- Synthesis: GENUS Synthesis Solution Tool
- Physical Design Implementation/Automatic Layout Generation: INNOVUS Implementation Tool

Module 1: Work Space Creation & Writing RTL Code for 4Bit Synchronous Counter

Getting Started:

1. Create **sub directory/Folder** or folder with name **counter**
2. Open the “**counter**” directory and make a Right Click to “**Open in Terminal**”.
3. Create and Save the Verilog & Test-bench source code for counter using command shown below:
gedit counter.v counter_test.v
4. Invoke the the simulation tool for checking the functionality of the code with help of simulation waveform window.



```
saraswati@saraswati:sim
File Edit View Search Terminal Help
[saraswati@saraswati sim]$ csh
[saraswati@saraswati sim]$ source /home/saraswati/install/cshrc

Welcome to Cadence Tools Suite

[saraswati@saraswati sim]$ nclaunch -new&
```

Verilog and Test bench Code for 4BIT Synchronous Counter using Flip Flop:

```

// 4-BIT Synchronous Counter Verilog Code
`timescale 1ns/1ps
module counter(clk,m,rst,count);
input clk,m,rst;
output reg [3:0] count;
always@(posedge clk or negedge rst)
begin
if(!rst)
count=0;
else if(m)
count=count+1;
else
count=count-1;
end
endmodule

```

```

// 4-BIT Synchronous Counter Testbench Code
`timescale 1ns/1ps
module counter_test;
reg clk, rst,m;
wire [3:0] count;
initial
begin
clk=0;
rst=0;#100;
rst=1;
end
initial
begin
m=0;
#600 m=1;
#500 m=0;
end

counter counter1(clk,m,rst, count);

always #5 clk=~clk;

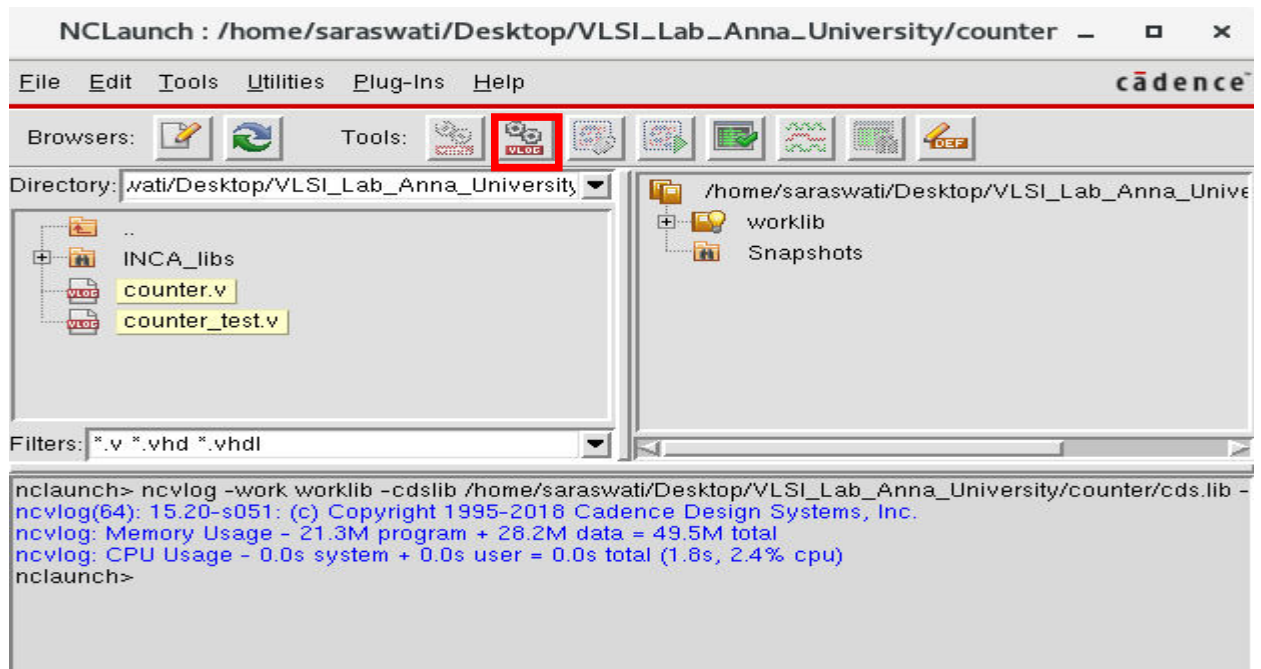
initial $monitor("Time=%t rst=%b clk=%b count=%b", $time,rst,clk,count);

initial
#1400 $finish;

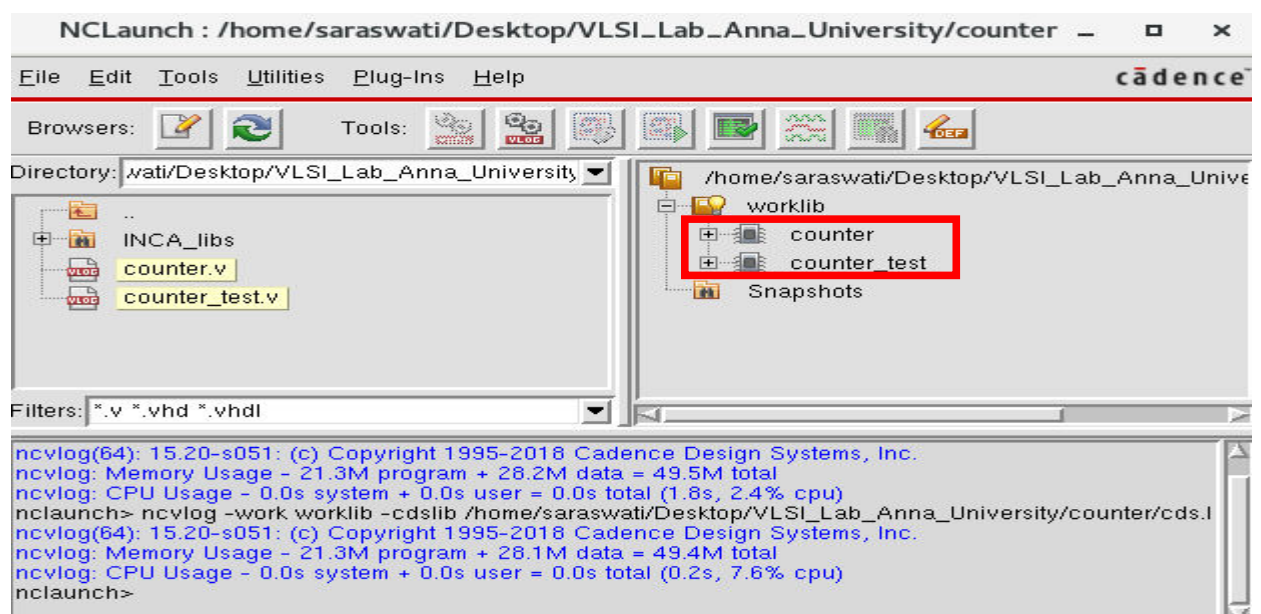
endmodule

```

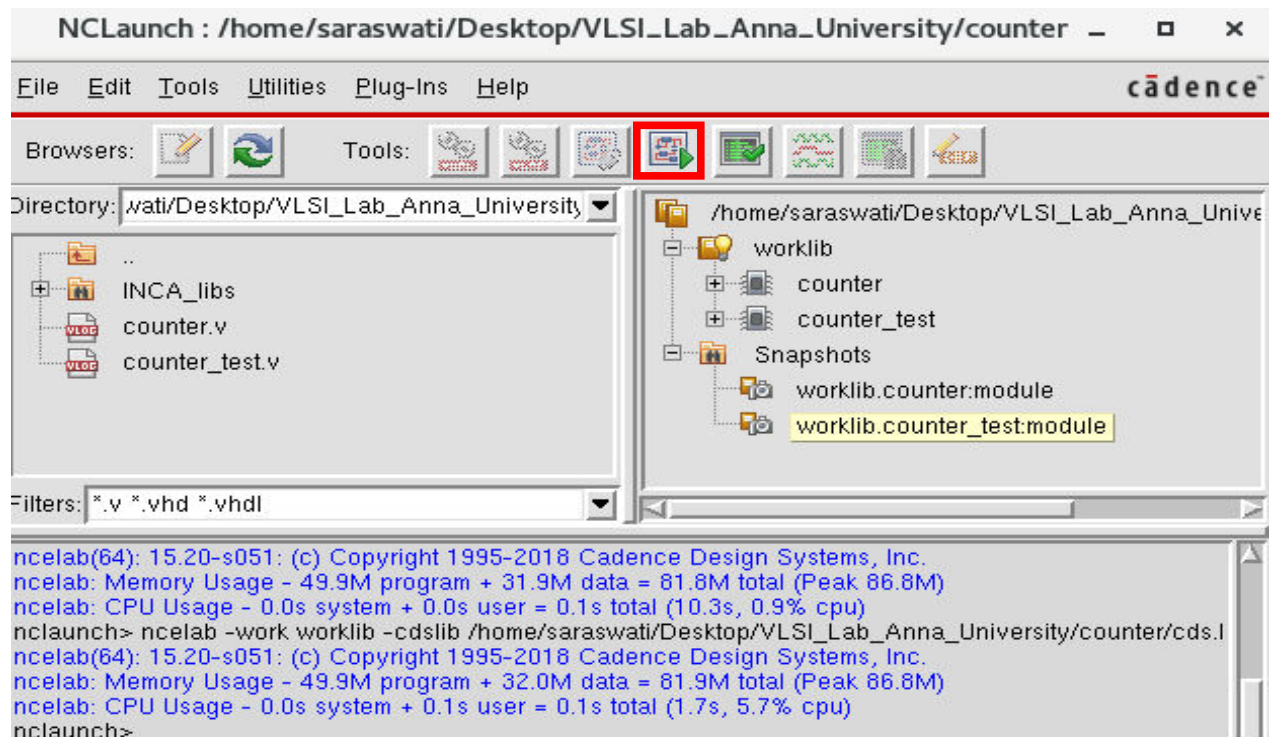
Module 2: Design Simulation using INCISIVE Simulator:



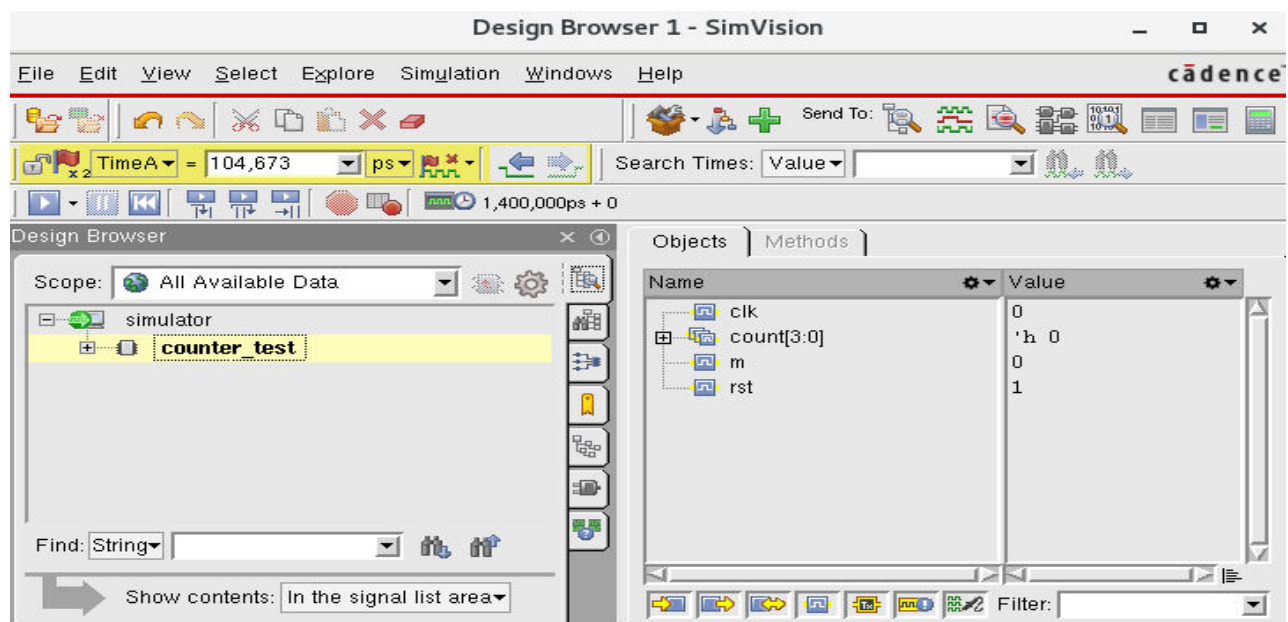
Compiled Data stored in worklib:



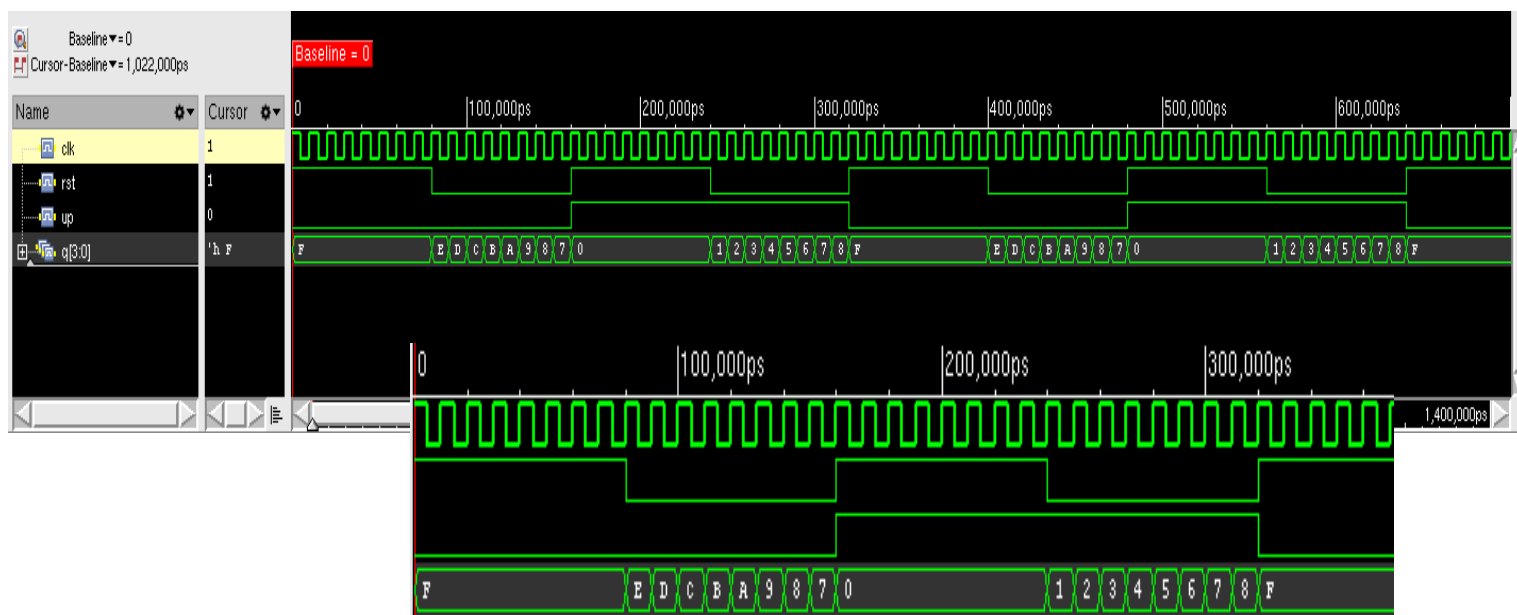
Simulation:



Chose the test scope and make a Right Click on the selected and Select “Send to Waveform Window”.



Simulation Result:



The screenshot shows the Cadence SimVision interface. The title bar is "Console - SimVision". The menu bar includes File, Edit, View, Simulation, Windows, and Help. The toolbar contains various icons for simulation and editing. The console window displays the following text:

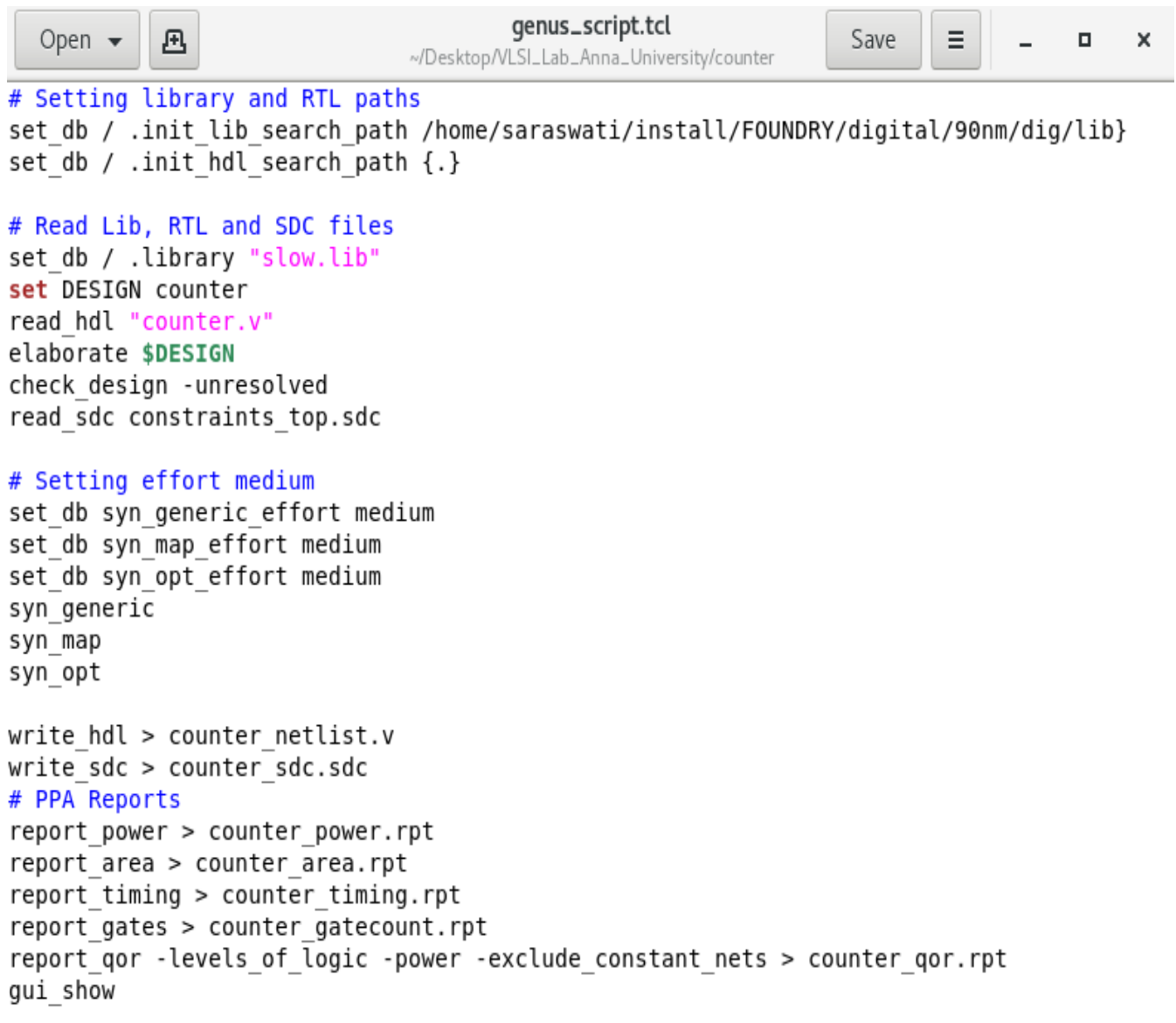
```
ncsim> database -open waves -into waves.shm -default
Created default SHM database waves
ncsim> probe -create -shm counter_test.clk counter_test.count counter_test.m counter_test.rst
Created probe 1
ncsim> run
Time=          0 rst=0 clk=0 count=0000
Time=         5000 rst=0 clk=1 count=0000
Time=        10000 rst=0 clk=0 count=0000
Time=        15000 rst=0 clk=1 count=0000
Time=        20000 rst=0 clk=0 count=0000
Time=        25000 rst=0 clk=1 count=0000
Time=        30000 rst=0 clk=0 count=0000
Time=        35000 rst=0 clk=1 count=0000
Time=        40000 rst=0 clk=0 count=0000
Time=        45000 rst=0 clk=1 count=0000
Time=        50000 rst=0 clk=0 count=0000
Time=        55000 rst=0 clk=1 count=0000
Time=        60000 rst=0 clk=0 count=0000
Time=        65000 rst=0 clk=1 count=0000
Time=        70000 rst=0 clk=0 count=0000
Time=        75000 rst=0 clk=1 count=0000
Time=        80000 rst=0 clk=0 count=0000
Time=        85000 rst=0 clk=1 count=0000
Time=        90000 rst=0 clk=0 count=0000
Time=        95000 rst=0 clk=1 count=0000
Time=       100000 rst=0 clk=0 count=0000
```


Module 3: Synthesis Using Genus Tool

Synthesis runs in following stages :

- Translation
- Mapping
- Optimization

To run the synthesis, the following script can be used. Inside the genus script file we have to mention the commands as shown below.



```

# Setting library and RTL paths
set_db / .init_lib_search_path /home/saraswati/install/FOUNDRY/digital/90nm/dig/lib}
set_db / .init_hdl_search_path {.}

# Read Lib, RTL and SDC files
set_db / .library "slow.lib"
set DESIGN counter
read_hdl "counter.v"
elaborate $DESIGN
check_design -unresolved
read_sdc constraints_top.sdc

# Setting effort medium
set_db syn_generic_effort medium
set_db syn_map_effort medium
set_db syn_opt_effort medium
syn_generic
syn_map
syn_opt

write_hdl > counter_netlist.v
write_sdc > counter_sdc.sdc
# PPA Reports
report_power > counter_power.rpt
report_area > counter_area.rpt
report_timing > counter_timing.rpt
report_gates > counter_gatecount.rpt
report_qor -levels_of_logic -power -exclude_constant_nets > counter_qor.rpt
gui_show
  
```

Chip Level SDC is as follows:

```

constraints_top.sdc
~/Desktop/VLSI_Lab_Anna_University/counter
Save

#-----> clk period = 2ns, f = 500MHz
create_clock -name clk -period 2 -waveform {0 1} [get_ports "clk"]

set_clock_transition -rise 0.1 [get_clocks "clk"]
set_clock_transition -fall 0.1 [get_clocks "clk"]
set_clock_uncertainty 0.01 [get_ports "clk"]

set_input_transition 0.1 [all_inputs]

set_input_delay -max 0.4 -clock clk [all_inputs]
set_output_delay -max 0.4 -clock clk [all_outputs]

set_load 0.2 [all_outputs]
set_max_fanout 20.00 [current_design]
#set_max_capacitance 20 [get_ports]

Plain Text ▾ Tab Width: 8 ▾ Ln 17, Col 1 ▾ INS

```

Invoke the Genus tool by typing the below command on your terminal: **genus -f genus_script.tcl**
 The tcl [Tool Command Language] script runs executing each command one after the other.

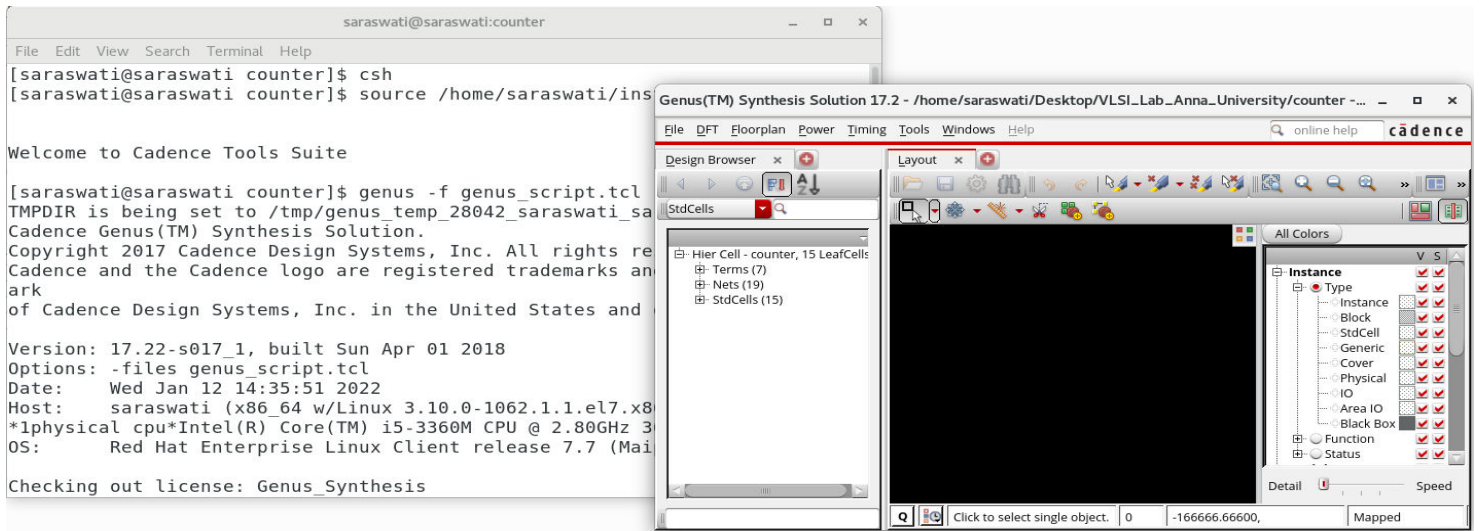
```

saraswati@saraswati:counter
File Edit View Search Terminal Help
[saraswati@saraswati counter]$ csh
[saraswati@saraswati counter]$ source /home/saraswati/install/cshrc

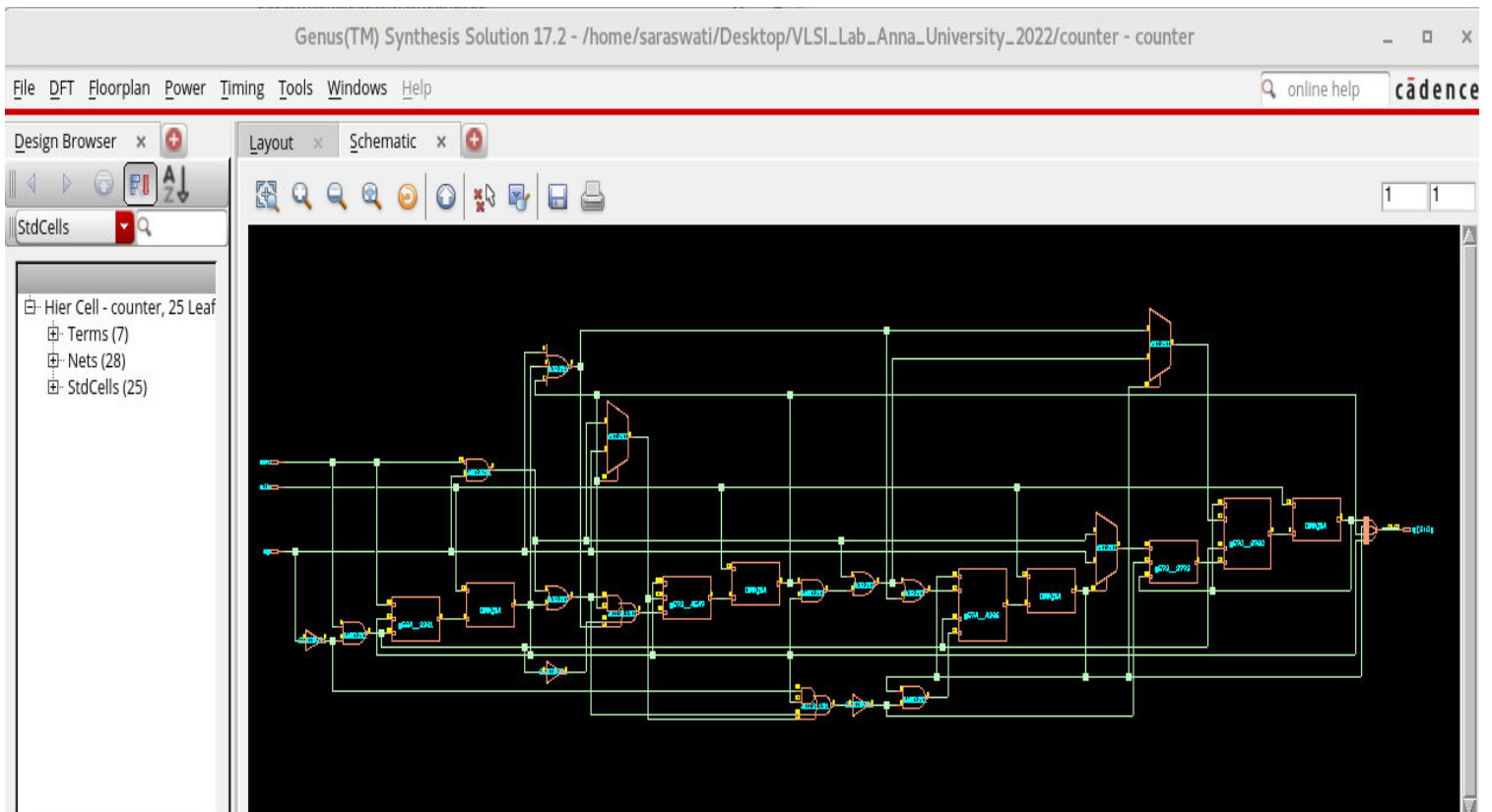
Welcome to Cadence Tools Suite

[saraswati@saraswati counter]$ genus -f genus_script.tcl

```



A window of **Genus GUI** pops – up with the top hier cell on the left top. Make a **Right Click** and select **Schematic Viewer** → **In Main**.



Power Report

```

=====
Generated by:      Genus(TM) Synthesis Solution 17.22-
Generated on:      Mar 14 2022  04:14:40 pm
Module:           counter
Technology library: slow
Operating conditions: slow (balanced_tree)
Wireload mode:    enclosed
Area mode:        timing library
=====

```

		Leakage	Dynamic	Total
Instance	Cells	Power(nW)	Power(nW)	Power(nW)
counter	25	1060.618	211192.463	212253.081

Gate Count Report

```

=====
Module:           counter
Operating conditions: slow (balanced_tree)
Wireload mode:    enclosed
Area mode:        timing library
=====

```

Gate	Instances	Area	Library
AOI211X1	1	5.298	slow
AOI211XL	1	5.298	slow
CLKINVX1	3	6.812	slow
DFFQX4	4	78.718	slow
MXI2XL	3	18.166	slow
NAND2BX1	1	4.541	slow
NAND2XL	3	9.083	slow
NOR2XL	3	9.083	slow
NOR3X1	1	4.541	slow
OAI211X1	2	10.597	slow
OAI21X1	2	9.083	slow
OAI2BB1XL	1	5.298	slow
total	25	166.518	

Type	Instances	Area	Area %
sequential	4	78.718	47.3
inverter	3	6.812	4.1
logic	18	80.988	48.6
physical_cells	0	0.000	0.0
total	25	166.518	100.0

Area Report:

```

=====
Generated by:      Genus(TM) Synthesis Solution 17.22-s017_1
Generated on:      Mar 14 2022  04:14:40 pm
Module:            counter
Technology library: slow
Operating conditions: slow (balanced_tree)
Wireload mode:     enclosed
Area mode:         timing library
=====

```

Instance	Module	Cell Count	Cell Area	Net Area	Total Area	Wireload
counter		25	166.518	0.000	166.518	<none> (D)

(D) = wireload is default in technology library

Timing Report:

```

-----
Module:            counter
Operating conditions: slow (balanced_tree)
Wireload mode:     enclosed
Area mode:         timing library
=====

```

Path 1: MET (595 ps) Setup Check with Pin q_reg[3]/CK->D

```

Group: clk
Startpoint: (R) q_reg[1]/CK
Clock: (R) clk
Endpoint: (F) q_reg[3]/D
Clock: (R) clk

          Capture          Launch
Clock Edge:+ 2000          0
Src Latency:+ 0            0
Net Latency:+ 0 (I)        0 (I)
Arrival:=    2000          0

```

```

Setup:- 123
Uncertainty:- 10
Required Time:= 1867
Launch Clock:- 0
Data Path:- 1272
Slack:= 595

```

#	Timing Point	Flags	Arc	Edge	Cell	Fanout	Load (fF)	Trans (ps)	Delay (ps)	Arrival (ps)	Instance Location
#	q_reg[1]/CK	-	-	R	(arrival)	4	-	100	-	0	(-, -)
#	q_reg[1]/Q	-	CK->Q	F	DFFQX4	5	209.8	376	605	605	(-, -)
#	g583__7675/Y	-	S0->Y	F	MXI2XL	2	4.5	222	241	846	(-, -)
#	g578__9682/Y	-	C0->Y	R	A0I211X1	1	2.7	157	158	1003	(-, -)
#	g577/Y	-	A->Y	F	CLKINVX1	2	4.4	72	65	1068	(-, -)
#	g572__3772/Y	-	A1->Y	R	OAI21X1	1	2.9	134	93	1161	(-, -)
#	g570__8780/Y	-	C0->Y	F	OAI211X1	1	1.6	149	111	1272	(-, -)
#	q_reg[3]/D	<<<	-	F	DFFQX4	1	-	-	0	1272	(-, -)

Pre Layout & Post Layout Simulation/Gate Level Simulation

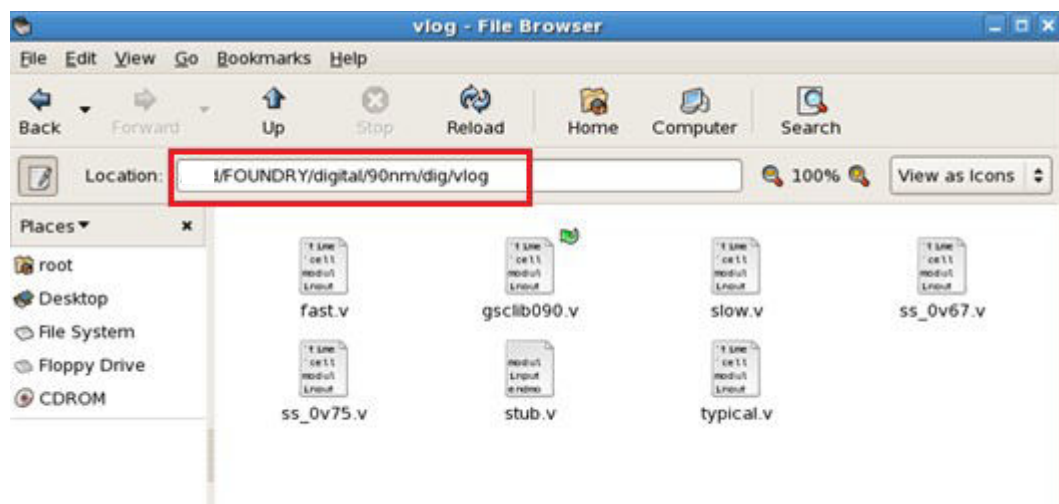
Overview:

Gate-level simulation (GLS) is the simulation of a netlist, with or without gate delays, and takes place after the synthesis of the RTL code, or post-P&R (placement & routing). RTL simulation, which is the simulation of RTL code, aims to check and verify the functionality and the source, which can be described in one or more HDL languages.

Gate-level simulation is used to check and ensure that the desired functionality is not lost in the translation from high-level RTL to low-level gate implementation, and also when timing is enabled, to ensure timing delays do not break the intended functionality.

Module 4: Post Synthesis Simulation/ Pre Layout Simulation:

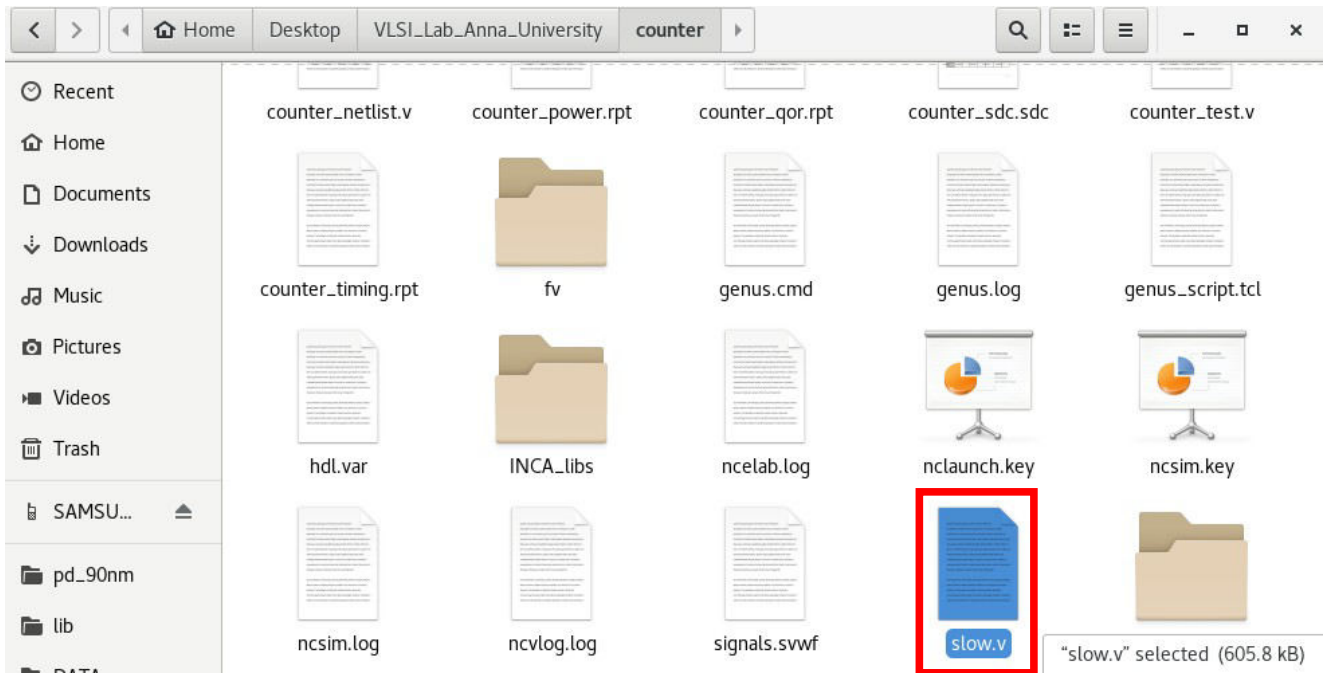
Post synthesis simulation using nclaunch. Using the netlist we can do the simulation so that we can see the delay for that we have to include the library file **slow.v**.



You can get the Verilog extension library files from

/home/install/FOUNDRY/digital/90nm/dig/vlog/slow.v

Copy that file and paste inside the counter folder or directory.



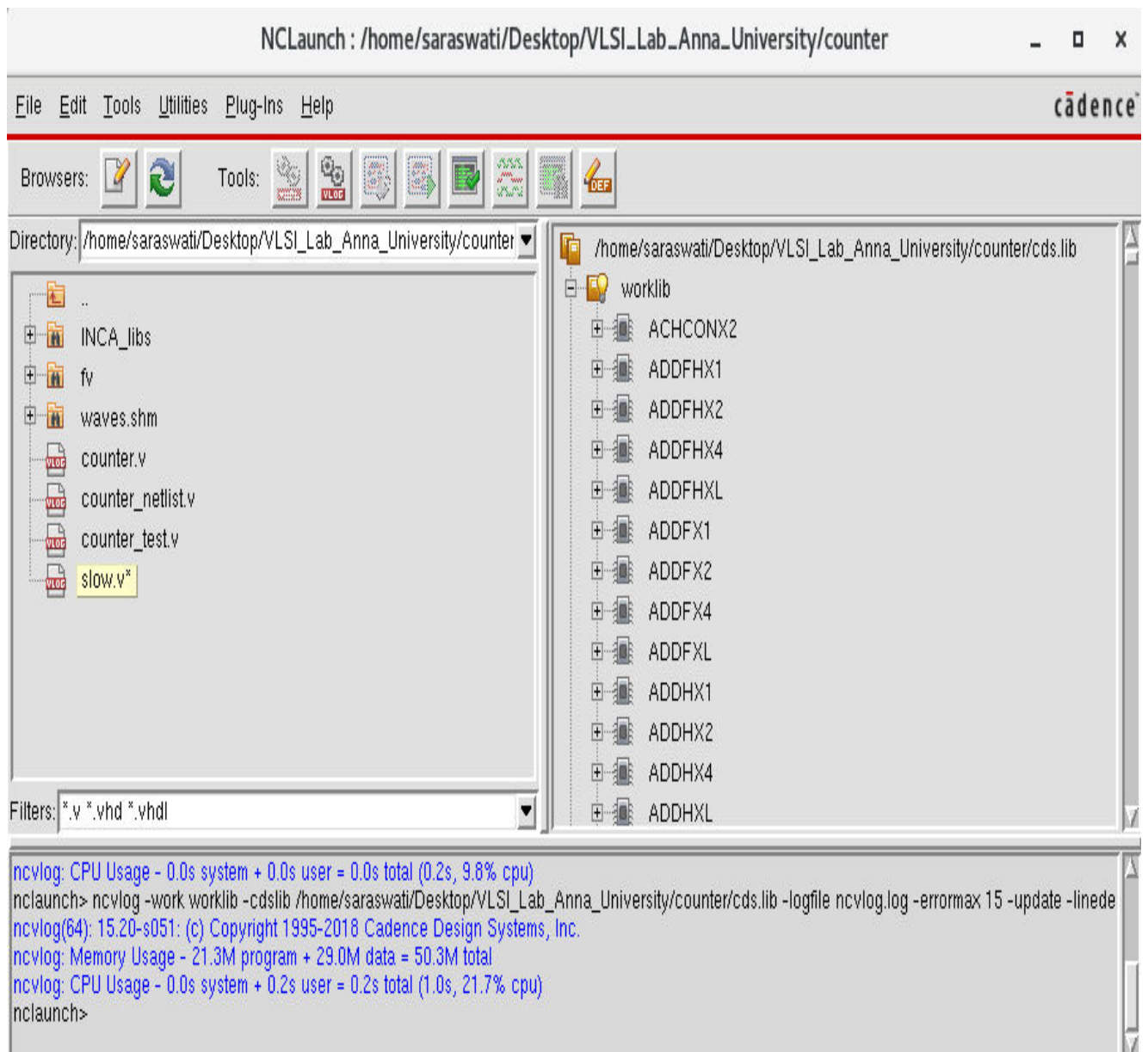
Right Click and open in terminal invoke the **nclaunch**. Compile the codes and elaborate it

Note: In Test bench and **Netlist** files we have to mention the **Timescale**

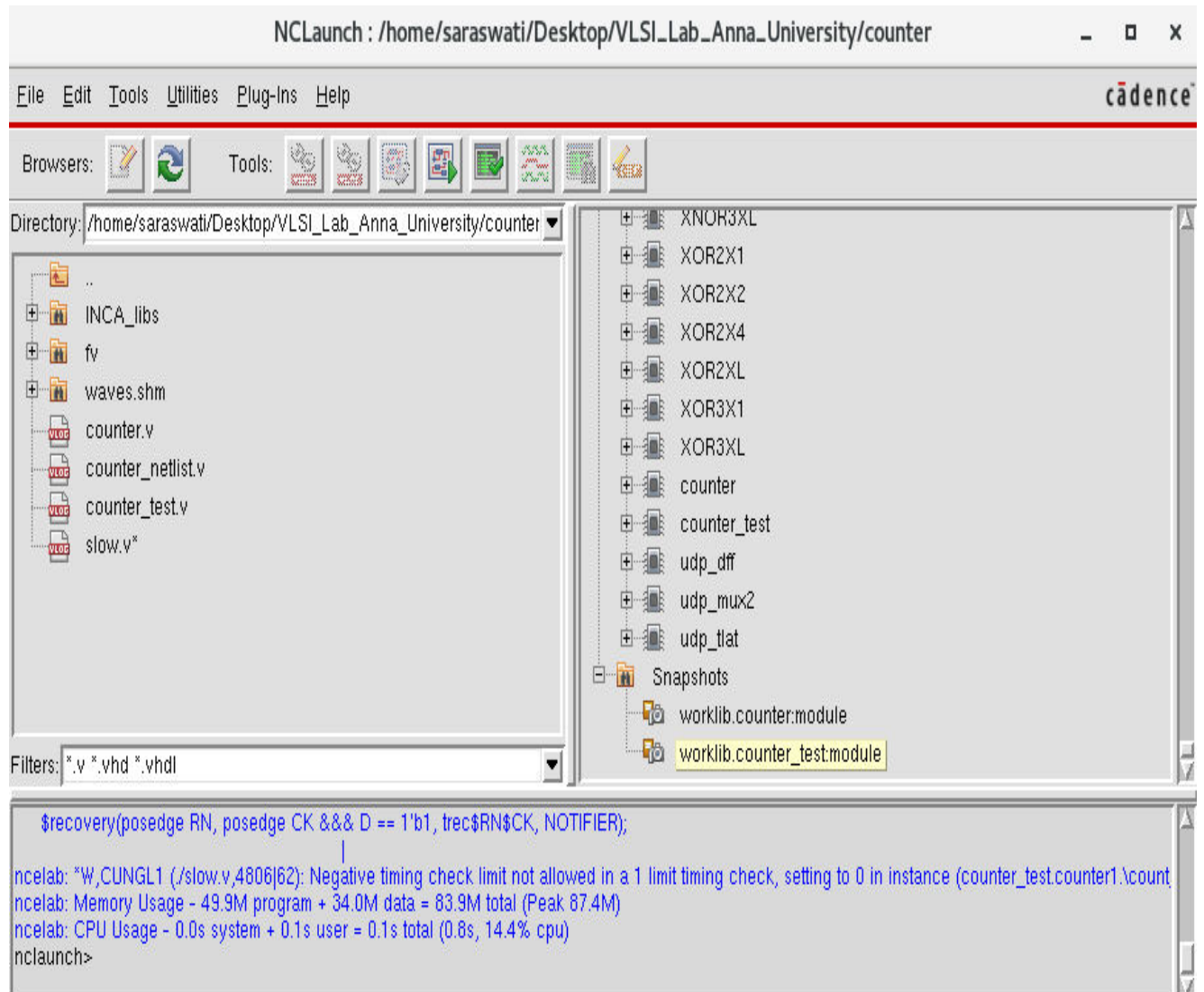
```
saraswati@saraswati:counter
File Edit View Search Terminal Help
[saraswati@saraswati counter]$ csh
[saraswati@saraswati counter]$ source /home/saraswati/install/cshrc

Welcome to Cadence Tools Suite

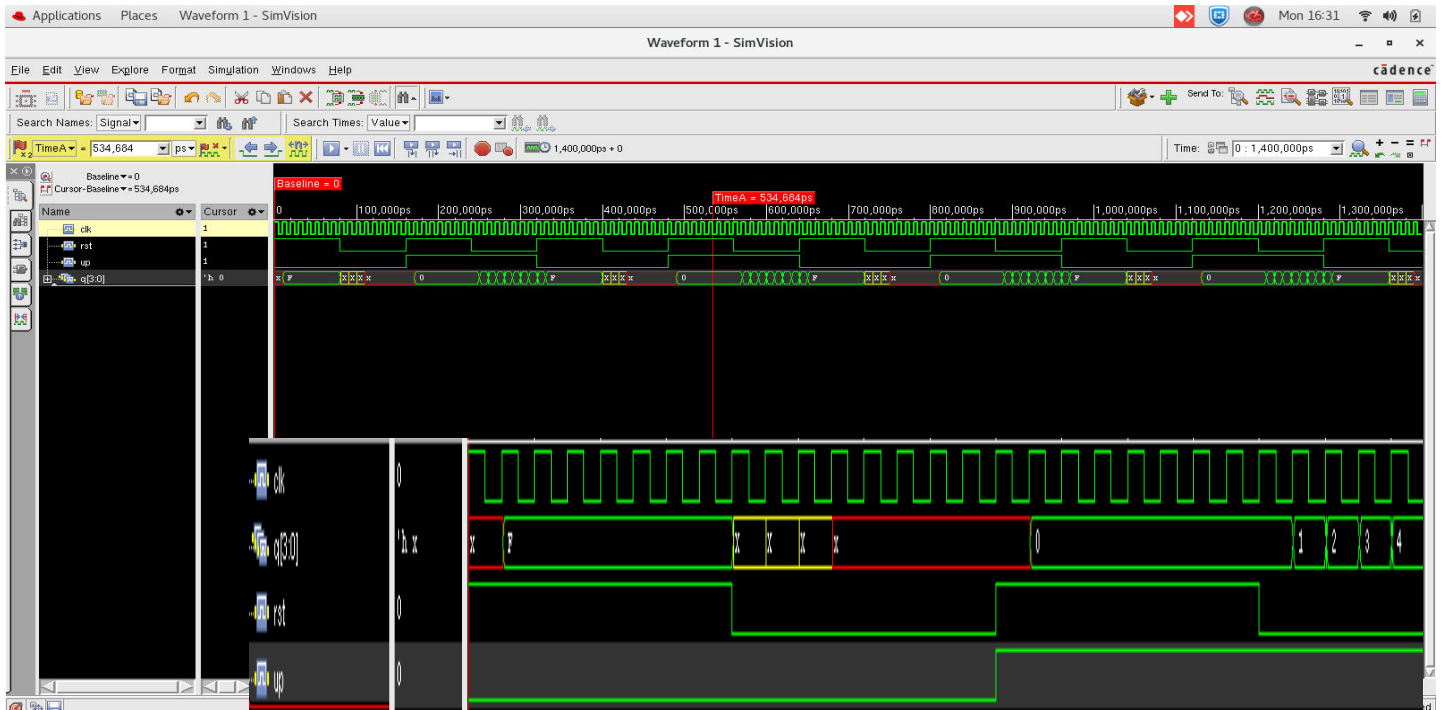
[saraswati@saraswati counter]$ nclaunch
```

After compile the technology library you will get the window like above and then Elaborate and Simulate the test bench



You can see the waveform like below window with Delay (for better view you can make use of zoom in option)



For checking the pre layout simulation or post synthesis simulation in batch mode or through terminal you can go with command given below.

irun -clean counter_netlist.v counter_test.v -mess -access +rwc -v slow.v -timescal 1ns/1ps -gui

Module 5: Physical Design Implementation/Automatic Layout Generation

Mandatory Inputs for PD:

1. Gate Level Netlist [Output of Synthesis]
2. Block Level SDC [Output of Synthesis]
3. Liberty Files (.lib)
4. LEF Files (Library Exchange Format)

Expected Outputs from PD:

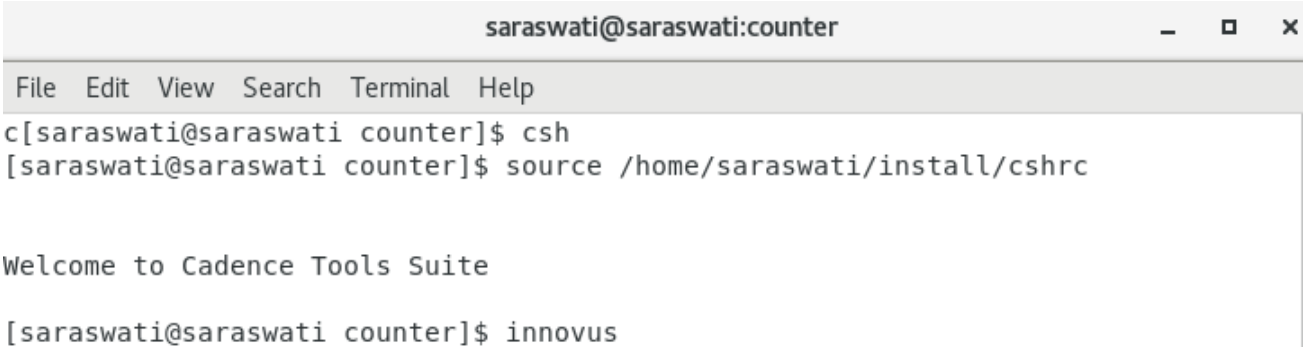
1. GDS II File (Graphical Data Stream for Information Interchange)
2. SDF (Standard Delay Format file)
3. SPEF (Standard Parasitic Extraction Format file)

Physical Design implementation involves 5 stages - Floor Planning, Power Planning, Placement, CTS (Clock Tree Synthesis) and Routing. You will use INNOVUS Implementation tool to implement the floor planning, placement, routing etc.

Importing the Design:

In this section, you import a gate-level netlist and libraries into the Innovus Implementation System.

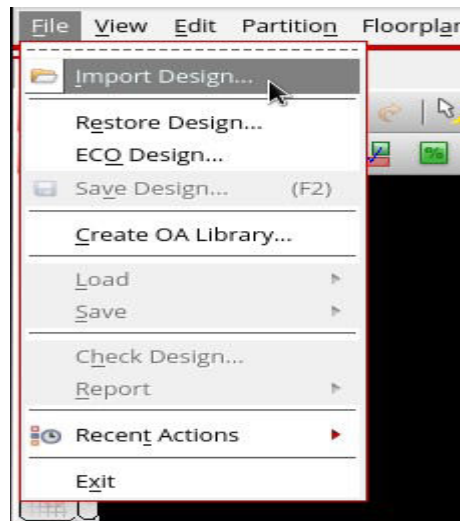
1. Open the “counter” directory and make a Right Click to “**Open in Terminal**”.
2. Before invoking any tool, invoke C shell by typing “**csh**” in terminal.
3. Source the cshrc file by typing ‘source <cshrc file>’ e.g.: - **source /home/install/cshrc**
4. Invoke the tool by typing “**innovus**” in the terminal. Default Innovus Graphical User Interface is visible.
3. To import a gate-level netlist, timing constraints, and libraries, choose **File – Import Design**.



```
saraswati@saraswati:counter
File Edit View Search Terminal Help
c[saraswati@saraswati counter]$ csh
[saraswati@saraswati counter]$ source /home/saraswati/install/cshrc

Welcome to Cadence Tools Suite

[saraswati@saraswati counter]$ innovus
```



The Design Import form appears.

Design Import

Netlist:

☒ Verilog
Files: ...

☐ OA
Library:
Cell:
View:

Top Cell: ☐ Auto Assign ☒ By User:

Technology/Physical Libraries:

☒ OA
Reference Libraries: ...
Abstract View Names:
Layout View Names:
☐ LEF Files ...

Floorplan

IO Assignment File: ...

Power

Power Nets:
Ground Nets:
CPF File: ...

Analysis Configuration

MMMC View Definition File: ...

Create Analysis Configuration ...

OK **Save...** **Load...** **Cancel** **Help**

5. In a different xterm window than where you brought up the tool, view the **counter.globals** file.

Notice the design and library files that are specified.

```
#####
#  Generated by:      Cadence Innovus 17.14-s077_1
#  OS:               Linux x86_64(Host ID saraswati)
#  Generated on:      Tue Jan 10 19:06:11 2022
#  Design:
#  Command:           save_global counter.globals
#####
#
# Version 1.1
#

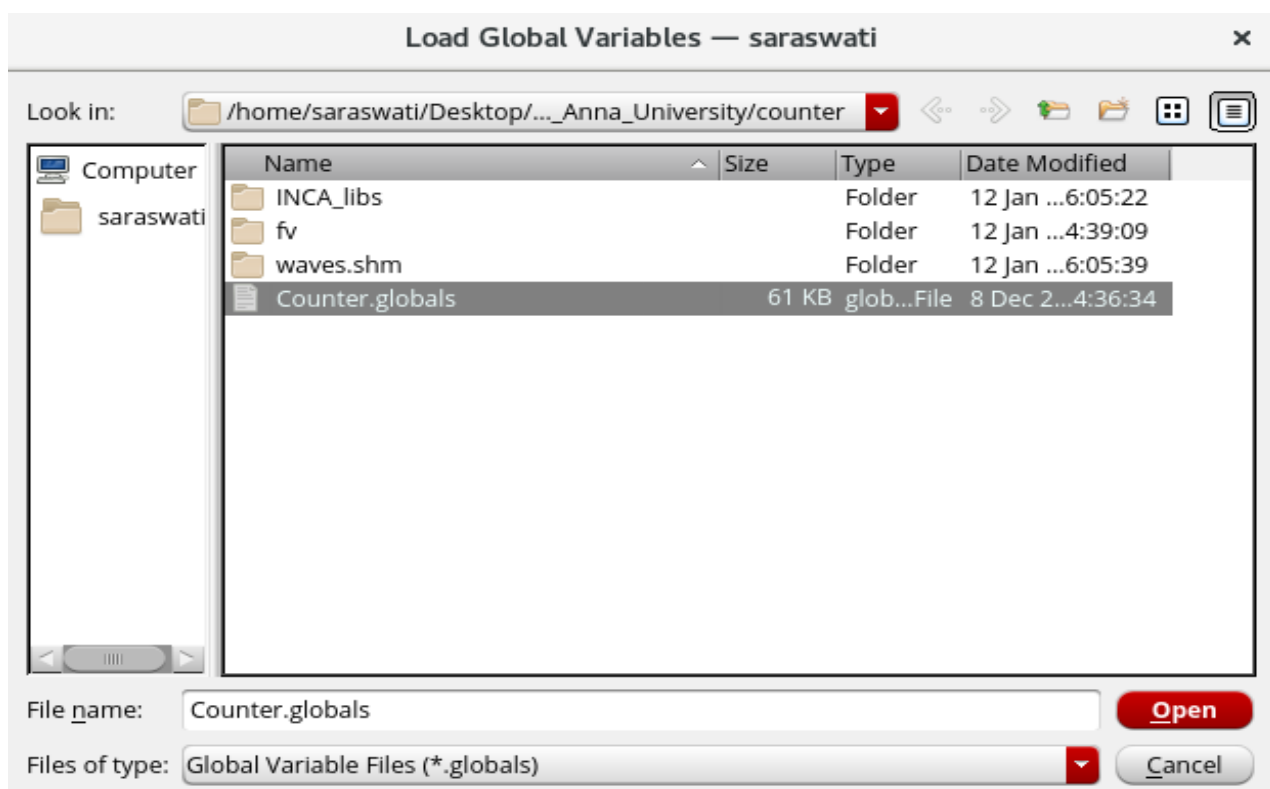
set ::TimeLib::tsgMarkCellLatchConstructFlag 1
set conf_qxconf_file {NULL}
set conf_qxlib_file {NULL}
set defHierChar {/}
set distributed_client_message_echo {1}
set distributed_mmmc_disable_reports_auto_redirection {0}
set enc_enable_print_mode_command_reset_options 1
set init_design_settop 0
set init_gnd_net {VSS}
set init_lef_file {/home/saraswati/install/FOUNDRY/digital/90nm/dig/lef/
gsclib090_tech.lef /home/saraswati/install/FOUNDRY/digital/90nm/dig/lef/
gsclib090_macro.lef}
set init_mmmc_file {counter.view}
set init_pwr_net {VDD}
set init_verilog {counter_netlist.v}
set latch_time_borrow_mode max_borrow
set pegDefaultResScaleFactor 1
set pegDetailResScaleFactor 1
set report_inactive_arcs_format {from to when arc_type sense reason}
set soft_stack_size_limit {31}
```

6. Next, view the **counter.view** file. This file specifies the files for timing analysis and extraction.

```
# Version:1.0 MMMC View Definition File
# Do Not Remove Above Line
create_rc_corner -name default_rc_corner -cap_table {/home/saraswati/install/FOUNDRY/digital/90nm/dig/captable/gpdk090.lef.extended.CapTbl} -T {125} -preRoute_res {1.0} -preRoute_cap {1.0} -preRoute_clkres {0.0} -preRoute_clkcap {0.0} -postRoute_res {1.0} -postRoute_cap {1.0} -postRoute_xcap {1.0} -postRoute_clkres {0.0} -postRoute_clkcap {0.0} -qx_tech_file {/home/saraswati/install/FOUNDRY/digital/90nm/dig/qrc/gpdk090_9l.tch}
create_library_set -name min_timing_lib -timing {/home/saraswati/install/FOUNDRY/digital/90nm/dig/lib/fast.lib}
create_library_set -name max_timing_lib -timing {/home/saraswati/install/FOUNDRY/digital/90nm/dig/lib/slow.lib}
create_constraint_mode -name constraints_sdc -sdc_files {counter_sdc.sdc}
create_delay_corner -name max_delay_corner -library_set {max_timing_lib} -rc_corner {default_rc_corner}
create_delay_corner -name min_delay_corner -library_set {min_timing_lib} -rc_corner {default_rc_corner}
create_analysis_view -name wc -constraint_mode {constraints_sdc} -delay_corner {max_delay_corner}
create_analysis_view -name bc -constraint_mode {constraints_sdc} -delay_corner {min_delay_corner}
set_analysis_view -setup {wc} -hold {bc}
```

7. Click the Load button in the Design Import form, the Load Global Variables window will appear.

8. Select the **counter.globals** file. Click Open.



The Design Import form will be populated by the parameters specified in the **counter.globals** file.

Design Import — saraswati
_ □ ×

Netlist:

☒ Verilog

Files: ...

Top Cell: ☒ Auto Assign ☐ By User:

☐ OA

Library:

Cell:

View:

Technology/Physical Libraries:

☐ OA

Reference Libraries: ...

Abstract View Names:

Layout View Names:

☒ LEF Files ...

Floorplan

IO Assignment File:

Power

Power Nets:

Ground Nets:

CPF File:

Analysis Configuration

MMMC View Definition File:

Create Analysis Configuration ...

OK
Save...
Load...
Cancel
Help

When you load the file, it fills in the fields in the Design Import window. The technology files and the counter.view file for the labs are displayed. The counter.view file contains pointers to the timing library, LEF and the constraints files.

9. To load the design and libraries, in the Design Import window, click OK.
Here is a brief description of the fields in the Design Import window.

Field	Description
Verilog files	Contains the names of gate-level Verilog netlist files.
LEF files	Library of components and physical data for the components in LEF format. Also contains routing layers and DRC rules.
MMMC View Definition file, <i>counter.view</i>	Contains pointers to timing libraries, technology files for extraction, and SDC constraints files.

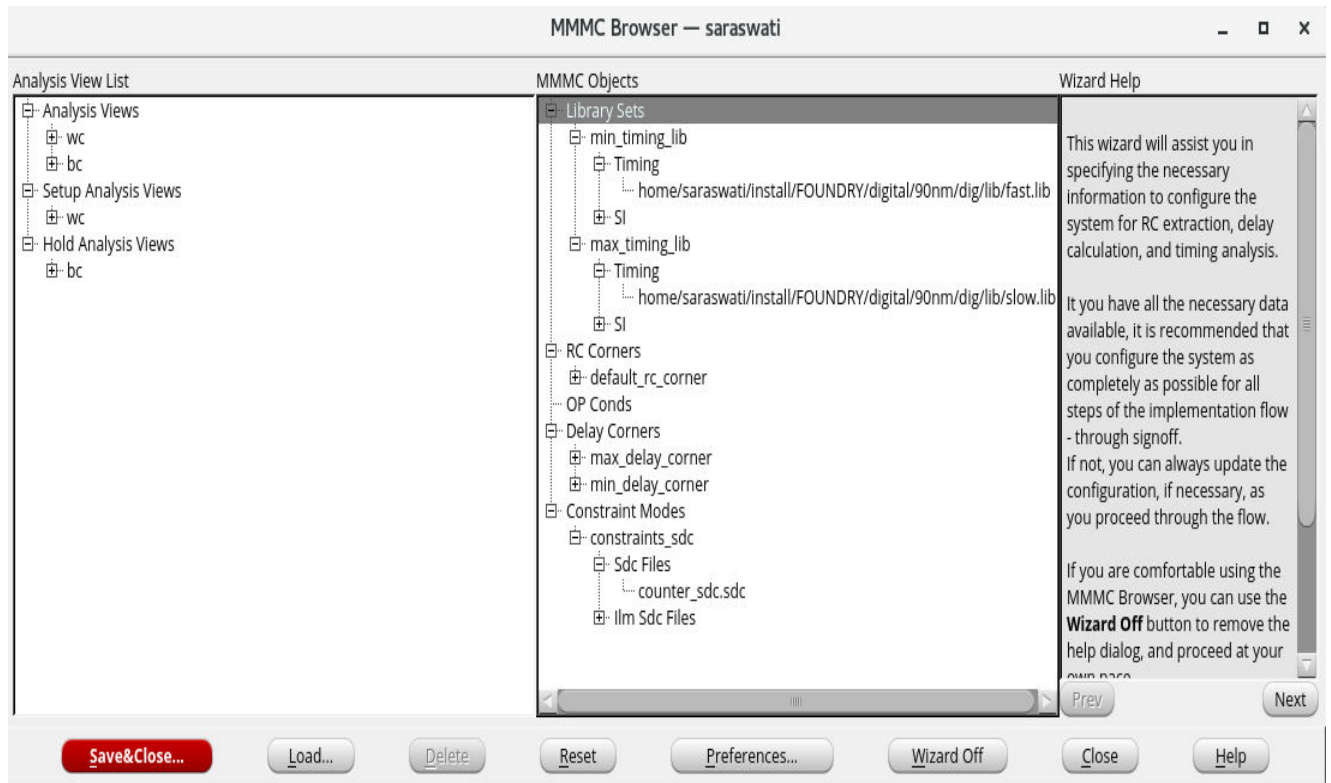
Viewing the Timing Setup and Design Import Results:

In this section, you learn how to view the MMMC settings, details about the objects on the screen, and how to view and interpret what you see in the design window.

1. Select Timing – MMMC Browser.

The MMMC Browser is a useful tool to debug the view file. Often, the view file is created manually and is therefore prone to errors.

2. Expand the parameters under Analysis View List and MMMC Objects.



Notice that the settings match the ones in the **counter.view** file.

3. Close the MMMC browser.

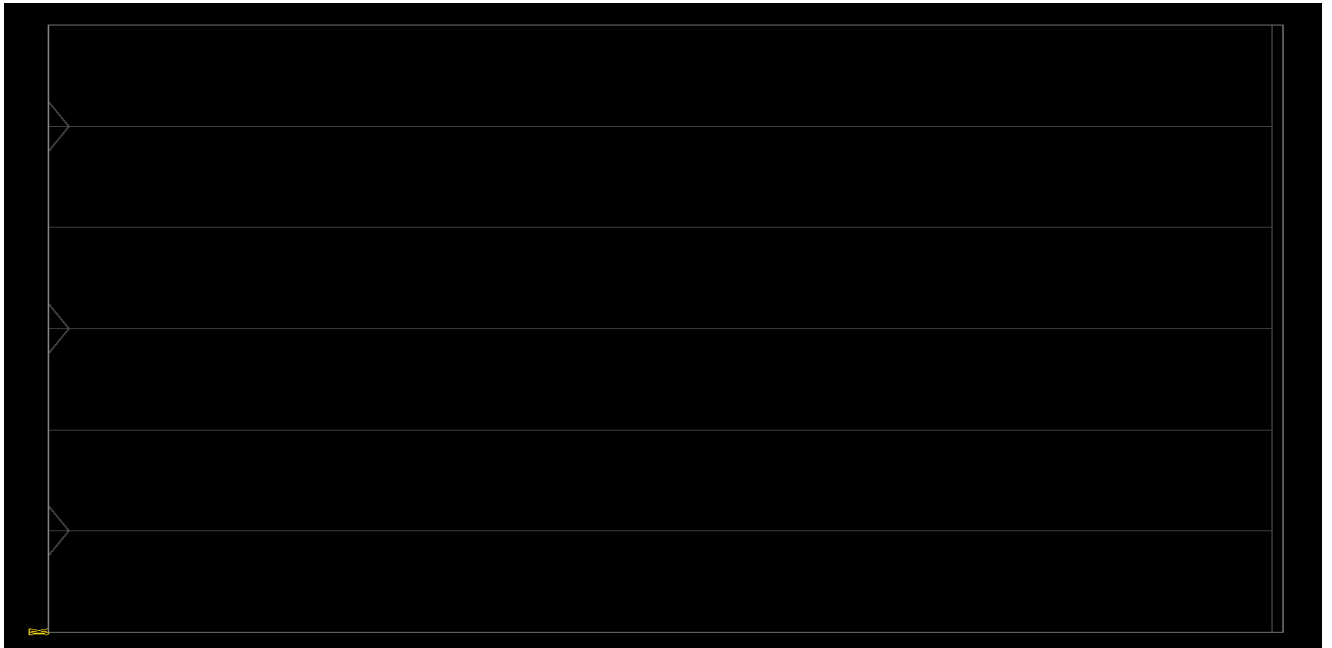
4. To enlarge the window, drag the corner of the window until you can see all the modules in your design, as well as all the Innovus menus.

5. Select the **Floorplan view** by clicking the Floorplan view icon

.

6. **Fit** the design to the window by pressing the **f** key.

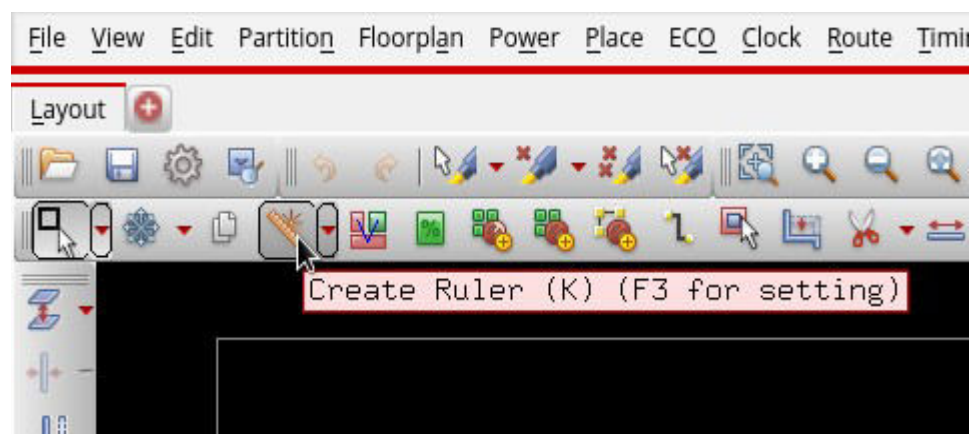
7. Zoom out by pressing Shift-Z or by clicking the Zoom Out icon



8. Expand the design window to display all the available pull-down menus.

9. To view more of the objects, click the Zoom Out icon or press Shift-Z.

10. Move your cursor over the icons and notice that their functions are displayed in text boxes, as shown here.

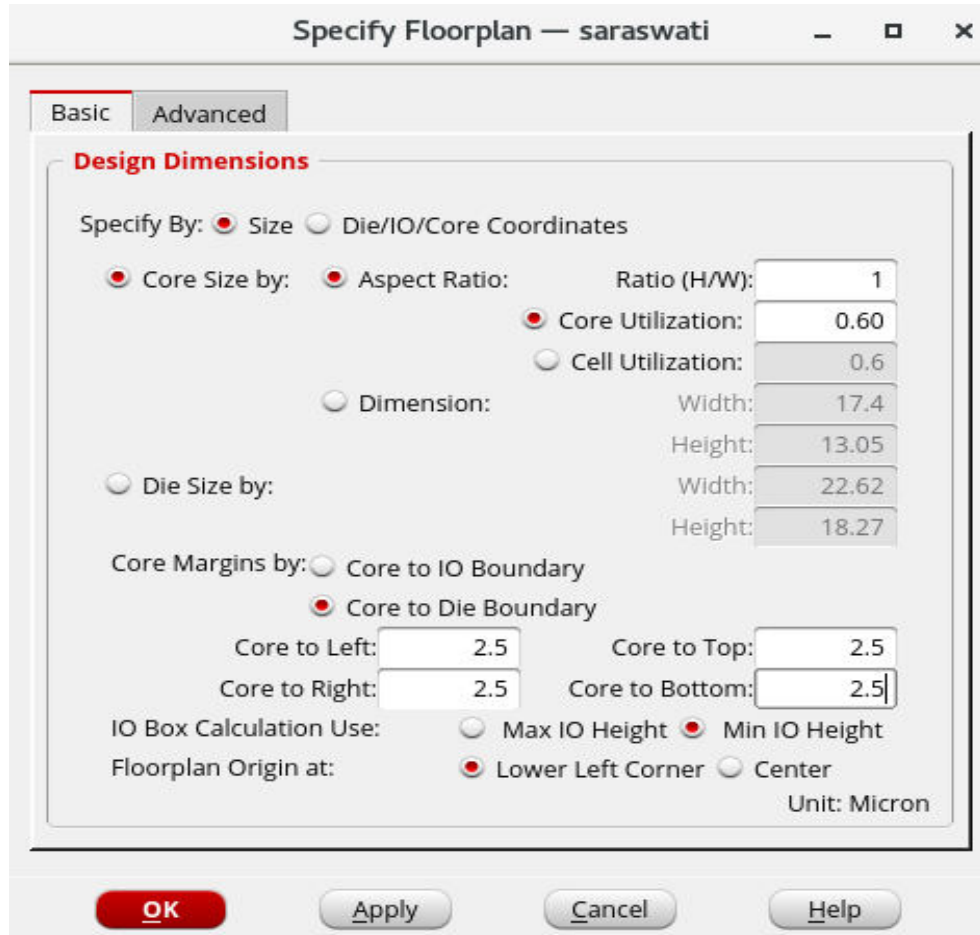


11. To zoom to a particular area, press and drag the right mouse button over a rectangular area. The window zooms to that area.

Module 5.1: Floor planning:

In this section, you create a floorplan. You become familiar with the floorplanning forms and icons.

1. Choose Floorplan – Specify Floorplan. The Specify Floorplan form appears.



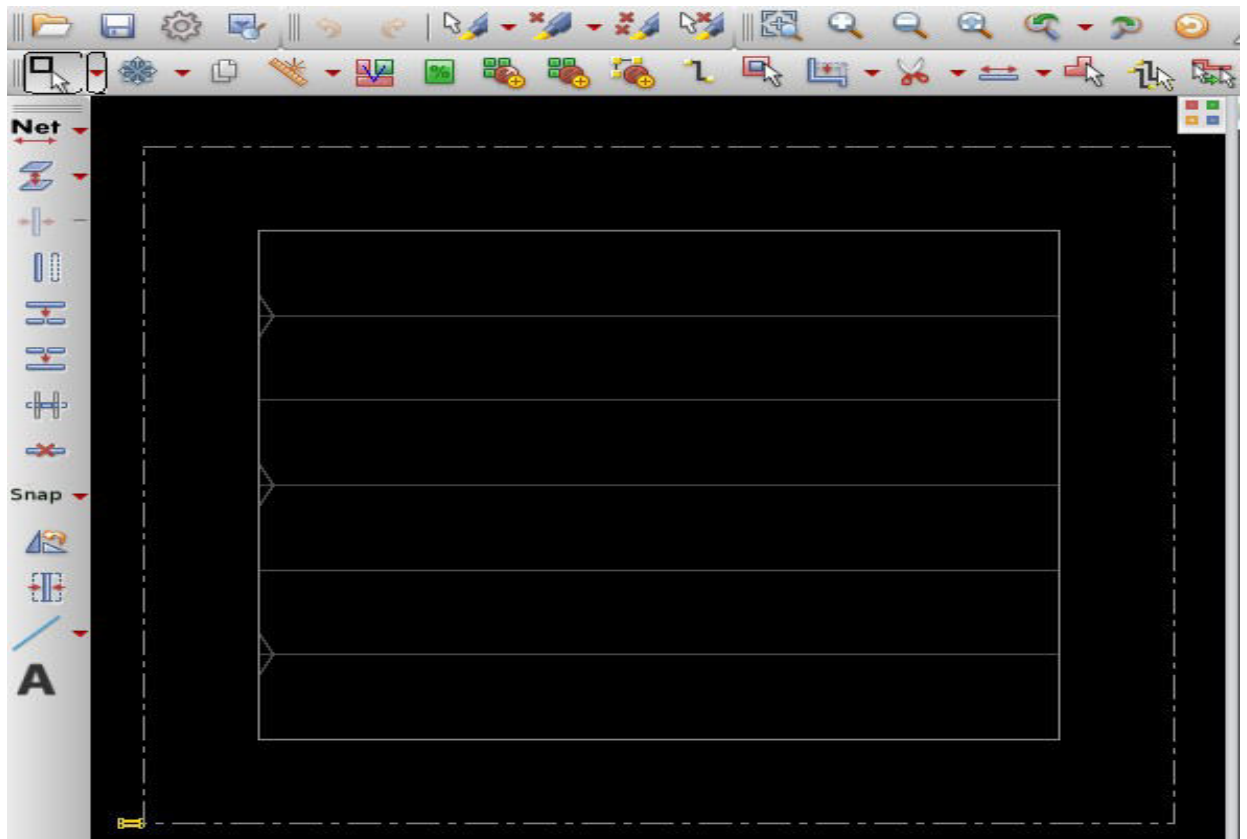
The image shows a screenshot of the 'Specify Floorplan' dialog box, titled 'Specify Floorplan — saraswati'. It has two tabs: 'Basic' and 'Advanced'. The 'Basic' tab is selected. The dialog is divided into sections for 'Design Dimensions' and 'Core Margins by:'. In the 'Design Dimensions' section, 'Specify By:' has 'Size' selected. Under 'Core Size by:', 'Aspect Ratio:' is selected with a value of 1. 'Core Utilization:' is set to 0.60, and 'Cell Utilization:' is set to 0.6. 'Dimension:' is unselected, showing Width: 17.4 and Height: 13.05. 'Die Size by:' is unselected, showing Width: 22.62 and Height: 18.27. In the 'Core Margins by:' section, 'Core to Die Boundary' is selected. The margins are set to 2.5 for Core to Left, Core to Right, Core to Top, and Core to Bottom. 'IO Box Calculation Use:' has 'Min IO Height' selected. 'Floorplan Origin at:' has 'Lower Left Corner' selected. The unit is set to 'Micron'. At the bottom, there are buttons for 'OK', 'Apply', 'Cancel', and 'Help'.

a. For aspect ratio, enter 1.

b. Select Core to Die Boundary.

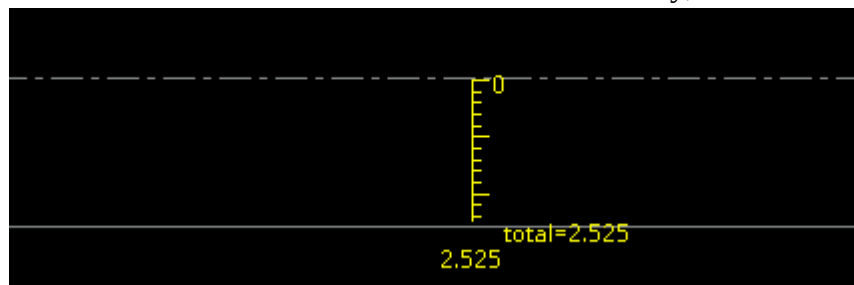
c. Enter 2.5 for Core to Left, Core to Right, Core to Top and Core to Bottom values. To display more information about the options, on the Specify Floorplan form, click Help.

d. To initialize the floorplan, click OK.



The floorplan you will see should look like the above screenshot.

2. To measure the distance between the core area and the I/O boundary, click the Ruler icon or press k.



3. To delete the ruler, click the Clear All ruler icon or press Shift+K.
4. To deselect ruler mode, click the Select By Box icon or press a.

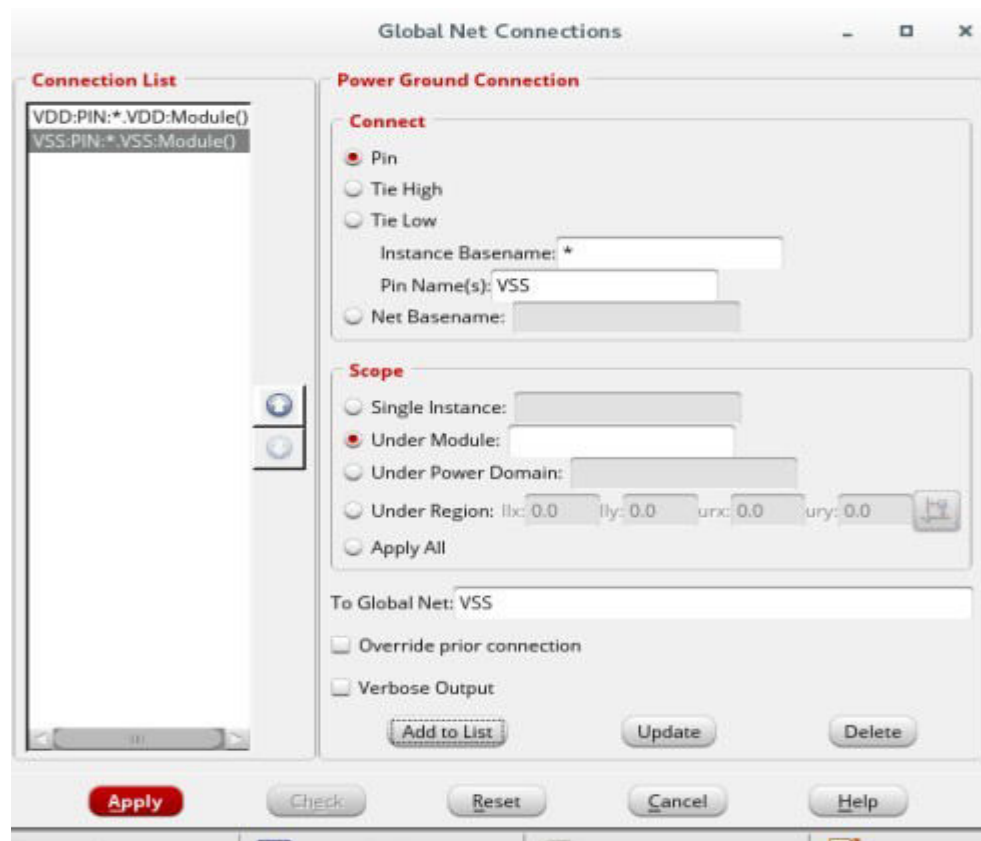
Module 5.2: Power Planning

1. Connect Global Net Connects
2. Adding Power Rings
3. Adding Power Strings
4. Special Route

During the stage of Importing Design, under the Globals file, Two command lines state the names of Power and Ground Nets. However, in order to Current flow through these Power nets, they are to converge at a point, preferably a common net connected to a Pin.

Under Connect Global Net Connects, we create two pins, one for VDD and one for VSS connecting them to corresponding Global Nets as mentioned in Globals file.

1. Select Power → Connect Global Nets.. to create **“Pin”** and **“Connect to Global Net”** as shown and use **“Add to list”**. Click on **“Apply”** to direct the tool in enforcing the Pins and Net connects to Design and then Close the window.



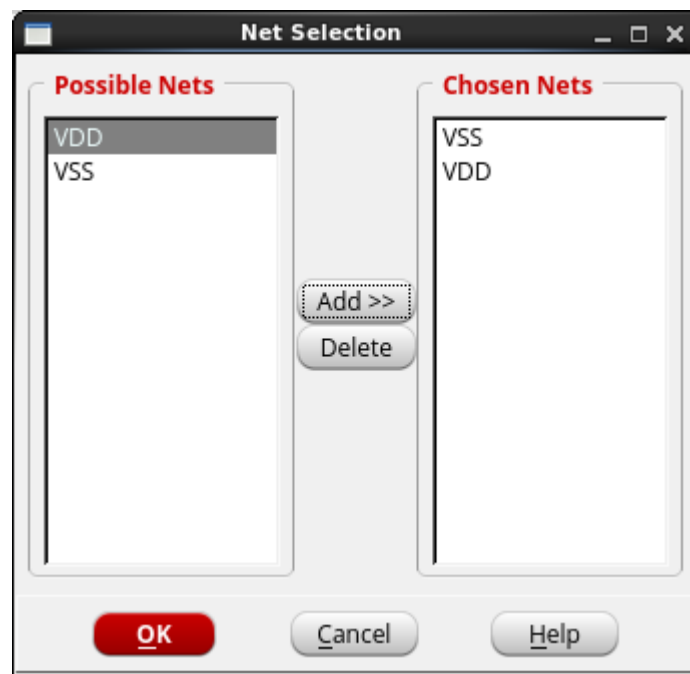
In order to Tap in Power from a distant Power supply, Wider Nets and Parallel connections improve efficiency. Moreover, the cells that would be placed inside the core area are expected to have shorter Nets for lower resistance.

Hence Power Rings [Around Core Boundary] and Power Stripes [Across Core Boundary] are added which satisfies the above conditions.

2. Choose **Power-Power Planning** – “**Add Ring**”. To add Power rings ‘around Core Boundary’.

The Add Rings form is displayed. There are 9 metal layers available for routing in the horizontal and vertical directions and you will be selecting some of them for creating rings in subsequent steps.

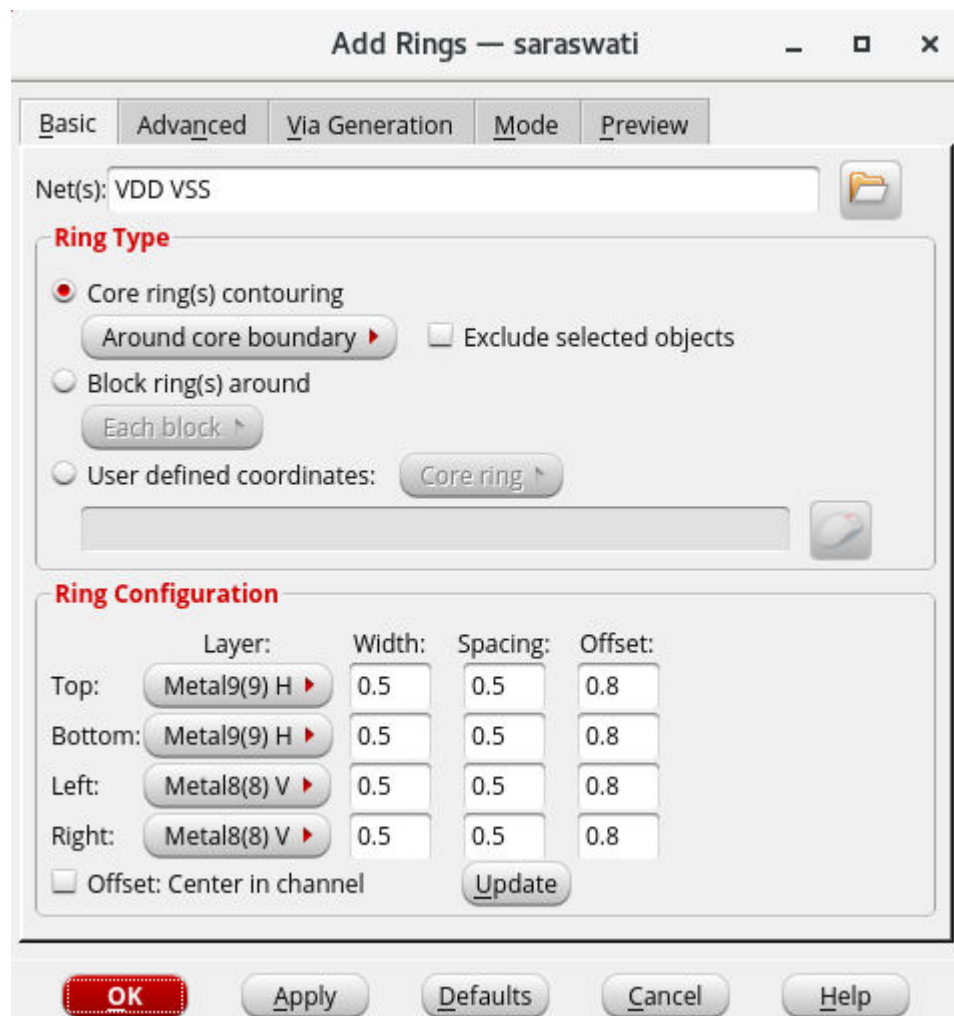
a. To select the VDD and VSS nets, click the icon in the Net(s) field. The Net Selection form is displayed.



b. In the Possible Nets pane, press Shift and VDD and VSS.

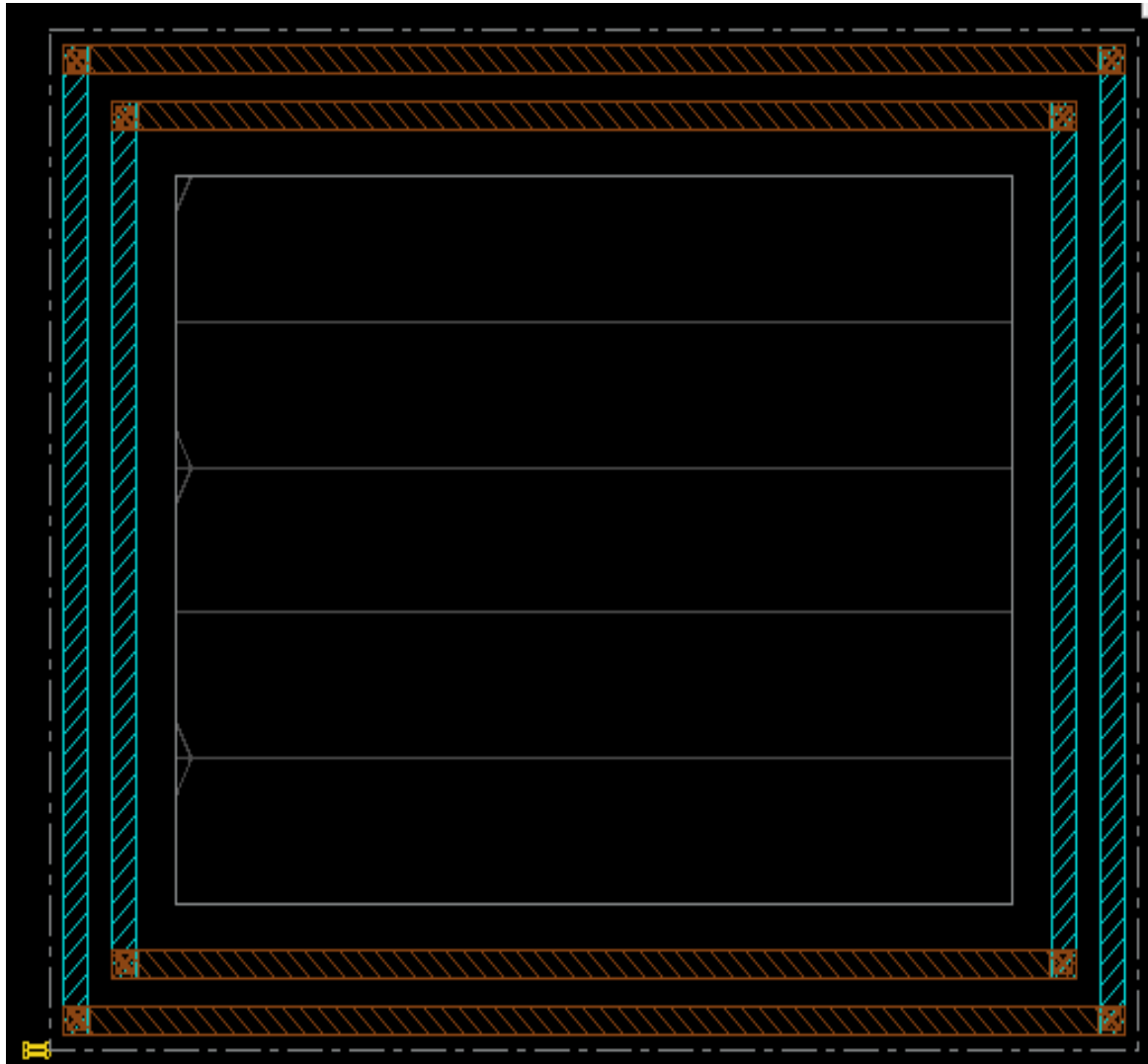
c. Click Add. The selected nets appear in the Chosen Nets pane.

d. Click OK.

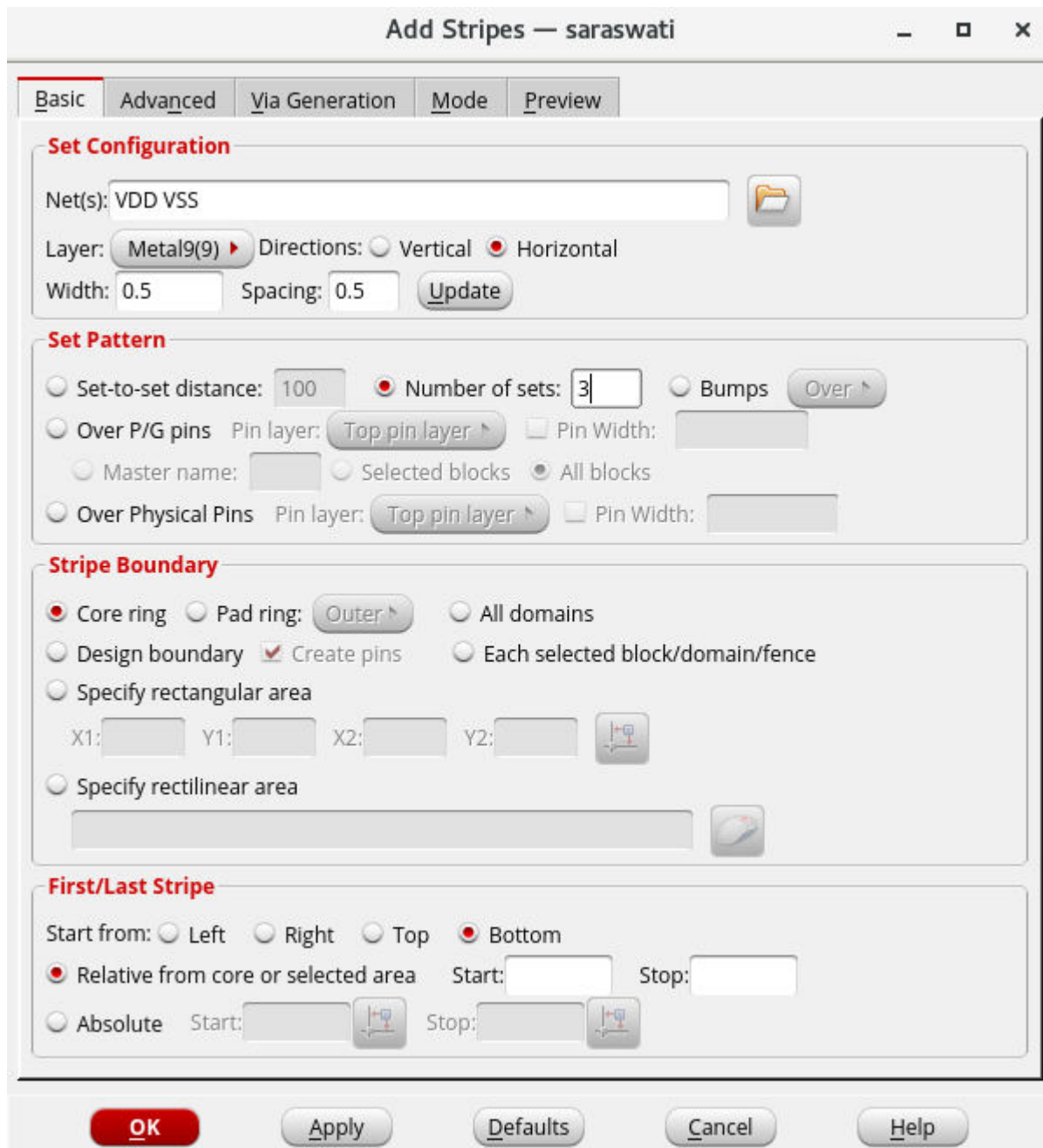


- e. Make sure that the Net(s) field contains **VDD** and **VSS**.
- f. Click Core ring(s) contouring.
- g. Select Around core boundary.
- h. In the Ring Configuration field, make sure that **Metal9 H layer** is selected for **Top** and **Bottom**.
- i. Use Metal8 V as the layer for Left and Right.
- j. For **Width**, change the default value to **0.5**
- k. For **Spacing**, change the default value to **0.5**
- l. For **Offset**, change the default value to **0.8** or Enable Offset Centre in channel
- m. To generate the power rings, click **Apply**

Notice that the power rings are created.



3. Choose **Power-Power Planning – Add Stripe**. The Add Stripes form appears.



Add Stripes — saraswati

Basic Advanced Via Generation Mode Preview

Set Configuration

Net(s): VDD VSS 

Layer: Metal9(9) ▾ Directions: ☐ Vertical ☒ Horizontal

Width: 0.5 Spacing: 0.5

Set Pattern

☐ Set-to-set distance: 100 ☒ Number of sets: 3 ☐ Bumps

☐ Over P/G pins Pin layer: Top pin layer ▾ ☐ Pin Width:

☐ Master name: ☐ Selected blocks ☒ All blocks

☐ Over Physical Pins Pin layer: Top pin layer ▾ ☐ Pin Width:

Stripe Boundary

☒ Core ring ☐ Pad ring: Outer ▾ ☐ All domains

☐ Design boundary ☒ Create pins ☐ Each selected block/domain/fence

☐ Specify rectangular area

X1: Y1: X2: Y2: 

☐ Specify rectilinear area 

First/Last Stripe

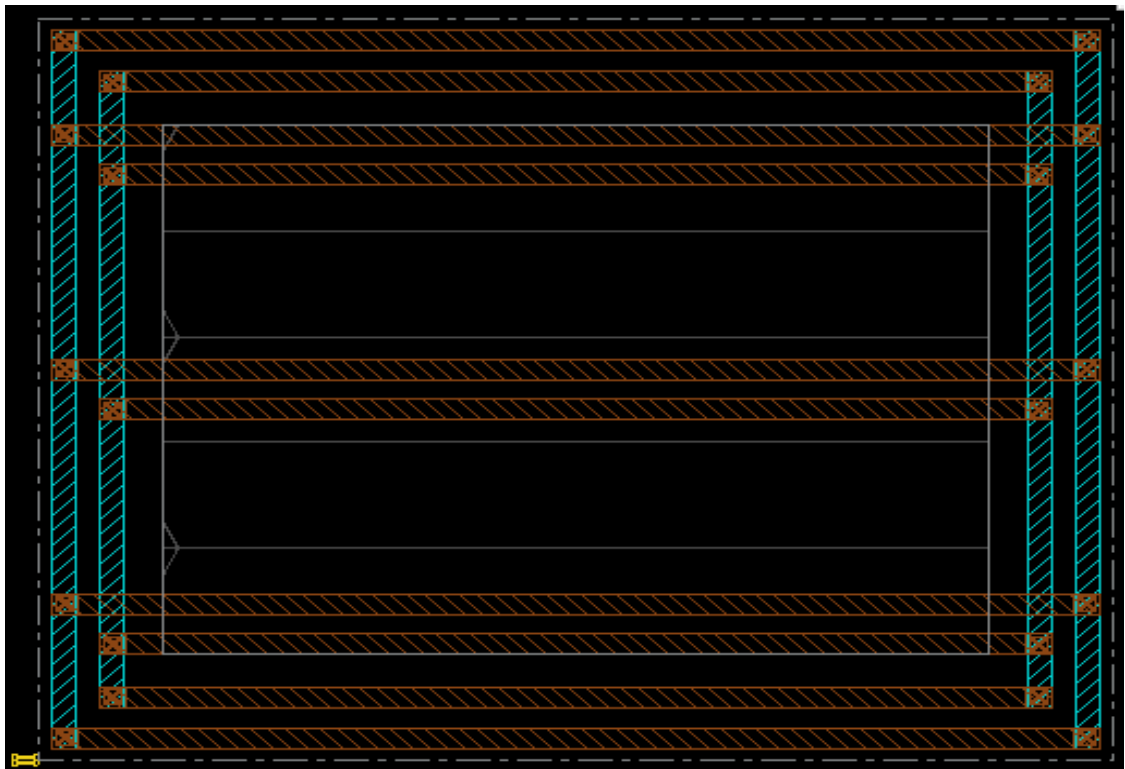
Start from: ☐ Left ☐ Right ☐ Top ☒ Bottom

☒ Relative from core or selected area Start: Stop:

☐ Absolute Start:  Stop: 

- n. Make sure that the Net(s) field contains **VDD** and **VSS**.
- o. Set **width** and **spacing** to **0.5** and **0.5** respectively
- p. Select **Metal9** and **Horizontal**, if it is not already selected
- q. Set Set-to-set distance or Number of set to 3
- r. Click on **Apply**
- s. Similarly you need to repeat the same and changes the **Metal9** to **Metal8** for Vertical and
- t. Click on **OK**

Notice the power stripes and the vias connecting the rings to the stripes are created.



Add Stripes — saraswati

Basic | Advanced | Via Generation | Mode | Preview

Set Configuration

Net(s): VDD VSS

Layer: Metal8(8) Directions: ☒ Vertical ☐ Horizontal

Width: 0.5 Spacing: 0.5

Set Pattern

☐ Set-to-set distance: 100 ☒ Number of sets: 3 ☐ Bumps

☐ Over P/G pins Pin layer: Top pin layer ▾ ☐ Pin Width:

☐ Master name: ☐ Selected blocks ☒ All blocks

☐ Over Physical Pins Pin layer: Top pin layer ▾ ☐ Pin Width:

Stripe Boundary

☒ Core ring ☐ Pad ring: Outer ▾ ☐ All domains

☐ Design boundary ☒ Create pins ☐ Each selected block/domain/fence

☐ Specify rectangular area

X1: Y1: X2: Y2:

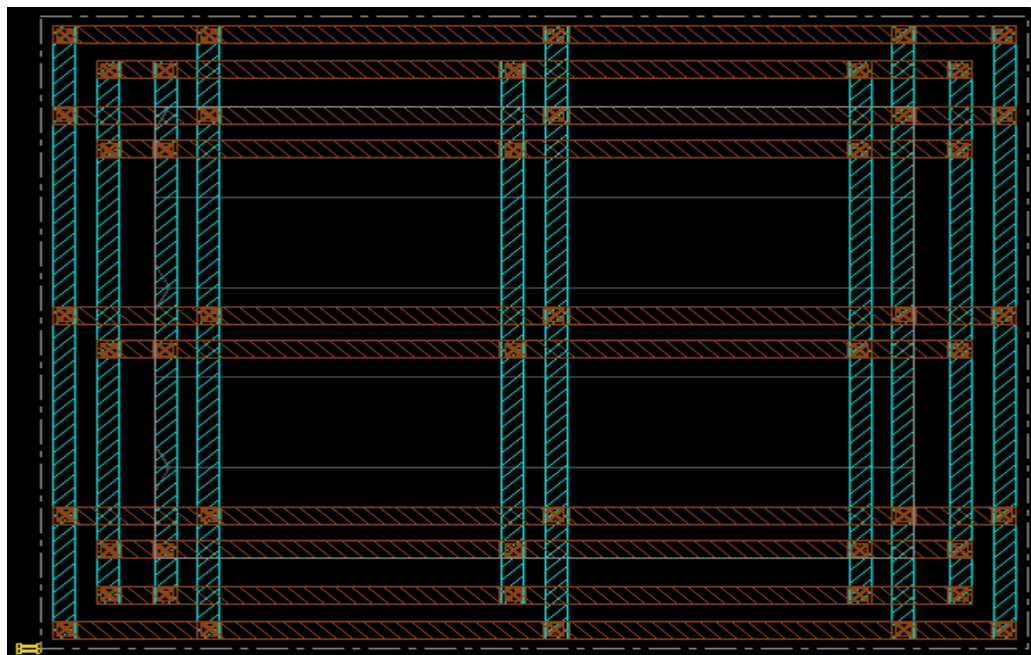
☐ Specify rectilinear area

First/Last Stripe

Start from: ☒ Left ☐ Right ☐ Top ☐ Bottom

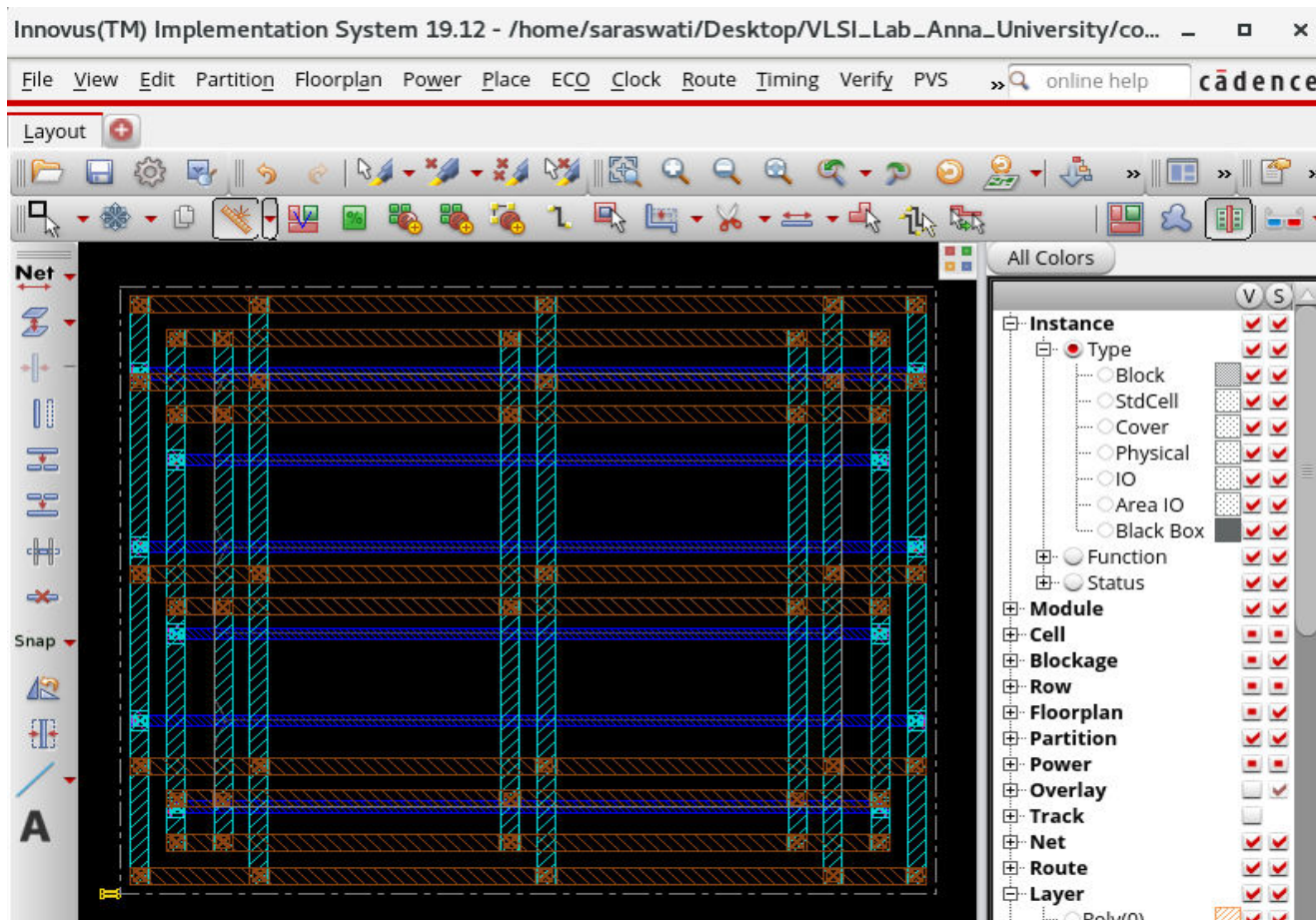
☒ Relative from core or selected area Start: Stop:

☐ Absolute Start: Stop:



Creating Power Rails with Special Route

4. Choose **Route – Special Route**. The SRoute form appears.



a. To populate the field with VDD and VSS, click the icon next to the Net(s) field.

b. Click Add to add the nets to the Chosen Nets field and click OK.

c. Deselect all options except Follow Pins.

d. Click OK.

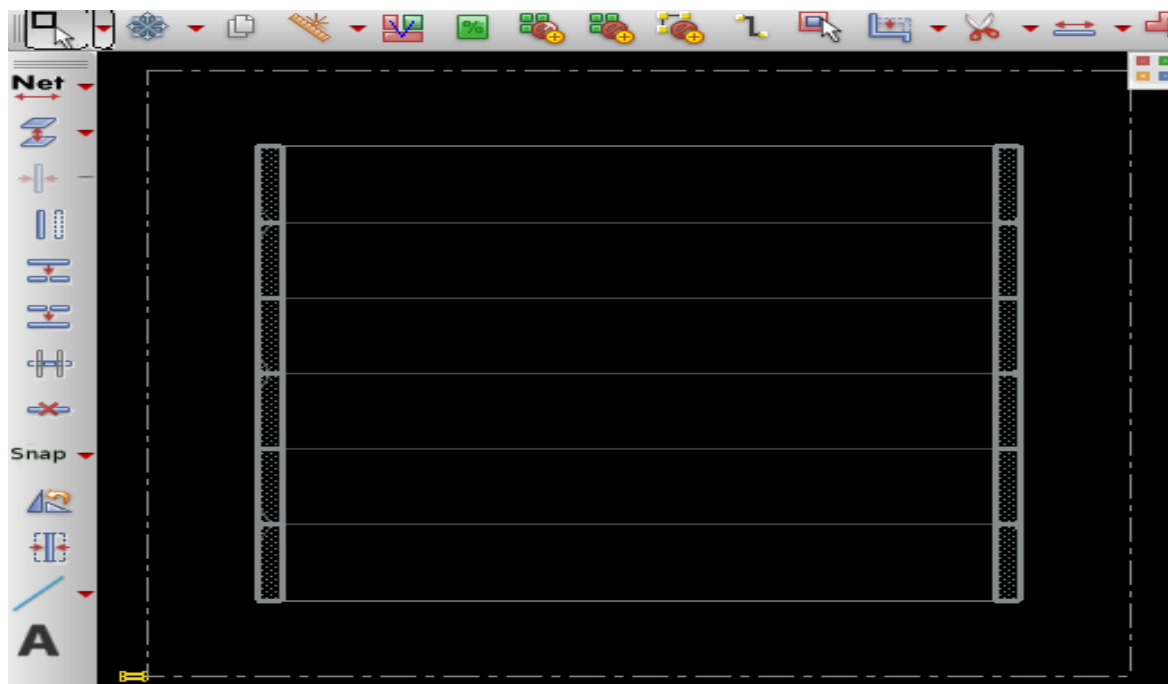
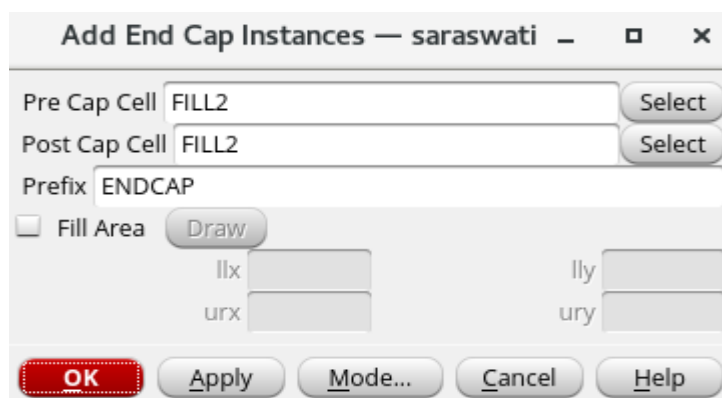
In the Physical view, zoom in to the follow pin routes. Notice that the power routes have been connected to the power planned targets with relevant Vias.

Pre – Placement:

After Power Planning, a few Physical Cells are added namely, **End Caps** and **Well Taps**.

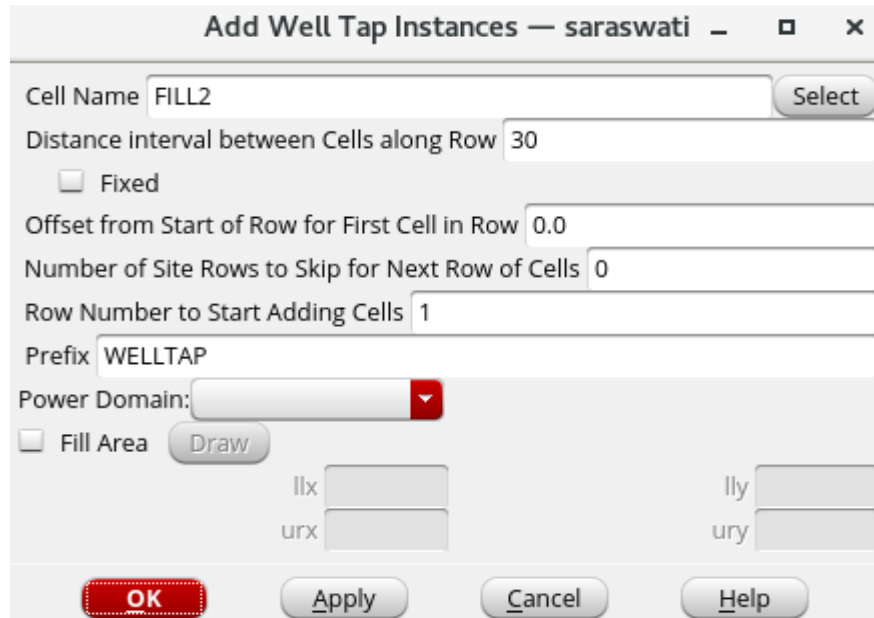
End Caps: They are Physical Cells which are added to the Left and Right Core Boundaries acting as blockages to avoid Standard Cells from moving out of boundary.

1. To add End Caps, Select **Place** → **Physical Cell** → **Add End Caps** and “Select” the FILL’s from the available list. Higher Fills have Higher Widths. As shown Below, The End Caps are added below your Power Mesh.



Well Taps: They act like Shunt Resistance to avoid Latch Up effects.

2. To add Well Taps, Select **Place** → **Physical Cell** → **Add Well Tap**, Select → FillX [X → Strength of Fill = 1,2,4 etc] → Distance Interval [Could be given in range of 30-45u] → **OK**



Add Well Tap Instances — saraswati

Cell Name: FILL2 Select

Distance interval between Cells along Row: 30

☐ Fixed

Offset from Start of Row for First Cell in Row: 0.0

Number of Site Rows to Skip for Next Row of Cells: 0

Row Number to Start Adding Cells: 1

Prefix: WELLTAP

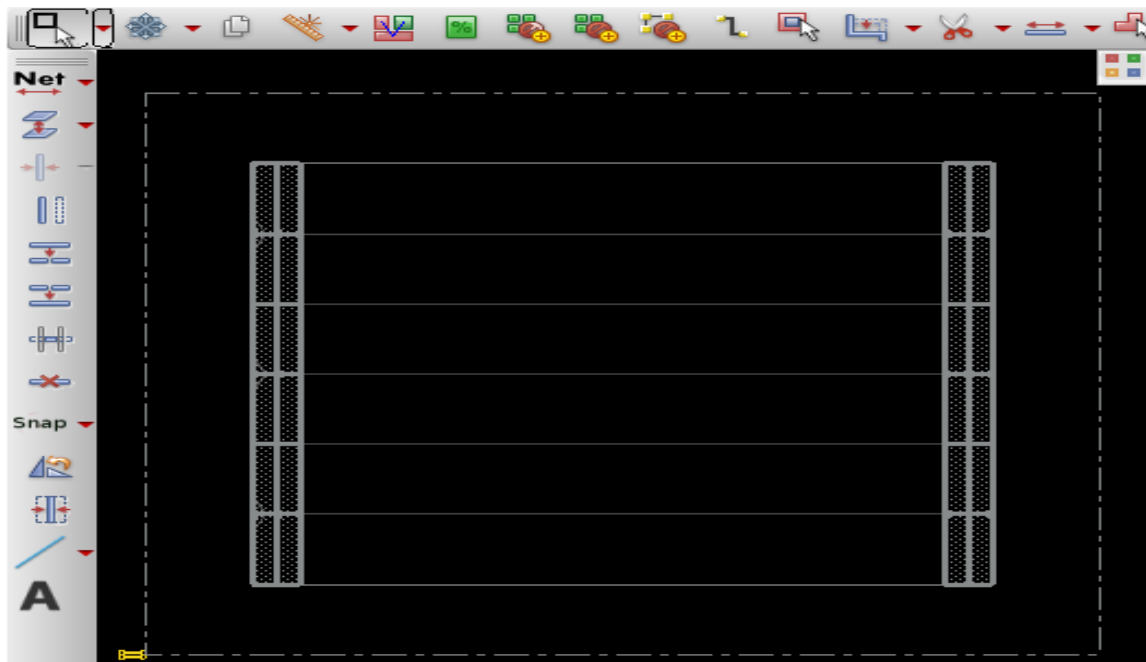
Power Domain: ▼

☐ Fill Area Draw

llx: lly:

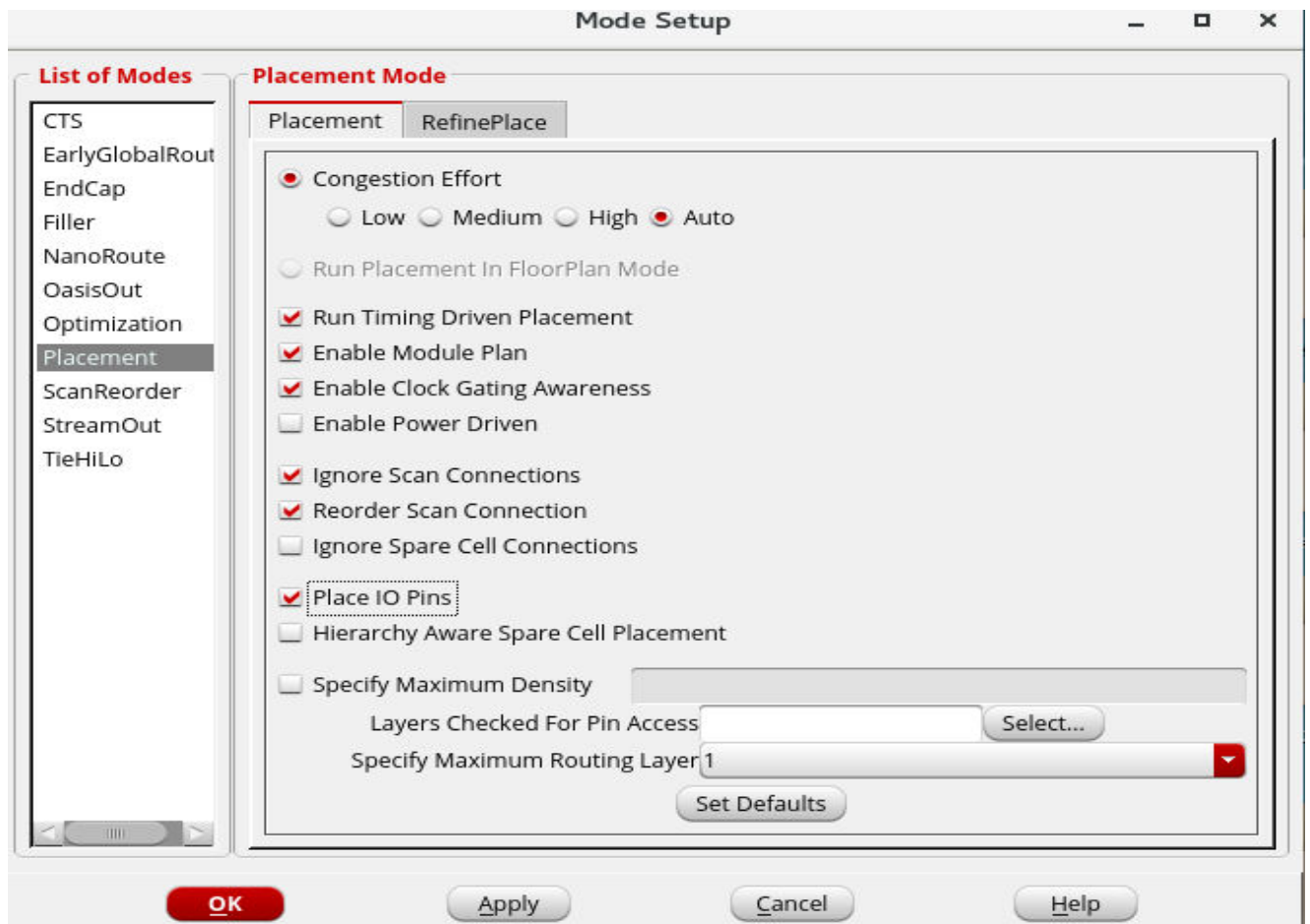
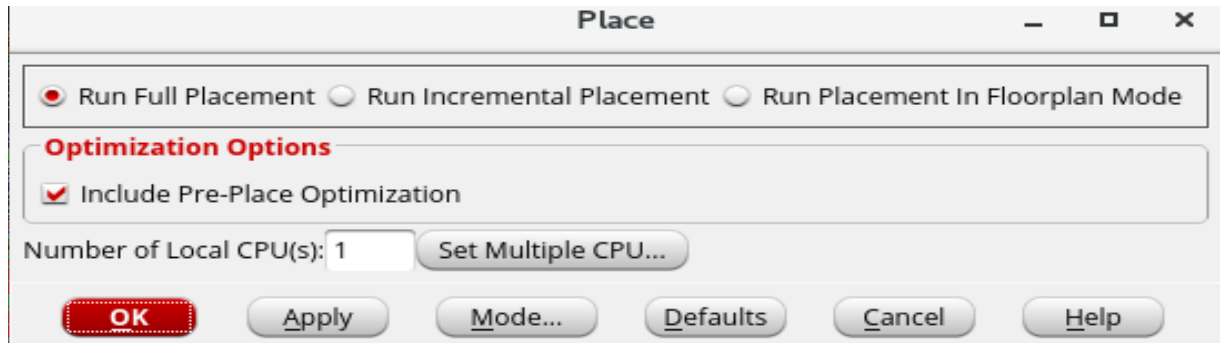
urx: ury:

OK Apply Cancel Help

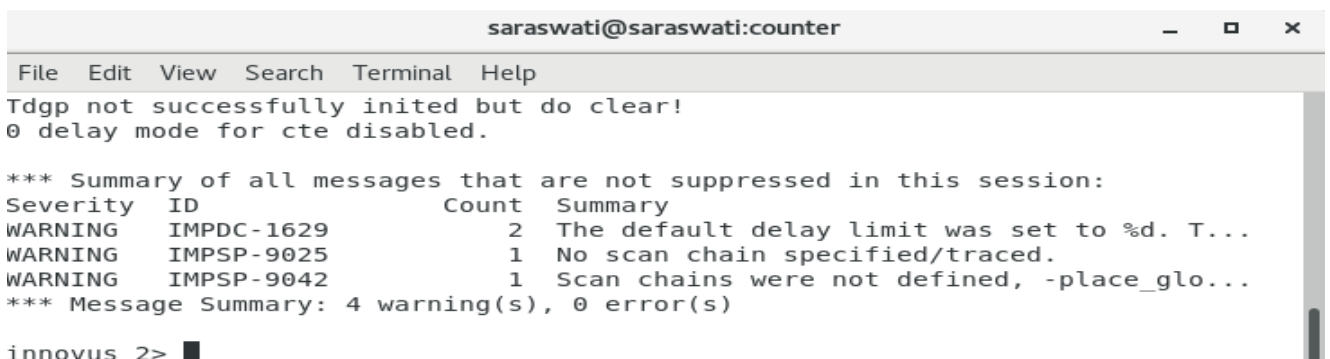
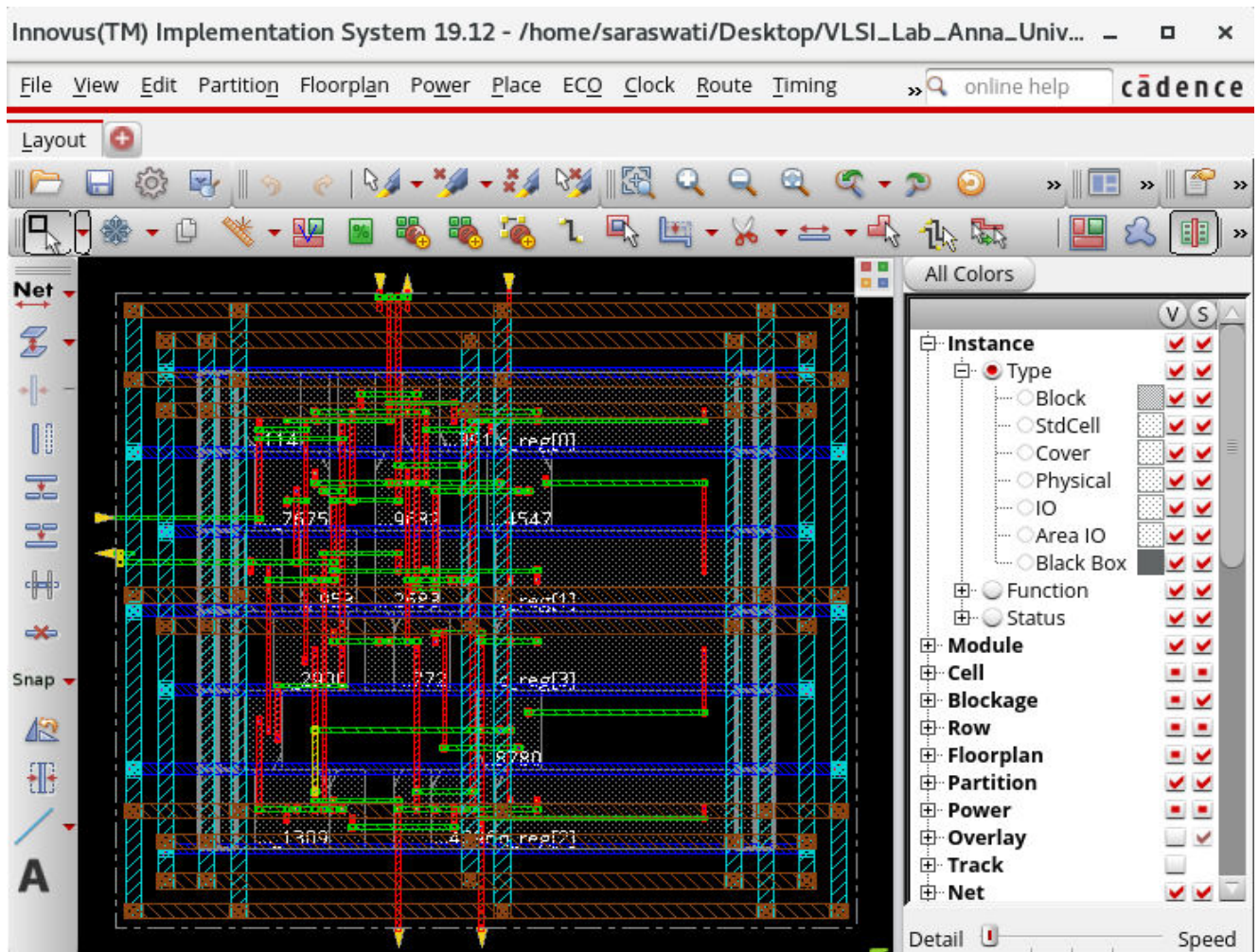


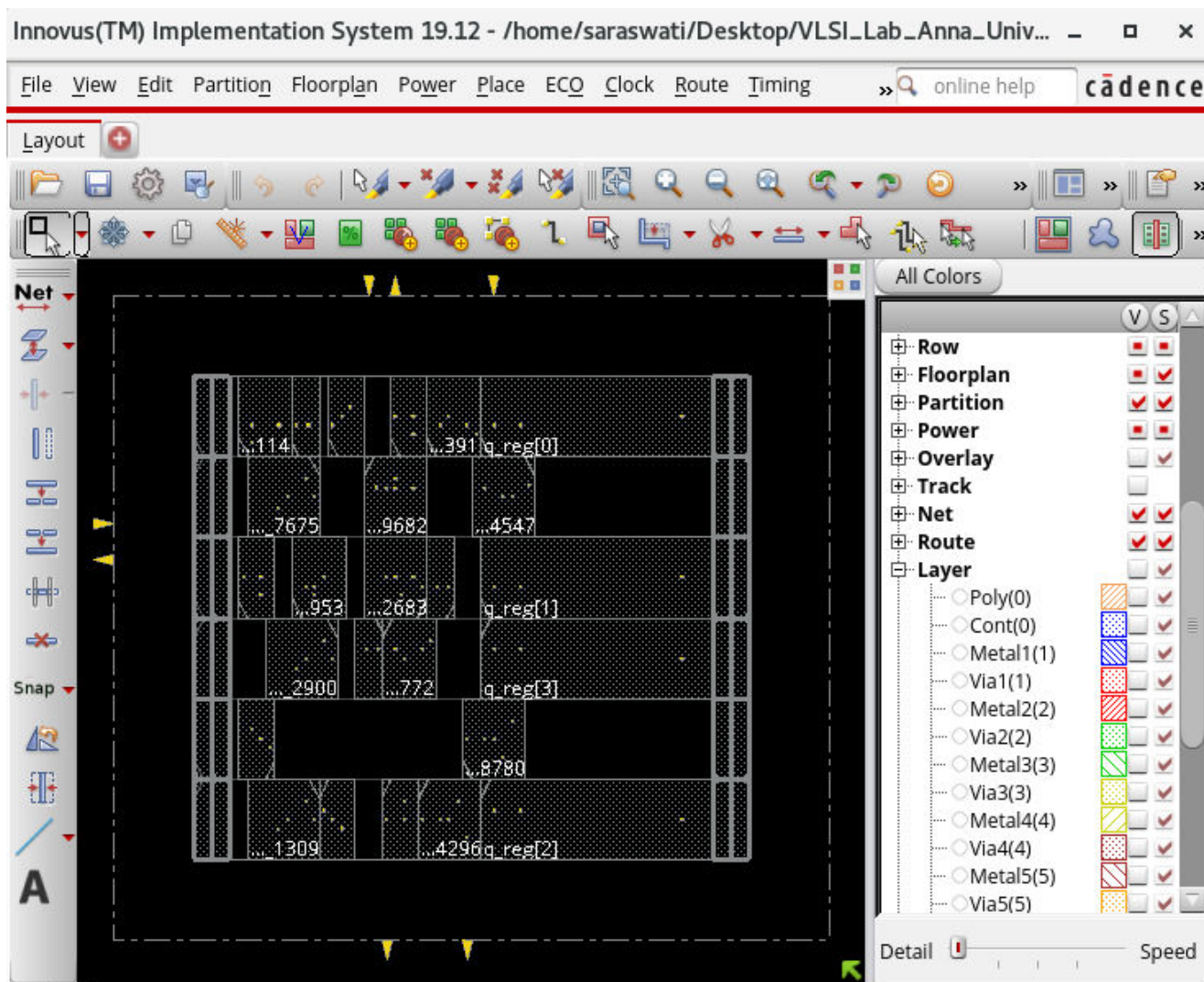
Module 5.3: Placement:

1. The Placement stage deals with Placing of Standard Cells as well as Pins.
2. Select Place → Place Standard Cell → Run Full Placement → Mode → Enable 'Place I/O Pins' → OK → OK.



All the Standard Cells and Pins are placed as per the communication between them, i.e., Two communicating Cells are placed as close as possible so that shorter Net lengths can be used for connections as Shorter Net Lengths enable Better Timing Results.



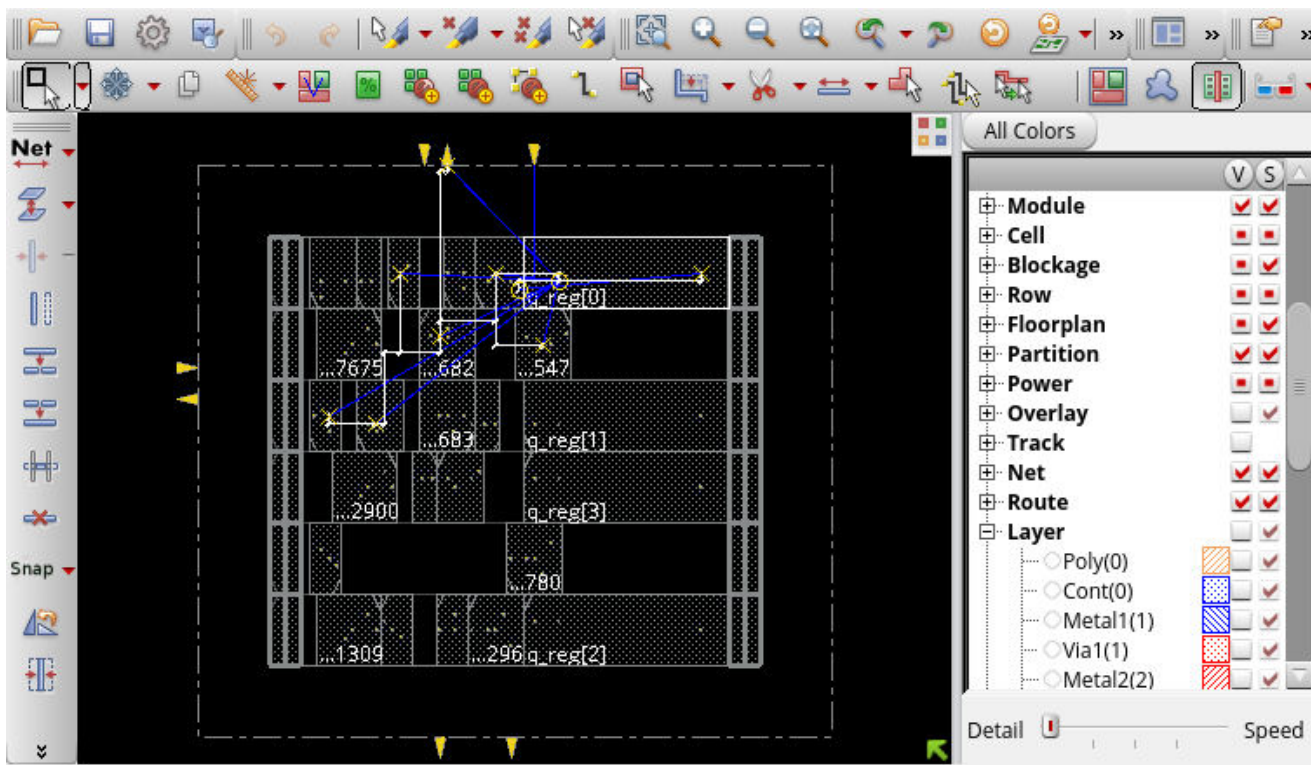


Standard Cells Placed

You can toggle the **Layer Visibility** from the **All Colors** window on the **Right** to see the standard cell placed in the core area.

You can be able to see the type of Standard cell or property of the cell by **selecting** the cell and **Right click** and choose **Attribute Editor** window as shown below.

The blue colour lines as shown are **Fly lines** which gives the **Virtual Connection** between the **Pin to Cell** or **Cell to Cell**.



Attribute Editor — saraswati

Object Type: Standard Cell

Name	Value	Type
Name	q_reg[0]	String
No. of Terminals	3	Integer
Cell Type	DFFQX4	String
Cell Width	7.54	Double
Cell Height	2.61	Double
Location X:	11.89	Location
Location Y:	15.66	Location
Box	{11.89 15.66} {19.43 18.27}	Box
Location Origin	Lower Left	Origin
Orientation	R0	Orientation
Status	PLACED	Enumerate
CTS Status	UNSET	String
Effective Status	PLACED	String
Routing Halo	None	String

OK Apply Add Prop Delete Prop Close Help

Report Generation and Optimization:

Timing Report:

1. Select **Timing** → **Report Timing**
 2. In the Design Stage section, Select → **Pre-CTS**
- In the Analysis Type section, select → **Setup** which is default and click **OK**.



When timing analysis is done, a timing summary is printed to the Innovus shell. This file is saved under directory timingReports. The **timingReports** directory contains several additional reports commonly used for debugging:

The file with suffix **.summary.gz**, is created after each phase once timing analysis is run. As shown below, file **counter_preCTS.summary.gz**, shows timing and **DRV** information at preCTS stage (In PreCTS stage **focus is on setup** whereas **hold is ignored** because clock tree synthesis is not run yet, an ideal clock is used).

Setup Timing Report:

timeDesign Summary

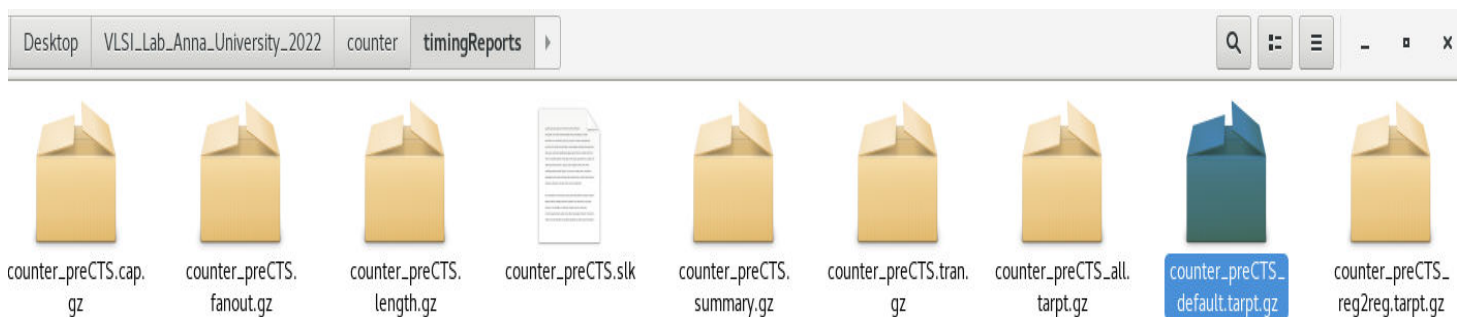
Setup mode	all	reg2reg	default
WNS (ns):	0.536	0.536	0.710
TNS (ns):	0.000	0.000	0.000
Violating Paths:	0	0	0
All Paths:	8	4	8

DRVs	Real		Total
	Nr nets(terms)	Worst Vio	Nr nets(terms)
max_cap	0 (0)	0.000	0 (0)
max_tran	0 (0)	0.000	0 (0)
max_fanout	0 (0)	0	0 (0)
max_length	0 (0)	0	0 (0)

Density: 67.901%

Routing Overflow: 0.00% H and 0.00% V

File → **counter_preCTS_default.tarpt.gz** shows details of the top critical setup paths in the design.



```
# Design: counter
# Command: timeDesign -preCTS -pathReports -drvReports -slackReports -numPaths 50 -prefix counter_preCTS -outDir timingReports
#####
Path 1: MET Setup Check with Pin q_reg[3]/CK
Endpoint: q_reg[3]/D (v) checked with leading edge of 'clk'
Beginpoint: q_reg[1]/Q (v) triggered by leading edge of 'clk'
Path Groups: {reg2reg}
Analysis View: wc
Other End Arrival Time 0.000
- Setup 0.123
+ Phase Shift 2.000
- Uncertainty 0.010
= Required Time 1.867
- Arrival Time 1.330
= Slack Time 0.536
Clock Rise Edge 0.000
= Beginpoint Arrival Time 0.000
Timing Path:
```

Pin	Edge	Net	Cell	Delay	Arrival Time	Required Time
clk	^	clk			0.000	0.536
q_reg[1]/CK	^	clk	DFFQX4	0.000	0.000	0.536
q_reg[1]/Q	v	q[1]	DFFQX4	0.608	0.608	1.145
g583_7675/S0	v	q[1]	MXI2XL	0.002	0.610	1.146
g583_7675/Y	v	n_12	MXI2XL	0.252	0.862	1.399
g578_9682/C0	v	n_12	A0I211X1	0.000	0.862	1.399
g578_9682/Y	^	n_13	A0I211X1	0.173	1.035	1.571
g577/A	^	n_13	CLKINVX1	0.000	1.035	1.571
g577/Y	v	n_15	CLKINVX1	0.072	1.107	1.643
g572_3772/A1	v	n_15	OAI21X1	0.000	1.107	1.643
g572_3772/Y	^	n_17	OAI21X1	0.101	1.208	1.744
g570_8780/C0	^	n_17	OAI211X1	0.000	1.208	1.744
g570_8780/Y	v	n_20	OAI211X1	0.123	1.330	1.867
q_reg[3]/D	v	n_20	DFFQX4	0.000	1.330	1.867

```

Clock Rise Edge 0.000
= Beginpoint Arrival Time 0.000
Other End Path:
```

Pin	Edge	Net	Cell	Delay	Arrival	Required
-----	------	-----	------	-------	---------	----------

Other files which are generated and saved under directory timingReports are:

counter_preCTS.cap → max capacitance violations
 counter_preCTS.fanout → max fanout violations
 counter_preCTS.slk → slack report for each endpoint
 counter_preCTS.summary → timing summary
 counter_preCTS.tran → max transition violations
 counter_preCTS_all.tarpt → timing report for all path groups
 counter_preCTS_in2out.tarpt → report for input to output paths
 counter_preCTS_clkgate.tarpt → report for clock gating paths
 counter_preCTS_in2reg.tarpt → report for input to register paths
 counter_preCTS_reg2out.tarpt → report for register to output paths
 counter_preCTS_reg2reg.tarpt → report for register to register paths

Area Report:

To generate Area Report, Switch to the Terminal and type the command , **report_area** to see the Cell Count and Area Occupied.

```

innovus 3> report_area
      Hinst Name      Module Name      Inst Count      Total Area
-----
counter              25              166.518
innovus 4> █

```

Power report:

To generate Power Report, In the Terminal type the command **report_power** to see the Power Consumption numbers.

```

Total Power
-----
Total Internal Power:      0.06590995      45.4954%
Total Switching Power:    0.07790113      53.7725%
Total Leakage Power:      0.00106062      0.7321%
Total Power:              0.14487169
-----

Group                      Internal Power      Switching Power      Leakage Power      Total Power      Percentage (%)
-----
Sequential                 0.05275           0.07257           0.0007135           0.126           87
Macro                      0                 0                 0                 0                 0
IO                          0                 0                 0                 0                 0
Combinational              0.01316           0.005328          0.0003471           0.01884          13
Clock (Combinational)      0                 0                 0                 0                 0
Clock (Sequential)         0                 0                 0                 0                 0
-----
Total                      0.06591           0.0779           0.001061           0.1449          100
-----

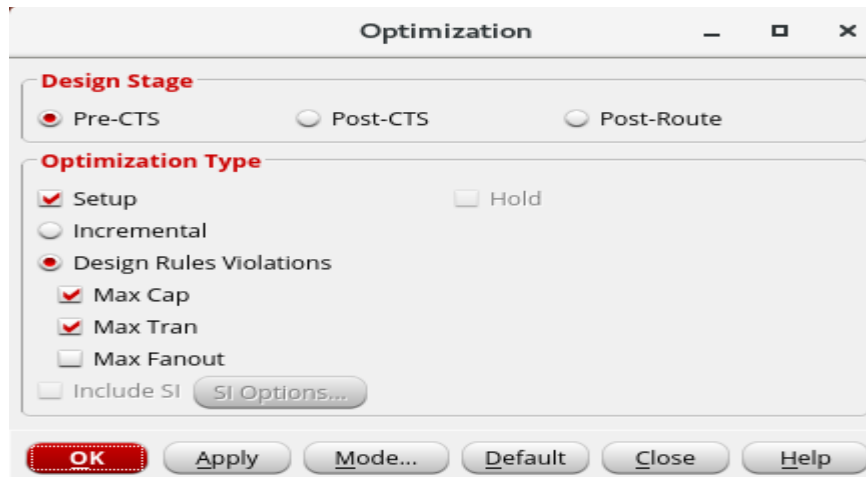
Rail                      Voltage      Internal Power      Switching Power      Leakage Power      Total Power      Percentage (%)
-----
VDD                        0.9         0.06591           0.0779           0.001061           0.1449          100
-----

*      Power Distribution Summary:
*      Highest Average Power:      q_reg[1] (DFFQX4):      0.03508
*      Highest Leakage Power:      q_reg[3] (DFFQX4):      0.0001784
*      Total Cap:      9.33099e-13 F
*      Total instances in design:      25
*      Total instances in design with no power:      0
*      Total instances in design with no activity:      0
*
*      Total Fillers and Decap:      0
-----

```

Design Optimization:

To optimize the Design, Select **ECO** → **Optimize Design** → Design Stage [**PreCTS**] → Optimization Type – **Setup** → **OK**



This step Optimizes your design in terms of Timing, Area and Power. You can Generate Timing, Area, Power in similar way as above report Post –Optimization to compare the Reports.

During the optimization stage of the command, the following operations may be performed to close timing:

- Adding buffers
- Resizing gates
- Reconstructing the circuit
- Remapping the logic
- Swapping the pins
- Deleting the buffers
- Moving the instances

In the placement stage **No Setup Violations** found in the timing report for the current design and we can **ignore** the optimization process. The optimization can be done only if any violations are there in the design because it will impact on area.

Module 5.4: Clock Tree Synthesis

The CTS Stage is meant to build a Clock Distribution Network such that every Register (Flip Flop) acquires Clock at the same time (At least Approximately) to keep them in proper communication.

Clock Tree Synthesis using CCOpt-CTS is run to build a clock tree which delivers the clock signal to the sequential elements within the specified constraints. To synthesize the clock tree you will:

- Define a non-default rule (**NDR**) that has double the width and spacing of the default rule. This will be used to route the clocks.
- Configure CCOpt and generate a **CCOpt** specification file.
- Run **CCOpt-CTS** to build the clock tree

Clock Tree constraints used for Building and Balancing the Clock Tree in CTS Stage

Creating Non Default Rule:

To Create Non Default Rules you need to set the width and spacing value of all metal layers from the lef file. Specify the rule name 2w2s and specify double the width and spacing of the default rule as shown below:

```
add_ndr -width {Metal1 0.24 Metal2 0.28 Metal3 0.28 Metal4 0.28 Metal5 0.28 Metal6 0.28 Metal7 0.28 Metal8 0.88 Metal9 0.88 } -spacing {Metal1 0.24 Metal2 0.28 Metal3 0.28 Metal4 0.28 Metal5 0.28 Metal6 0.28 Metal7 0.28 Metal8 0.8 Metal9 0.8 } -name 2w2s
```

Create a route type to define the NDR & layers to use for routing the clock tree:

```
create_route_type -name clkroute -non_default_rule 2w2s -bottom_preferred_layer Metal5 -top_preferred_layer Metal6
```

specify this route type should be used for trunk and leaf nets:

```
set_ccopt_property route_type clkroute -net_type trunk
set_ccopt_property route_type clkroute -net_type leaf
```

specify the Clock Buffer, Clock Inverter & Clock Gating Cells to use:

```
set_ccopt_property buffer_cells {CLKBUFX6 CLKBUFX8 CLKBUFX12}
set_ccopt_property inverter_cells {CLKINVX6 CLKINVX8 CLKINVX12}
set_ccopt_property clock_gating_cells TLATNTSCA*
```


Generate the CCOpt spec file & source it:

```
create_ccopt_clock_tree_spec -file ccopt.spec
source ccopt.spec
```

Run CCOpt-CTS:

```
ccopt_design -cts
```

Once after running **ccopt_design -cts** in the **innovus** terminal you should need to check for the **error**. If any errors then your cts stage so, you should need to fix the errors. Until unless fixing the error you shouldn't need to proceed further.

```
CCOpt::Phase::PreparingToBalance done. (took cpu=0:00:01.3 real=0:00:20.4)
Runtime done. (took cpu=0:00:01.4 real=0:00:24.5)
Runtime Summary
=====
Clock Runtime: (66%) Core CTS          15.32 (Init 15.32, Construction 0.00, Implementation 0.00, eGRPC 0.00, Post
Conditioning 0.00, Other 0.00)
Clock Runtime: (0%) CTS services       0.00 (RefinePlace 0.00, EarlyGlobalClock 0.00, NanoRoute 0.00, ExtractRC 0
.00, TimingAnalysis 0.00)
Clock Runtime: (33%) Other CTS         7.71 (Init 7.71, CongRepair/EGR-DP 0.00, TimingUpdate 0.00, Other 0.00)
Clock Runtime: (100%) Total            23.03

**ERROR: (IMPCCOPT-2196):      Cannot run ccopt_design because the command prerequisites were not met. Review the
previous error messages for more details about the failure.
3

*** Summary of all messages that are not suppressed in this session:
Severity ID          Count Summary
WARNING  IMPCCOPT-1361    3 Routing configuration for %s nets in clo...
ERROR    IMPCCOPT-2196    1 Cannot run ccopt_design because the comm...
ERROR    IMPCCOPT-1013    1 The target_max_trans is too low for at l...
*** Message Summary: 3 warning(s), 2 error(s)

#% End ccopt_design (date=03/22 14:00:44, total cpu=0:00:01.6, real=0:00:25.0, peak res=939.9M, current mem=939.9M)
innovus 16> █
```

In CTS stage getting an error due to target_max_trans is tool low.

To fix this error you need to set the target_max_trans value in innovus tool and try to run the ccopt_design -cts once again.

Set the target max tarns using below command

```
set_ccopt_property target_max_trans 0.200
```

```

Runtime Summary
=====
Clock Runtime: (31%) Core CTS          4.59 (Init 1.75, Construction 1.62, Implementation 0.46, eRPC 0.34, PostC
onditioning 0.17, Other 0.24)
Clock Runtime: (54%) CTS services      7.87 (RefinePlace 1.57, EarlyGlobalClock 0.77, NanoRoute 5.37, ExtractRC 0
.16, TimingAnalysis 0.00)
Clock Runtime: (14%) Other CTS         2.04 (Init 0.13, CongRepair/EGR-DP 0.67, TimingUpdate 1.24, Other 0.00)
Clock Runtime: (100%) Total            14.50

Synthesizing clock trees with CCOpt done.
**WARN: (IMPSP-9025): No scan chain specified/traced.
Type 'man IMPSP-9025' for more detail.

*** Summary of all messages that are not suppressed in this session:
Severity ID Count Summary
WARNING IMPEXT-3530 3 The process node is not set. Use the com...
WARNING IMPSP-9025 1 No scan chain specified/traced.
WARNING IMPCCOPT-1041 1 The source_output_max_trans is set for %...
WARNING IMPCCOPT-1361 3 Routing configuration for %s nets in clo...
WARNING IMPTCM-77 1 Option "%s" for command %s is obsolete a...
*** Message Summary: 9 warning(s), 0 error(s)

#% End ccopt design (date=03/22 14:15:35, total cpu=0:00:06.4, real=0:00:15.0, peak res=956.0M, current mem=944.7M)
innovus 32> █

```

Generate Reports for Clock Tree & Skew Groups

```

report_ccopt_clock_trees -file clk_trees.rpt
report_ccopt_skew_groups -file skew_groups.rpt

```

Save the Design

```

saveDesign DBS/cts.enc

```

A Script running CTS:

```

cts.tcl
~/Desktop/VLSI_Lab_Anna_University_2022/counter

Open Save

#90nm technology
## Adding Non Default Rule:
add_ndr -width {Metal1 0.24 Metal2 0.28 Metal3 0.28 Metal4 0.28 Metal5 0.28 Metal6
0.28 Metal7 0.28 Metal8 0.88 Metal9 0.88 } -spacing {Metal1 0.24 Metal2 0.28 Metal3
0.28 Metal4 0.28 Metal5 0.28 Metal6 0.28 Metal7 0.28 Metal8 0.8 Metal9 0.8 } -name 2w2s

## Create a route type to define the NDR & layers to use for routing the clock tree:
create_route_type -name clkroute -non_default_rule 2w2s -bottom_preferred_layer Metal5
-top_preferred_layer Metal6

## specify this route type should be used for trunk and leaf nets:
set_ccopt_property route_type clkroute -net_type trunk
set_ccopt_property route_type clkroute -net_type leaf

## specify the Clock Buffer, Clock Inverter & Clock Gating Cells to use:
set_ccopt_property buffer_cells {CLKBUF6 CLKBUF8 CLKBUF12}
set_ccopt_property inverter_cells {CLKINV6 CLKINV8 CLKINV12}
set_ccopt_property clock_gating_cells TLATNTSCA*

## Set the target max tarns value to fix the error
set_ccopt_property target_max_trans 0.200

## Generate the CCOpt spec file & source it:
create_ccopt_clock_tree_spec -file ccopt.spec
source ccopt.spec

## Run CCOpt-CTS:
ccopt_design -cts

## Generate Reports for Clock Tree & Skew Groups
report_ccopt_clock_trees -file clk_trees.rpt
report_ccopt_skew_groups -file skew_groups.rpt

## Save the Design
saveDesign db/cts.enc

Tcl Tab Width: 8 Ln 85, Col 18 INS

```

Report Generation and Design Optimization:

CTS Stage adds real clock into the Design and hence “**Hold**” Analysis also becomes prominent. Hence, Optimizations can be done for both Setup & Hold, Timing Reports are to be Generated for Setup and Hold Individually.

Timing Report:

1. Select **Timing** → **Report Timing**

2. In the Design Stage section, Select → **Post-CTS**

In the Analysis Type section, Select → **Hold** which is default and click **OK**.



At this point the clock tree has been synthesized and we can analyze timing based on the actual clock tree delay and skew.

When timing analysis is done, a timing summary is printed to the Innovus shell. This file is saved under directory timingReports. The **timingReports** directory contains several additional reports commonly used for debugging :

The file with suffix .summary.gz, is created after each phase once timing analysis is run. As shown below, file **counter_postCTS.summary.gz**, shows timing and **DRV** information at postCTS stage (In Post-CTS stage focus is on Hold whereas Setup is ignored)

```
#####
# Generated by:      Cadence Innovus 19.12-s087_1
# OS:               Linux x86_64(Host ID saraswati)
# Generated on:      Tue Mar 22 14:52:20 2022
# Design:           counter
# Command:          timeDesign -postCTS -hold -pathReports -slackReports -numPaths
                    50 -prefix counter_postCTS -outDir timingReports
#####

-----
timeDesign Summary
-----

+-----+-----+-----+
| Hold mode | all | reg2reg | default |
+-----+-----+-----+
| WNS (ns): | 0.140 | 0.140 | 0.000 |
| TNS (ns): | 0.000 | 0.000 | 0.000 |
| Violating Paths: | 0 | 0 | 0 |
| All Paths: | 4 | 4 | 0 |
+-----+-----+-----+

Density: 67.901%
Routing Overflow: 0.00% H and 0.00% V
-----
```

File → **counter_postCTS_default_hold.tarpt.gz** shows details of the top critical setup paths in the design.

```
# Design: counter
# Command: timeDesign -postCTS -hold -pathReports -slackReports -numPaths 50 -prefix counter_postCTS -outDir timingReports
#####
Path 1: MET Hold Check with Pin q_reg[2]/CK
Endpoint: q_reg[2]/D (v) checked with leading edge of 'clk'
Beginpoint: q_reg[2]/Q (^) triggered by leading edge of 'clk'
Path Groups: {reg2reg}
Analysis View: bc
Other End Arrival Time 0.000
+ Hold 0.070
+ Phase Shift 0.000
+ Uncertainty 0.010
= Required Time 0.080
Arrival Time 0.220
Slack Time 0.140
Clock Rise Edge 0.000
= Beginpoint Arrival Time 0.000
Timing Path:
```

Pin	Edge	Net	Cell	Delay	Arrival Time	Required Time
clk	^	clk			0.000	-0.140
q_reg[2]/CK	^	clk	DFFQX4	0.000	0.000	-0.140
q_reg[2]/Q	^	q[2]	DFFQX4	0.195	0.195	0.055
g574_4296/A0	^	q[2]	0AI211X1	0.001	0.196	0.055
g574_4296/Y	v	n_19	0AI211X1	0.024	0.220	0.080
q_reg[2]/D	v	n_19	DFFQX4	0.000	0.220	0.080

```

Clock Rise Edge 0.000
= Beginpoint Arrival Time 0.000
Other End Path:
```

Pin	Edge	Net	Cell	Delay	Arrival Time	Required Time
clk	^	clk			0.000	0.140
q_reg[2]/CK	^	clk	DFFQX4	0.000	0.000	0.140

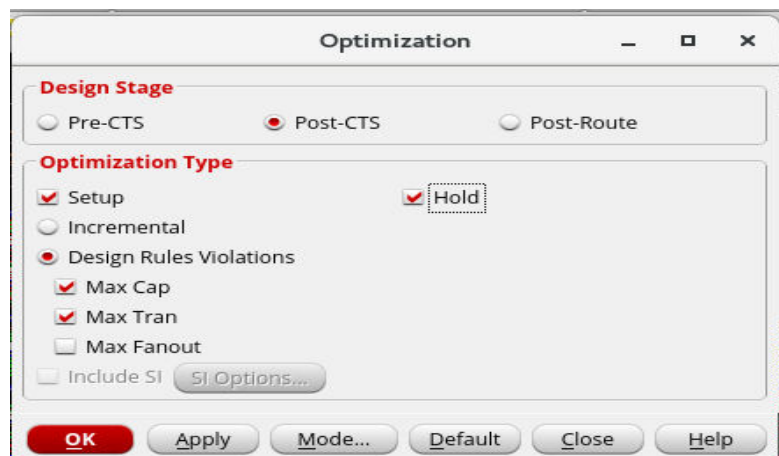
Area and Power Report Generation, use the below commands:

report_area

report_power

Here we are not getting any Hold violation so, we can ignore the optimization with hold analysis at clock synthesis stage.

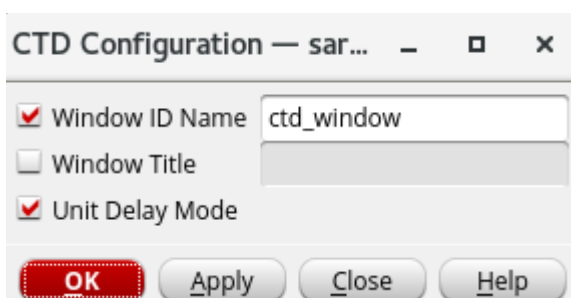
Design Optimizations:



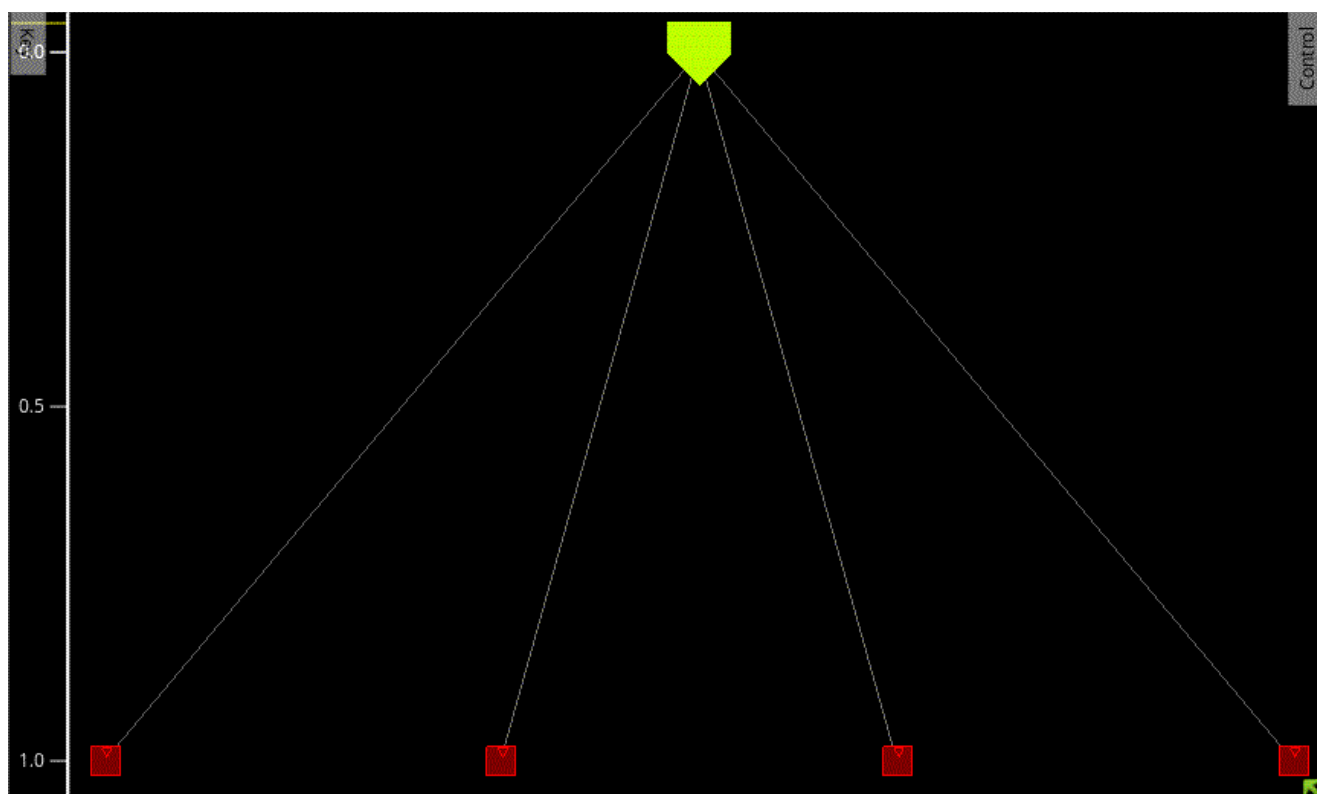
Running CCOpt Clock Tree Debugger (CTD):

The CCOpt Clock Tree Debugger is a graphical tool for analyzing and debugging the clock tree results.

1. Select **Clock - CCOpt Clock Tree Debugger**. The CCOpt Clock Tree Debugger appears.
2. The dialog box **CTD Configuration** is shown. Click **OK** without changing any settings.



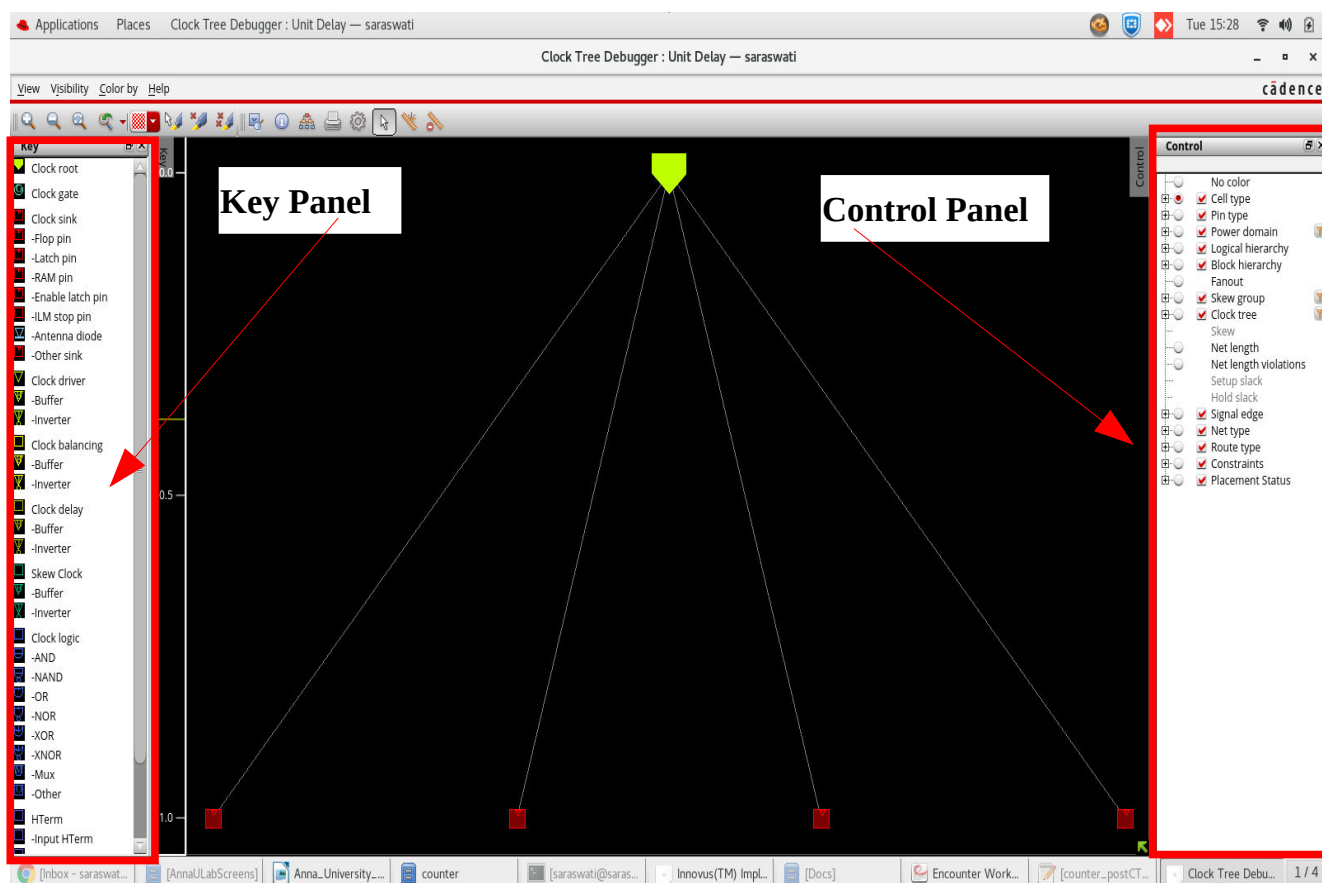
3. In view - enable browser



The debugger shows the logic in the clock tree with the clock roots at the top and the leaf cells at the bottom. Analysis is performed by colouring the clock tree based on selected criteria such as objects, paths, groups, and so on.

4. Display the **Key Panel** by selecting **View** → **Key Panel** or by selecting the Key tab in the upper-left corner.

5. Select **View** → **Control Panel** or click the **Control tab** in the upper-right to open the Control Panel. Initially, the clock tree is coloured based on the cell type. Use the Key Panel to determine the symbols that represent each cell type.



The Visibility menu filters which coloured objects are to be made visible.

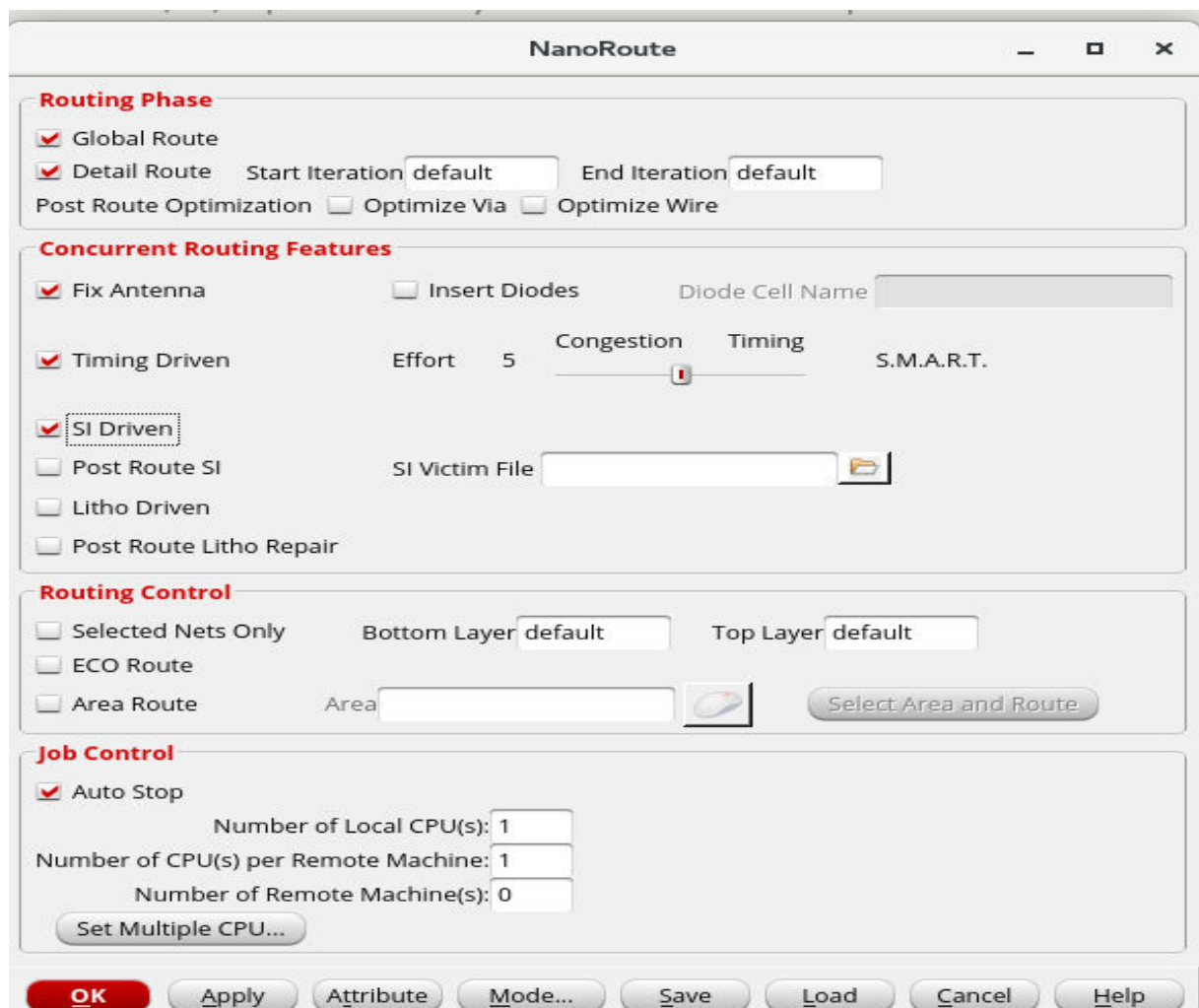
6. Enable the check box next to **Visibility** → **Cell type** → **Clock sink** to disable the colouring of all the clock sinks.

Module 5.5: Routing

Timing is not closed on a design unless crosstalk is prevented, analyzed, and then fixed. To really achieve the above, the design is run with global and detail routing using NanoRoute. When running NanoRoute, we enable both the Timing Driven and SI Driven (Signal Integrity) options in the NanoRoute form. These options are important in helping to close timing and preventing crosstalk.

All the net connections shown in the GUI till CTS are only based on the Logical connectivity. These connections are to be replaced with real Metals avoiding Opens, Shorts, Signal Integrity [Cross Talks], Antenna Violations etc.

To run Routing, Select **Route** → **Nano Route** → **Route** and enable **Timing Driven** and **SI Driven** for Design Physical Efficiency and Reliability. Click **OK**.



The screenshot shows the NanoRoute dialog box with the following settings:

- Routing Phase:**
 - ☒ Global Route
 - ☒ Detail Route (Start Iteration: default, End Iteration: default)
 - Post Route Optimization: ☐ Optimize Via, ☐ Optimize Wire
- Concurrent Routing Features:**
 - ☒ Fix Antenna, ☐ Insert Diodes (Diode Cell Name:)
 - ☒ Timing Driven (Effort: 5, Congestion: S.M.A.R.T., Timing: S.M.A.R.T.)
 - ☒ SI Driven (SI Victim File:)
 - ☐ Post Route SI
 - ☐ Litho Driven
 - ☐ Post Route Litho Repair
- Routing Control:**
 - ☐ Selected Nets Only (Bottom Layer: default, Top Layer: default)
 - ☐ ECO Route
 - ☐ Area Route (Area: , Select Area and Route button)
- Job Control:**
 - ☒ Auto Stop
 - Number of Local CPU(s): 1
 - Number of CPU(s) per Remote Machine: 1
 - Number of Remote Machine(s): 0
 - Set Multiple CPU... button

Buttons at the bottom: OK, Apply, Attribute, Mode..., Save, Load, Cancel, Help.

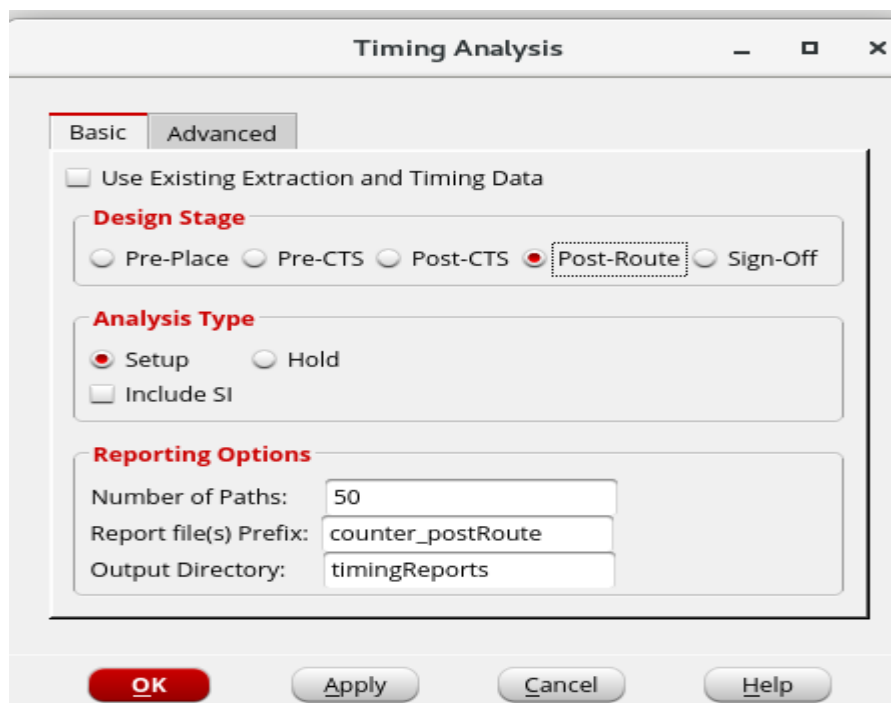
Running NanoRoute with the SI Driven option requires the capacitance table file (or QRC technology file), and this file was read in during design import. Choosing SI Driven option in NanoRoute is our first line of defense against noise.

Save the design: **saveDesign db/route.enc -verilog**

Report Generation and Design Optimization:

Setup Timing Analysis:

1. Select **Timing** → **Report Timing**
 2. In the Design Stage section, Select → **Post-Route**
- In the Analysis Type section, Select → **Setup** which is default and click **OK**.



```
#Complete globalDetailRoute on Tue Mar 22 15:57:37 2022
#
#Default setup view is reset to wc.
#routeDesign: cpu time = 00:00:32, elapsed time = 00:01:29, memory = 928.56 (MB), peak = 1089.95 (MB)

*** Summary of all messages that are not suppressed in this session:
Severity ID Count Summary
WARNING IMPEXT-3530 1 The process node is not set. Use the com...
WARNING IMPCK-8086 1 The command %s is obsolete and will be r...
WARNING IMPESI-2017 2 There is no coupling capacitance found i...
WARNING TCLCMD-1403 2 '%s'
*** Message Summary: 6 warning(s), 0 error(s)

innovus 33> Switching SI Aware to true by default in postroute mode

**ERROR: (IMPOPT-7027): The analysis mode needs to be set to 'OCV' in post route stage for post route timing & optimization. To avoid this message & allow post route steps to proceed set 'setAnalysisMode -analysisType onChipVariation'. It is also recommended to set '-cpr both' alongside this to remove clock re-convergence pessimism for both setup and hold modes.

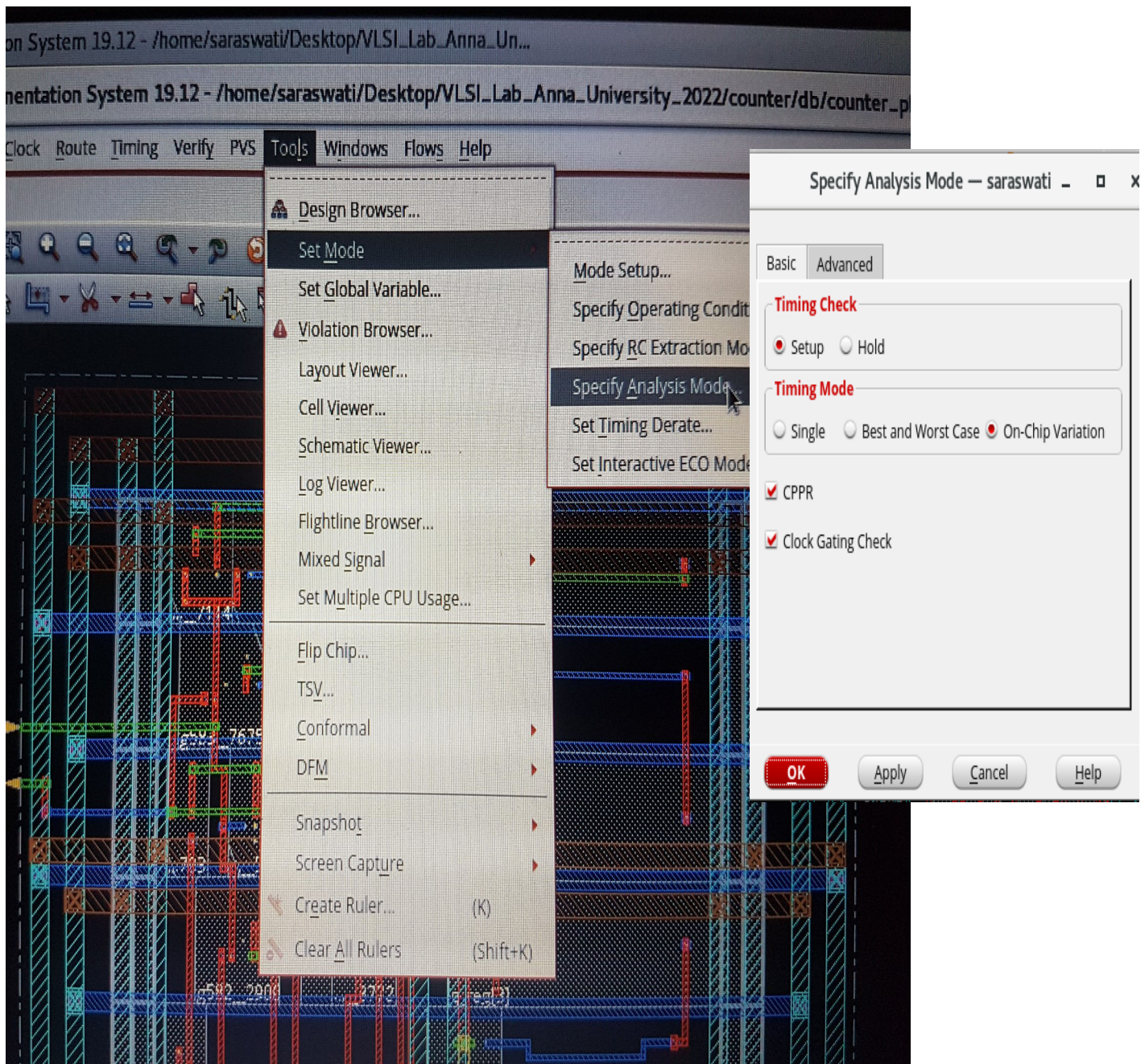
innovus 33> █
```

In the Routing while checking the timing analysis, The analysis mode need to be set to '**OCV**' in post route stage for post route timing and optimization. To avoid this message & allow post route steps to proceed set '**setAnalysisMode -analysisType onChipVariation**'. It is also recommended to set '**-cpr both**' alongside this to remove clock re-convergence pessimism for both setup and hold modes.

Use the below command to fix this error or set it from Innovus GUI

```
innovus 5>
innovus 5> setAnalysisMode -analysisType onChipVariation -cpr both
innovus 6> █
```

or



Rerun the Setup timing Analysis once again and check with report.

When timing analysis is done, a timing summary is printed to the Innovus shell. This file is saved under directory timingReports. The **timingReports** directory contains several additional reports commonly used for debugging :

The file with suffix .summary.gz, is created after each phase once timing analysis is run. As shown below, file **counter_postRoute.summary.gz**, shows timing and **DRV** information at postRoute stage (In Post-Route stage focus is on both Setup & Hold mode)

timeDesign Summary			
Setup mode	all	reg2reg	default
WNS (ns):	0.516	0.516	0.687
TNS (ns):	0.000	0.000	0.000
Violating Paths:	0	0	0
All Paths:	8	4	8

DRVs	Real		Total
	Nr nets(terms)	Worst Vio	Nr nets(terms)
max_cap	0 (0)	0.000	0 (0)
max_tran	0 (0)	0.000	0 (0)
max_fanout	0 (0)	0	0 (0)
max_length	0 (0)	0	0 (0)

Density: 67.901%

Total number of glitch violations: 0

File → **counter_postRoute_default.tarpt.gz** shows details of the top critical setup paths in the design.

Applications Places Text Editor

counter_postRoute_default.tarpt
~/cache/fr-liczst

Open Save

#####

Path 1: MET Setup Check with Pin q_reg[3]/CK
 Endpoint: q_reg[3]/D (v) checked with leading edge of 'clk'
 Beginpoint: rst (*) triggered by leading edge of 'clk'
 Path Groups: {default}
 Analysis View: wc
 Other End Arrival Time 0.000
 - Setup 0.123
 + Phase Shift 2.000
 + CPPR Adjustment 0.000
 - Uncertainty 0.010
 = Required Time 1.867
 - Arrival Time 1.180
 = Slack Time 0.687

Clock Rise Edge 0.000
 + Input Delay 0.400
 = Beginpoint Arrival Time 0.400

Timing Path:

Pin	Edge	Net	Cell	Delay	Arrival Time	Required Time
rst	^	rst			0.400	1.087
g591_7114/AN	^	rst	NAND2BX1	0.000	0.400	1.087
g591_7114/Y	^	n_3	NAND2BX1	0.146	0.546	1.232
g583_7675/A	^	n_3	MXI2XL	0.000	0.546	1.232
g583_7675/Y	v	n_12	MXI2XL	0.149	0.694	1.381
g578_9682/C0	v	n_12	AOI211X1	0.000	0.694	1.381
g578_9682/Y	^	n_13	AOI211X1	0.181	0.876	1.562
g577/A	^	n_13	CLKINX1	0.000	0.876	1.562
g577/Y	v	n_15	CLKINX1	0.075	0.950	1.637
g572_3772/A1	v	n_15	OAI21X1	0.000	0.950	1.637
g572_3772/Y	^	n_17	OAI21X1	0.104	1.055	1.741
g570_8780/C0	^	n_17	OAI211X1	0.000	1.055	1.741
g570_8780/Y	v	n_20	OAI211X1	0.125	1.180	1.867
q_reg[3]/D	v	n_20	DFFQX4	0.000	1.180	1.867

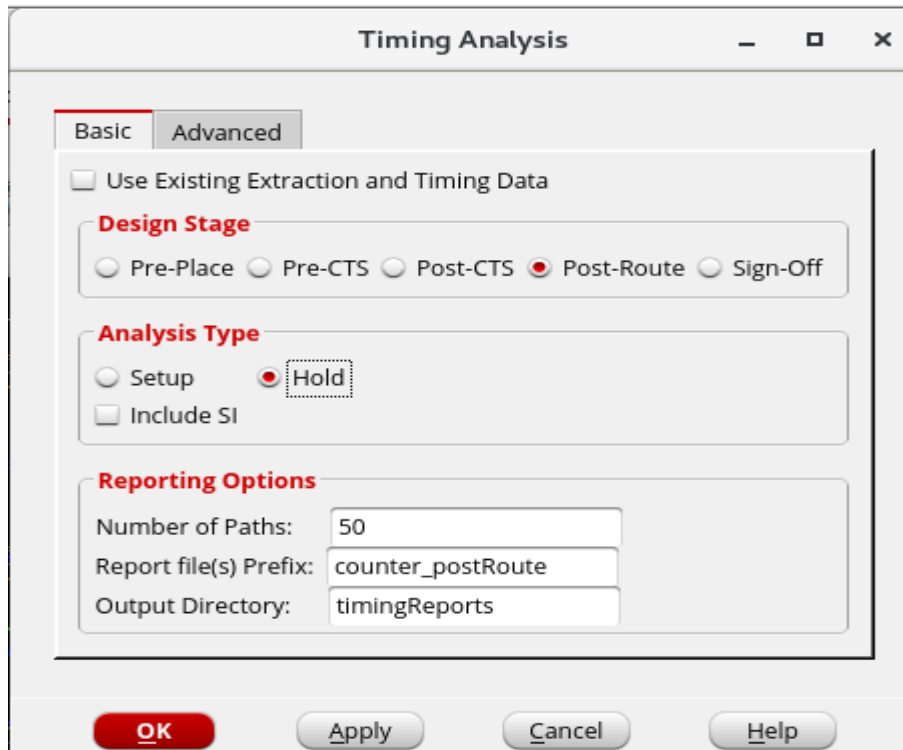
Clock Rise Edge 0.000
 = Beginpoint Arrival Time 0.000

Other End Path:

Pin	Edge	Net	Cell	Delay	Arrival Time	Required Time
-----	------	-----	------	-------	--------------	---------------

Hold Timing Analysis:

1. Select **Timing** → **Report Timing**
 2. In the Design Stage section, Select → **Post-Route**
- In the Analysis Type section, Select → **Hold** which is default and click **OK**.



The image shows the 'Timing Analysis' dialog box with the 'Basic' tab selected. The 'Design Stage' section has 'Post-Route' selected. The 'Analysis Type' section has 'Hold' selected. The 'Reporting Options' section shows 'Number of Paths' as 50, 'Report file(s) Prefix' as 'counter_postRoute', and 'Output Directory' as 'timingReports'. The 'OK' button is highlighted in red.

timeDesign Summary

Hold mode	all	reg2reg	default
WNS (ns):	0.148	0.148	0.000
TNS (ns):	0.000	0.000	0.000
Violating Paths:	0	0	0
All Paths:	4	4	0

Density: 67.901%

File → **counter_postRoute_all_hold.tarpt.gz** shows details of the top critical setup paths in the design.counter_postRoute_all_hold.tarpt.gz

Open
counter_postRoute_all_hold.tarpt
~/cache/fr-Ca8OGd
Save

```
# Design: counter
# Command: timeDesign -postRoute -hold -pathReports -slackReports -numPaths 50 -prefix counter_postRoute -outDir timingReports
#####
Path 1: MET Hold Check with Pin q_reg[2]/CK
Endpoint: q_reg[2]/D (v) checked with leading edge of 'clk'
Beginpoint: q_reg[2]/Q (^) triggered by leading edge of 'clk'
Path Groups: {reg2reg}
Analysis View: bc
Other End Arrival Time      0.000
+ Hold                      0.063
+ Phase Shift               0.000
- CPPR Adjustment           0.000
+ Uncertainty               0.010
= Required Time             0.073
Arrival Time                0.221
Slack Time                  0.148

Clock Rise Edge              0.000
= Beginpoint Arrival Time    0.000
Timing Path:
```

Pin	Edge	Net	Cell	Delay	Arrival Time	Required Time
clk	^	clk			0.000	-0.148
q_reg[2]/CK	^	clk	DFFQX4	0.000	0.000	-0.148
q_reg[2]/Q	^	q[2]	DFFQX4	0.195	0.195	0.047
g574_4296/A0	^	q[2]	OAI211X1	0.001	0.196	0.048
g574_4296/Y	v	n_19	OAI211X1	0.025	0.221	0.073
q_reg[2]/D	v	n_19	DFFQX4	0.000	0.221	0.073

```

Clock Rise Edge              0.000
= Beginpoint Arrival Time    0.000
Other End Path:
```

Pin	Edge	Net	Cell	Delay	Arrival Time	Required Time
clk	^	clk			0.000	0.148
q_reg[2]/CK	^	clk	DFFQX4	0.000	0.000	0.148

```

Path 2: MET Hold Check with Pin q_reg[1]/CK
Endpoint: q_reg[1]/D (v) checked with leading edge of 'clk'
Beginpoint: q_reg[1]/Q (^) triggered by leading edge of 'clk'
Path Groups: {reg1reg}
Analysis View: bc
Other End Arrival Time      0.000
+ Hold                      0.063
+ Phase Shift               0.000
- CPPR Adjustment           0.000
+ Uncertainty               0.010
= Required Time             0.073
Arrival Time                0.221
Slack Time                  0.148

Clock Rise Edge              0.000
= Beginpoint Arrival Time    0.000
Other End Path:
```


Area and Power Reports:

Area and Power Report Generation, use the following commands:

report_area

report_power

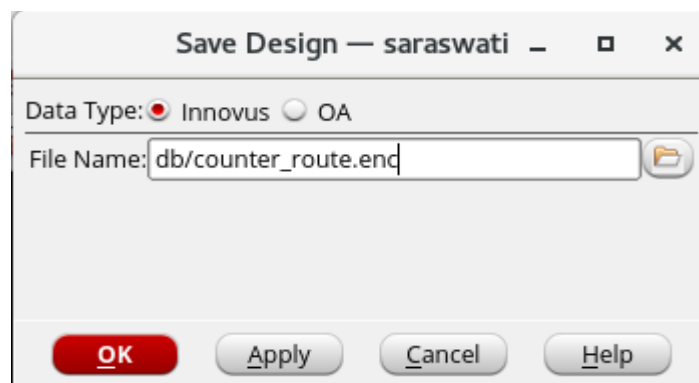
Design Optimization: (Only if Setup/Hold Voilation)

Enter the above shown command in the Terminal in order to run the Design Optimization first Post-Route.



The Report generation is same as shown prior to Design Optimization.

Save Database: File → Save Design → Data Type : Innovus → <DesignName>.enc → OK



Module 5.6 : Output File Generation (.spef, .sdf, .v& .gds)

Extracting RC Data:

The capacitance and resistance values for all the nets in the design are extracted by the Extract RC form.

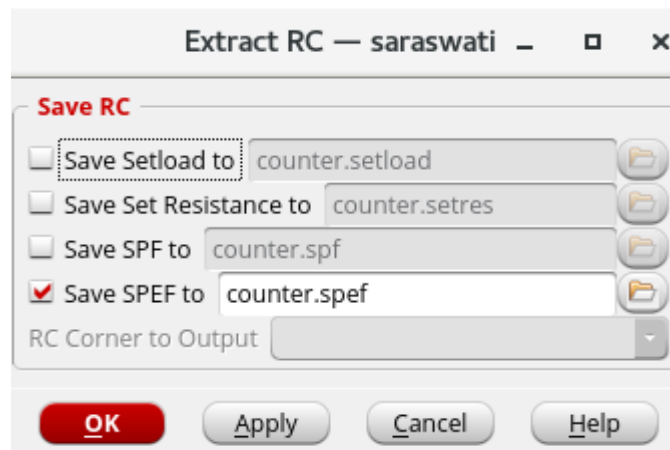
Extraction is run in pre-route mode prior to signal routing and in post-route mode after the signals are routed with NanoRoute. In post-route mode there are four effort levels to choose from (low, medium, high and signoff) which increase with accuracy at the expense of longer run-times.

The RC extraction mode can be changed by the **Tools → Set Mode → Specify RC Extraction Mode form**.

1. Select **Timing - Extract RC**

Select → **Save SPEF** to and specify file counter.spef

Select → **RC Corner to Output**: wc



2. Click **OK**.

If you review the log file you'll see extraction is run and then a SPEF file is written out. The equivalent text commands to run extraction and export SPEF are:

```
extractRC
```

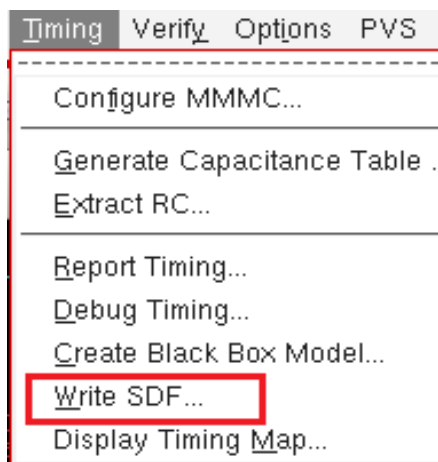
```
rcOut -spef counter.spef -rc_corner wc
```

Write the SDF:

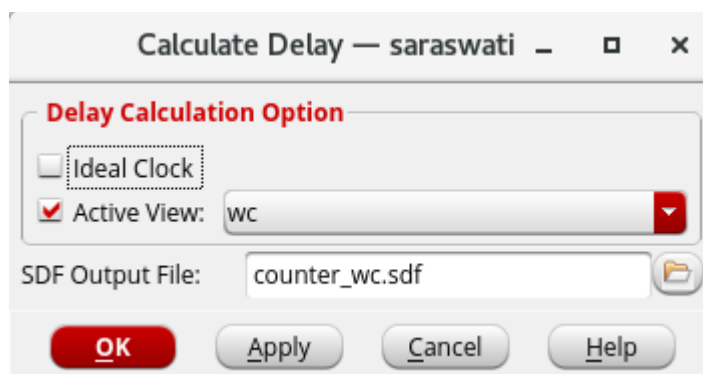
1. Select **Timing** → **Write SDF**

Select → **Save SDF** to and specify file counter_wc.sdf

Select → **Active View**: wc



you will get the below window

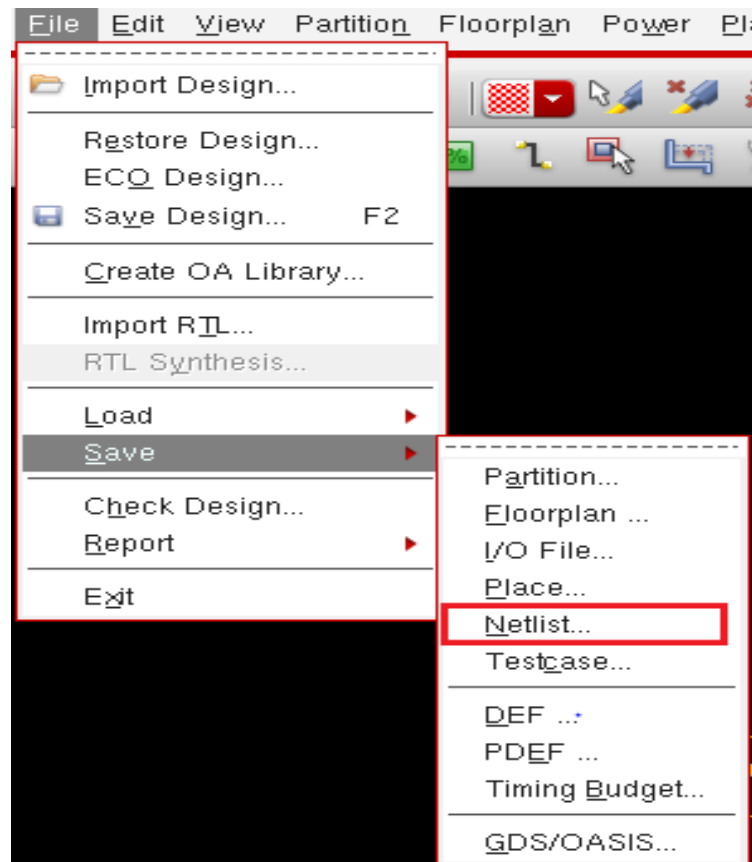


2. Click **OK**.

If you review the log file you'll see write_sdf is run and then a SDF file is written out. The equivalent text commands to export SDF are:

write_sdf -view wc counter_wc.sdf

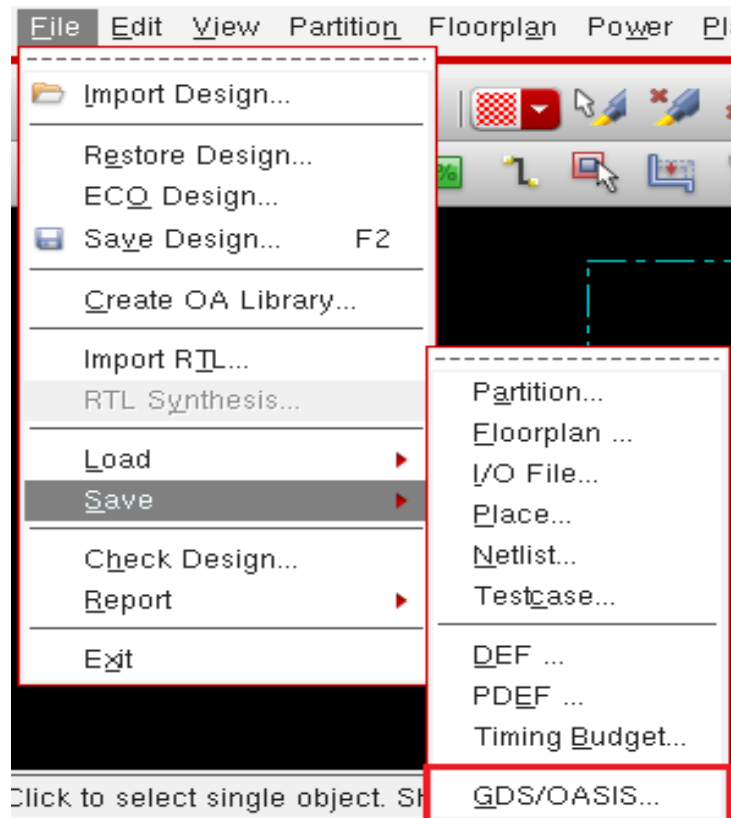
2. **Layout Netlist:** In Innovus Window choose File->save-> Netlist

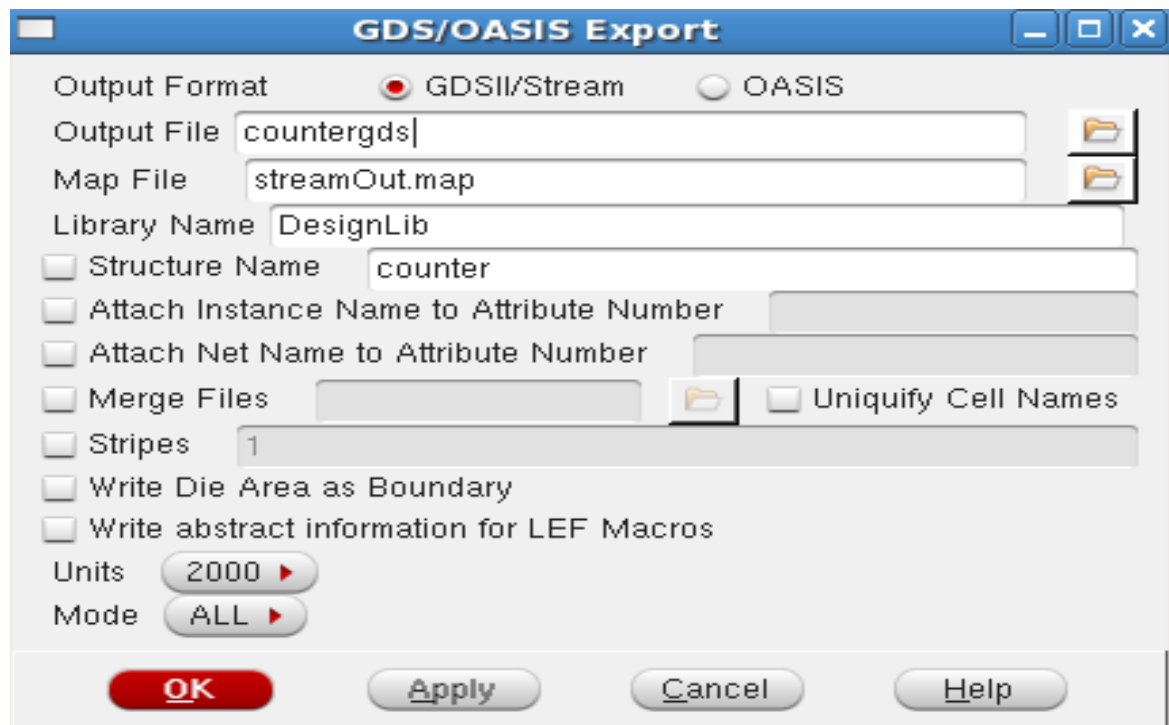


GDSII:

The GDS File is a Binary Format File which is not human readable and is fed to the Fabrication unit with data of various layers used depicted in terms of Geometrical Shapes.

3. Saving GDS => File → Save → GDS/OASIS → <FileName>.gds → OK





Give name to the Output File like Countergds and give ok. It will generate the GDSII file

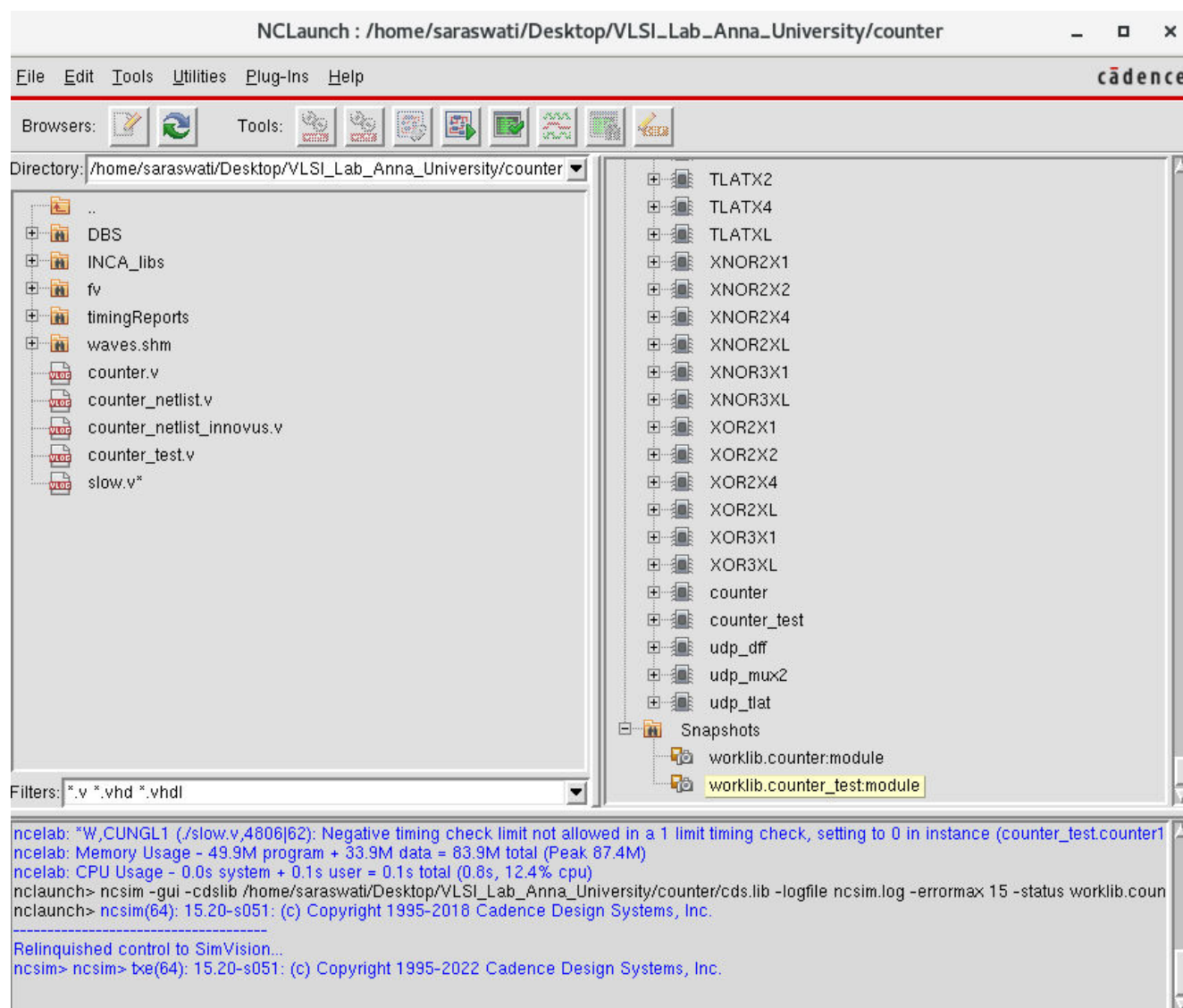
or

In Batch mode you can use the command given below in the terminal to generate GDS file

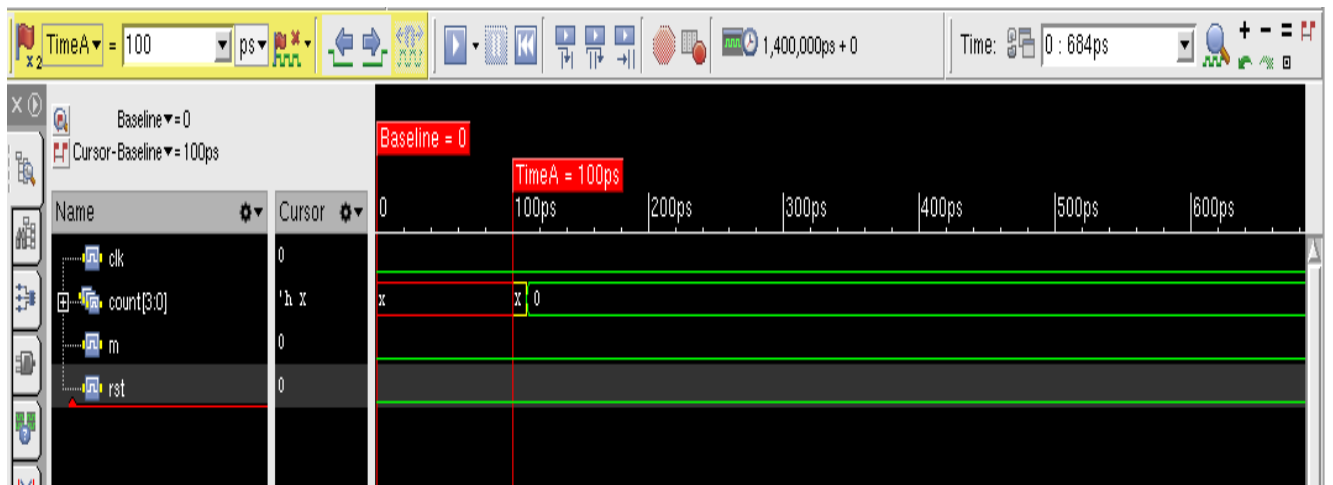
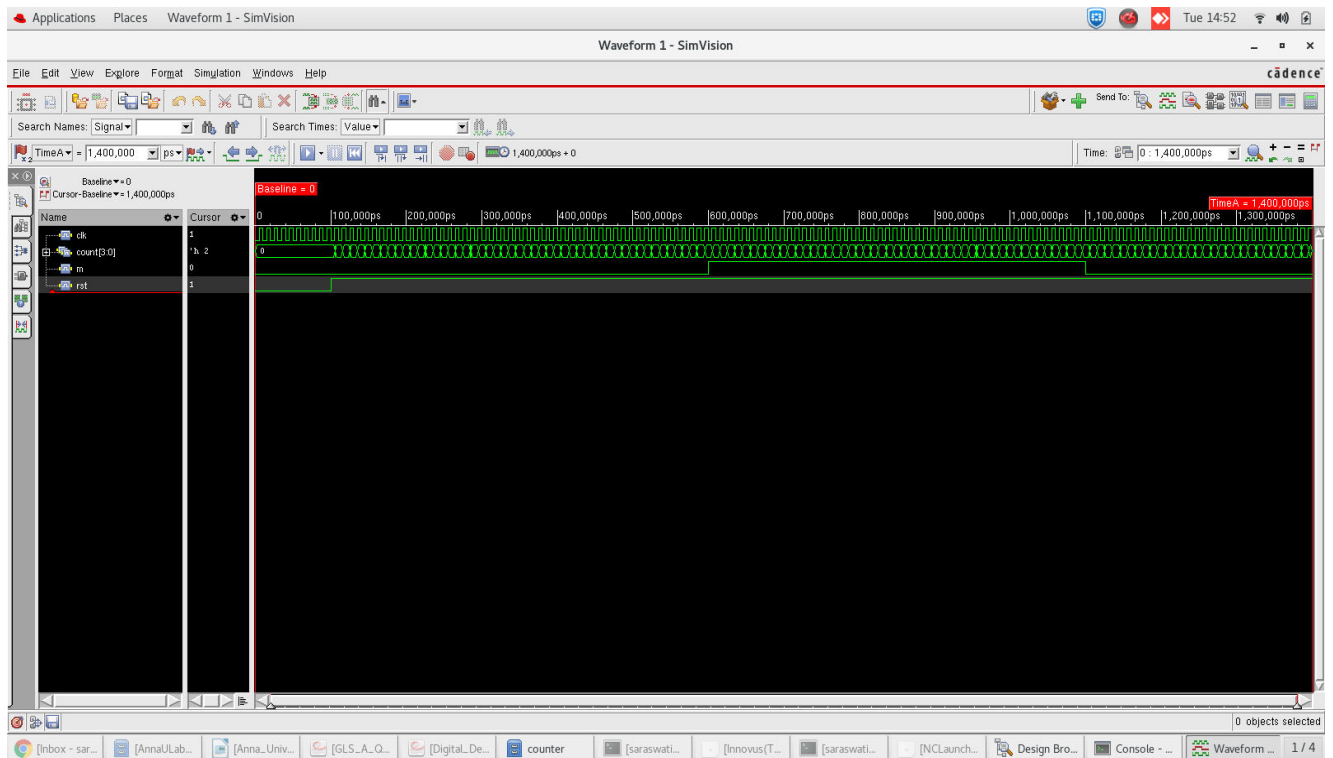
```
streamOut counter_GDS.gds -mapFile  
../../FOUNDRY/digital/90nm/dig/STREAM_MAP/90nmNew_streamOut.map -libName  
Mydesign -merge {FOUNDRY/digital/90nm/gds/*.gds} -units 2000 -mode ALL
```

Module 6: Post Layout Simulation/Gate Level Simulation

For simulation invoke the nclaunch and compile the netlist file generated after layout and elaborate it.



Simulate the test bench and see the wave form.



You can see the delay using this layout netlist with extra Zoom in option from the waveform window.


```
saraswati@saraswati:counter
File Edit View Search Terminal Help

[saraswati@saraswati counter]$ irun -clean counter_netlist_innovus.v counter_test.v -mess -access +rwc -v slow.v -timescal lns/lps -sdf_cmd_file counter.sdf -gui
irun(64): 15.20-s051: (c) Copyright 1995-2018 Cadence Design Systems, Inc.
irun: *N,CLEAN: Removing existing directory ./INCA_libs.
define worklib /home/saraswati/Desktop/VLSI_Lab_Anna_University/counter/INCA_libs/worklib
|
irun: *W,DLCPTH (./cds.lib,1): cds.lib Invalid path '/home/saraswati/Desktop/VLSI_Lab_Anna_University/counter/INCA_libs/worklib' (cds.lib command ignored).
define worklib /home/saraswati/Desktop/VLSI_Lab_Anna_University/counter/INCA_libs/worklib
|
irun: *W,DLCPTH (./cds.lib,1): cds.lib Invalid path '/home/saraswati/Desktop/VLSI_Lab_Anna_University/counter/INCA_libs/worklib' (cds.lib command ignored).
define worklib /home/saraswati/Desktop/VLSI_Lab_Anna_University/counter/INCA_libs/worklib
|
irun: *W,DLCPTH (./cds.lib,1): cds.lib Invalid path '/home/saraswati/Desktop/VLSI_Lab_Anna_University/counter/INCA_libs/worklib' (cds.lib command ignored).
define worklib /home/saraswati/Desktop/VLSI_Lab_Anna_University/counter/INCA_libs/worklib
|
irun: *W,DLCPTH (./cds.lib,1): cds.lib Invalid path '/home/saraswati/Desktop/VLSI_Lab_Anna_University/counter/INCA_libs/worklib' (cds.lib command ignored).
file: counter_netlist_innovus.v
  module worklib.counter:v
    errors: 0, warnings: 0
file: counter_test.v
  module worklib.counter_test:v
    errors: 0, warnings: 0
file: slow.v
  module slow.AND2X1:v
    errors: 0, warnings: 0
  module slow.AOI22X1:v
    errors: 0, warnings: 0
  module slow.CLKINVX1:v
    errors: 0, warnings: 0
  module slow.DFFRHQX2:v
    errors: 0, warnings: 0
  module slow.DFFRX2:v
    errors: 0, warnings: 0
  module slow.NAND2BX1:v
    errors: 0, warnings: 0
  module slow.NAND2XL:v
    errors: 0, warnings: 0
  module slow.NOR2XL:v
    errors: 0, warnings: 0
  module slow.OAI21X1:v
    errors: 0, warnings: 0
  module slow.OR2X1:v
    errors: 0, warnings: 0
  ...
  module slow.SDFFRHQX2:v
    errors: 0, warnings: 0
  primitive slow.udp_dff:v
    errors: 0, warnings: 0
  primitive slow.udp_mux2:v
    errors: 0, warnings: 0
    Caching library 'slow' ..... Done
    Caching library 'worklib' ..... Done
  Elaborating the design hierarchy:
  Top level design units:
    counter_test
  ncelab: *W,FLFIVN: Illegal Verilog name (count_reg[3])D ) in SDF annotation command file counter.sdf, line 69.
  ncelab: *W,FLFIVN: Illegal Verilog name (count_reg[0]) DELAY) in SDF annotation command file counter.sdf, line 210.
  ncelab: *W,FLFIVN: Illegal Verilog name (count_reg[2]) DELAY) in SDF annotation command file counter.sdf, line 231.
  ncelab: *W,FLFIVN: Illegal Verilog name (count_reg[1])0) in SDF annotation command file counter.sdf, line 254.
  ncelab: *W,FLFHSR: Missing separator (,;) in SDF annotation command file counter.sdf, line 274, trying to continue.
  ncelab: *W,FLFHSR: Unexpected separator (,;) in SDF annotation command file counter.sdf, line 1, trying to continue.
  ncelab: *W,FLFUKW: Unknown keyword "DELAYFILE" in SDF annotation command file counter.sdf, line 2.
  Srecovery(posedge RN, posedge CK &&& D == 1'b1, trceRMSCK, NOTIFIER);
  ncelab: *W,CUNGL1 (./slow.v,4806162): Negative timing check limit not allowed in a 1 limit timing check, setting to 0 in instance (counter_test.counter1.\count_reg[3] ).
  Building instance overlay tables: ..... Done
  Generating native compiled code:
    slow.DFFRHQX2:v <0x63f5ffag>
      streams: 0, words: 0
    slow.DFFRX2:v <0x38b62a99>
      streams: 0, words: 0
    slow.SDFFRHQX2:v <0x180202c0>
      streams: 0, words: 0
    worklib.counter_test:v <0x28d754c0>
      streams: 9, words: 6354
  Building instance specific data structures.
  Loading native compiled code: ..... Done
  Design hierarchy summary:
    Modules: 17 13
    UDPs: 6 2
    Primitives: 41 4
    Timing outputs: 10 0
    Registers: 7 7
    Scalar wires: 28 -
    Always blocks: 1 1
    Initial blocks: 4 4
```

Gate-Level Simulation can be either of the following three:

1. Zero Delay Simulation

Zero delay simulation means simulating the netlist without annotating any timing data. It is mainly meant for checking and validating the functionality of the design once it is translated into a gate-level netlist. In zero delay simulation mode, specify block delays are ignored.

These kinds of simulations are much faster than simulations with timing, and can be started earlier in the design cycle, when the netlist is not yet frozen, or you do not have timing information available.

Zero delay mode can be enabled using the `-nospecify` switch or by adding `-delay_mode Zero` in command line. This option works for Verilog designs only and disables timing information described in the specify blocks, such as module paths, and delays and timing checks.

The `xrun` command looks like the following:

```
irun -clean counter_netlist_innovus.v counter_test.v -v slow.v -mess -access +rwc -timescale 1ns/1ps -nospecify -gui
```

Running this command will give you the following messages:

```
file: counter_netlist_innovus.v
    module worklib.counter:v
        errors: 0, warnings: 0
file: counter_test.v
    module worklib.counter_test:v
        errors: 0, warnings: 0
file: slow.v
    module slow.AOI211X1:v
        errors: 0, warnings: 0
    module slow.AOI211XL:v
        errors: 0, warnings: 0
    module slow.CLKINVX1:v
        errors: 0, warnings: 0
    module slow.DFFQX4:v
        errors: 0, warnings: 0
    module slow.MXI2XL:v
        errors: 0, warnings: 0
    module slow.NAND2BX1:v
```

```

errors: 0, warnings: 0
module slow.NAND2XL:v
    errors: 0, warnings: 0
module slow.NOR2XL:v
    errors: 0, warnings: 0
module slow.NOR3X1:v
    errors: 0, warnings: 0
module slow.OAI211X1:v
    errors: 0, warnings: 0
module slow.OAI21X1:v
    errors: 0, warnings: 0
module slow.OAI2BB1XL:v
    errors: 0, warnings: 0
primitive slow.udp_dff:v
    errors: 0, warnings: 0
primitive slow.udp_mux2:v
    errors: 0, warnings: 0
    Caching library 'slow' ..... Done
    Caching library 'worklib' ..... Done
Elaborating the design hierarchy:
Top level design units:
    counter_test
Building instance overlay tables: ..... Done
Generating native compiled code:
    slow.DFFQX4:v <0x0f9b55db>
        streams: 0, words: 0
    worklib.counter_test:v <0x54634332>
        streams: 10, words: 5924
Building instance specific data structures.
Loading native compiled code: ..... Done
Design hierarchy summary:

```

	Instances	Unique
Modules:	27	14
UDPs:	7	2
Primitives:	53	3
Registers:	7	5
Scalar wires:	8	-

```

Always blocks:      3    3
Initial blocks:     3    3
Pseudo assignments: 3    3
Timing checks:      24   -
Simulation timescale: 1ps

```

Writing initial simulation snapshot: worklib.counter_test:v
waiting for SimVision to connect...

Relinquished control to SimVision...

Warning at line 2 of

/home/saraswati/Desktop/VLSI_Lab_Anna_University_2022/counter/INCA_libs/
irun.lnx8664.15.20.nc/cdsrun.lib for library worklib defined in
/home/saraswati/Desktop/VLSI_Lab_Anna_University_2022/counter/INCA_libs/
irun.lnx8664.15.20.nc/cdsrun.lib at line 2, redefines

library worklib defined in
/home/saraswati/Desktop/VLSI_Lab_Anna_University_2022/counter/cds.lib at line 1.

Insert UNDEFINE worklib

before DEFINE worklib on line 2

of /home/saraswati/Desktop/VLSI_Lab_Anna_University_2022/counter/INCA_libs/
irun.lnx8664.15.20.nc/cdsrun.lib

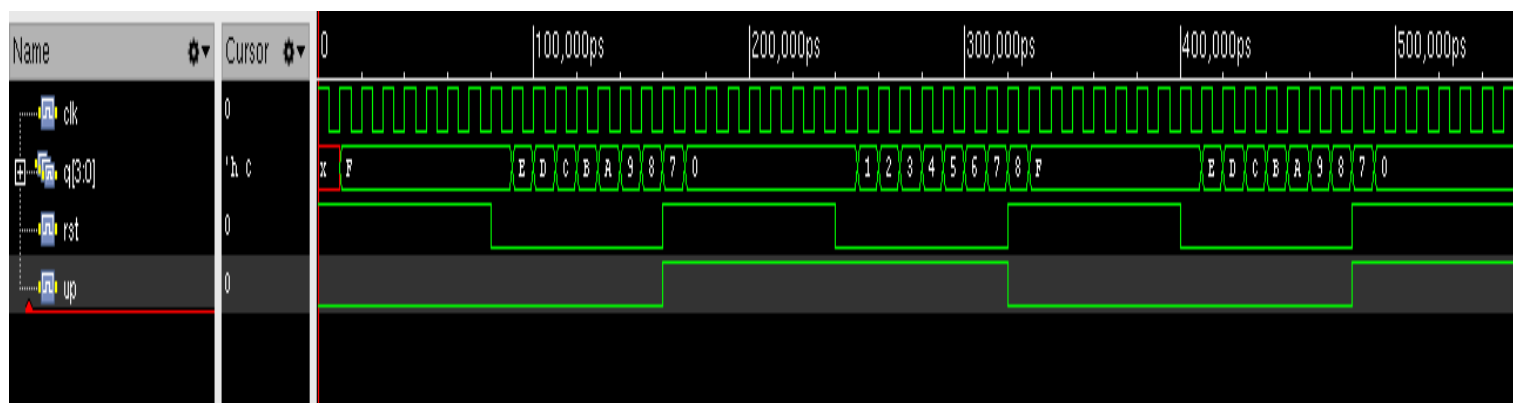
Or remove or comment out DEFINE worklib on line 1

of /home/saraswati/Desktop/VLSI_Lab_Anna_University_2022/counter/cds.lib
to suppress this warning message.:

Library redefined

ncsim>

ncsim> source /home/saraswati/install/INCISIVE152/tools/inca/files/ncsimrc



2. Unit Delay Simulation

Unit delay simulation operates on the assumption that all the elements in a circuit possess identical delay time. Hence, providing any random delay value is similar to adding a unit delay (1 time unit). This has the advantage that it is easier to implement than SDF simulation.

However, zero delay simulation can induce false race conditions and zero-delay loops. But you can run unit delay simulation to detect and resolve genuine race conditions and timing-sensitive loops.

In unit delay mode, simulator ignores all module path delay information and timing checks and converts all non-zero structural and continuous assignment delay expressions to a unit delay of one simulation time unit. There are two ways to specify the delay mode to unit delay mode:

- Use compiler directives in the source file to set delay modes specific to particular modules: ``delay_mode_unit`
- Use the command-line plus options at compile time to set a global delay mode for the simulation run: `+delay_mode_unit`

3. SDF Simulation (Simulation Delay File specifies timing)

SDF Simulation is the actual delay simulation where the delay specified in the SDF file is annotated into the design and all the timing checks are carried out. This type of simulation takes time and is much slower than zero delay or unit delay simulation.

In SDF Simulation, delays are annotated to the design, and different types of timing checks are performed. Delays can be annotated by the Verilog system function `$sdf_annotate()`, or with the `xrun` command line option `-sdf_cmd_file`. All delay annotation occurs at elaboration time. However, if it is required to annotate delays during simulation time, you have to use the `-sdf_sim_time` switch.

To run Gate level Simulation using SDF file for annotated delay, use the following command:

```
irun -clean counter_netlist_innovus.v counter_test.v -v slow.v -mess -access +rwc -timescale  
1ns/1ps -sdf_cmd_file counter_wc.sdf -gui
```

Running the command displays the following messages:

```
file: counter_netlist_innovus.v
    module worklib.counter:v
        errors: 0, warnings: 0
file: counter_test.v
    module worklib.counter_test:v
        errors: 0, warnings: 0
file: slow.v
    module slow.AOI211X1:v
        errors: 0, warnings: 0
    module slow.AOI211XL:v
        errors: 0, warnings: 0
    module slow.CLKINVX1:v
        errors: 0, warnings: 0
    module slow.DFFQX4:v
        errors: 0, warnings: 0
    module slow.MXI2XL:v
        errors: 0, warnings: 0
    module slow.NAND2BX1:v
        errors: 0, warnings: 0
    module slow.NAND2XL:v
        errors: 0, warnings: 0
    module slow.NOR2XL:v
        errors: 0, warnings: 0
    module slow.NOR3X1:v
        errors: 0, warnings: 0
    module slow.OAI211X1:v
        errors: 0, warnings: 0
    module slow.OAI21X1:v
        errors: 0, warnings: 0
    module slow.OAI2BB1XL:v
        errors: 0, warnings: 0
    primitive slow.udp_dff:v
        errors: 0, warnings: 0
    primitive slow.udp_mux2:v
        errors: 0, warnings: 0
```

```

Caching library 'slow' ..... Done
Caching library 'worklib' ..... Done
Elaborating the design hierarchy:
Top level design units:
    counter_test
ncelab: *W,FLFIVN: Illegal Verilog name (q_reg\[3\])D ) in SDF annotation command file
counter_wc.sdf, line 92.
ncelab: *W,FLFIVN: Illegal Verilog name (q_reg\[2\]) in SDF annotation command file
counter_wc.sdf, line 108.
ncelab: *W,FLFIVN: Illegal Verilog name (q_reg\[1\]) in SDF annotation command file
counter_wc.sdf, line 173.
ncelab: *W,FLFIVN: Illegal Verilog name (q_reg\[0\]) in SDF annotation command file
counter_wc.sdf, line 250.
ncelab: *W,FLFMSP: Missing separator (;) in SDF annotation command file counter_wc.sdf, line
399, trying to continue.
ncelab: *W,FLFUSP: Unexpected separator '(' in SDF annotation command file counter_wc.sdf, line
1, trying to continue.
ncelab: *W,FLFUKW: Unknown keyword "DELAYFILE" in SDF annotation command file
counter_wc.sdf, line 2.
Building instance overlay tables: ..... Done
Generating native compiled code:
    slow.DFFQX4:v <0x2c576f63>
        streams: 0, words: 0
    worklib.counter_test:v <0x16d52e68>
        streams: 10, words: 5924
Building instance specific data structures.
Loading native compiled code: ..... Done
Design hierarchy summary:

```

	Instances	Unique
Modules:	27	14
UDPs:	7	2
Primitives:	53	3
Timing outputs:	25	8
Registers:	7	5
Scalar wires:	33	-
Always blocks:	3	3
Initial blocks:	3	3

Pseudo assignments: 3 3
 Timing checks: 24 5
 Simulation timescale: 1ps

Writing initial simulation snapshot: worklib.counter_test:v

Relinquished control to SimVision...

Warning at line 2 of

/home/saraswati/Desktop/VLSI_Lab_Anna_University_2022/counter/INCA_libs/
 irun.lnx8664.15.20.nc/cdsrun.lib for library worklib defined in
 /home/saraswati/Desktop/VLSI_Lab_Anna_University_2022/counter/INCA_libs/
 irun.lnx8664.15.20.nc/cdsrun.lib at line 2, redefines

library worklib defined in
 /home/saraswati/Desktop/VLSI_Lab_Anna_University_2022/counter/cds.lib at line 1.
 Insert UNDEFINE worklib

before DEFINE worklib on line 2

of /home/saraswati/Desktop/VLSI_Lab_Anna_University_2022/counter/INCA_libs/
 irun.lnx8664.15.20.nc/cdsrun.lib

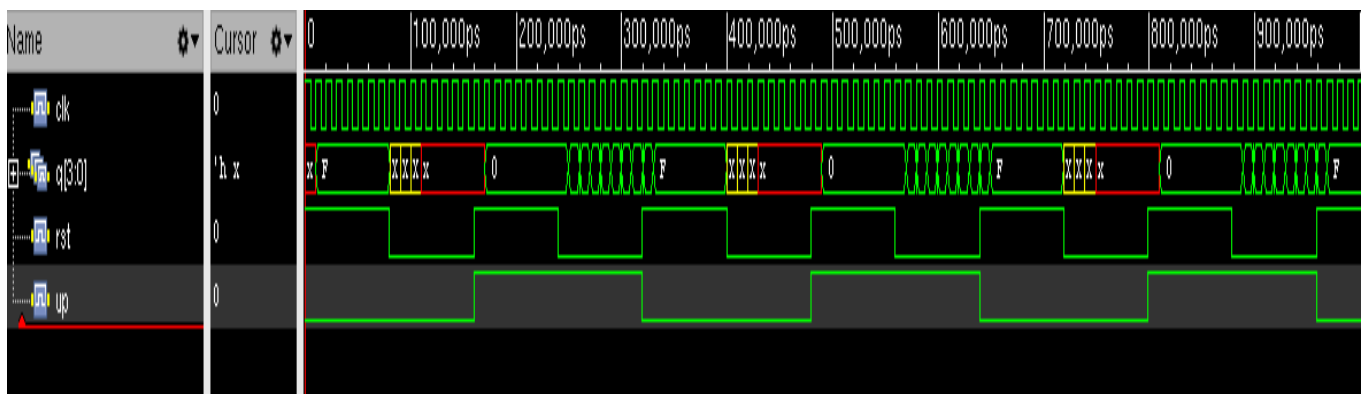
Or remove or comment out DEFINE worklib on line 1

of /home/saraswati/Desktop/VLSI_Lab_Anna_University_2022/counter/cds.lib
 to suppress this warning message.:

Library redefined

ncsim>

ncsim> source /home/saraswati/install/INCISIVE152/tools/inca/files/ncsimrc



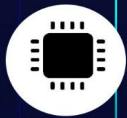
In this document, the counter example will be used to run zero delay and SDF Simulations.

In Zero delay mode, the delay in specify blocks and the SDF file will be ignored, and delays will not be annotated to the design. Zero delay mode can be activated by adding the `–nospecify` switch to the elaboration command line.

In SDF simulations, the delay information in the SDF file and library will be annotated to the design and the functionality will be an almost exact replica of what it will be in silicon.

Nurturing Excellence in VLSI

Entuple Training Domains



VLSI



Mechanical



Antenna design



Internet of
Things

- Career Builder Programs
- Internships
- Short Term Programs

Entuple Technologies Pvt. Ltd., # 2730, 'Trikanika', 80 Feet Road,
HAL 3rd Stage, Indiranagar, Bangalore - 560 038
+91 80 61222600 (30 Lines) | ask@entuple.com | training@entuple.com